

Relaxed Modular PCS from Arbitrary PCS and Applications to SNARKs for Integers

Alireza Shirzad¹, Sriram Sridhar²,
Dimitrios Papadopoulos³, Charalampos Papamanthou^{4,5}

¹University of Pennsylvania

²University of California, Berkeley

³Hong Kong University of Science and Technology

⁴Lagrange Labs*

⁵Yale University

*Work partially done while the first two authors were at Lagrange Labs

Abstract

Modular Polynomial Commitment Schemes (Mod-PCS) extend standard PCSs by enabling provable evaluation of integer polynomials modulo a random modulus, providing a natural foundation for SNARKs that operate directly over large integers without emulating arithmetic in finite fields. Only two Mod-PCS constructions are known. The first (Campanelli and Hall-Andersen, IACR ePrint 2024) serves primarily as a feasibility result and is impractical and not post-quantum secure due to its reliance on groups of unknown order. The second (Garett et al., CRYPTO 2025) introduces the weaker notion of *relaxed* Mod-PCS, but is not fully succinct: committing to a multilinear polynomial with N terms and B -bit coefficients requires $O(\sqrt{NB})$ proof size and verification time.

We present a black-box transformation that builds relaxed Mod-PCS from any standard PCS, enabling new constructions. Instantiating our transformation with a tensor-code PCS yields the first relaxed Mod-PCS with $O(\log(N + B))$ proof size and verifier time, which is transparent and plausibly post-quantum secure. Using this scheme within the framework of Garett et al., we obtain the first fully succinct SNARK for the Customizable Constraint System over $\mathbb{Z}_B$, achieving $O(B \log N + N \log N \log B)$ prover time and $O(\log(N + B))$ verifier time and proof size.

Our approach relies on a commitment-switching technique for integer polynomials and a new batched integer commitment scheme from any PCS. We further introduce improved arguments for integer addition and multiplication, correctness of the number-theoretic transform, and general Diophantine relations over committed integers.

Contents

1	Introduction	1
1.1	Our Contributions	3
1.2	Related Works	4
2	Techniques	6
2.1	Prelude	6
2.2	Warmup: A PCS with Known Modulus	7
2.3	Division and Multiplication Check	9
2.4	From PC over $\mathbb{F}_p$ to a relaxed mod-PC	11
3	Preliminaries	12
3.1	Polynomials	12
3.2	Idealized Oracle Relations and Proof Systems	13
3.3	NP Relations and Arguments Systems	17
3.4	Commitment Schemes	18
3.5	Generalized Polynomial Commitment Scheme	18
4	Toolbox: Integer commitment and arguments	20
4.1	Integer Commitment Schemes (ICS)	20
4.2	A Suite of Primitive Integer Arguments	21
4.3	Summary of the costs	30
5	Q̄aren: A framework for building relaxed mod-pc	30
5.1	Committing to an integer multilinear polynomial	31
5.2	A None-Adaptive Relaxed Mod-PC	32
5.3	Lifting to Adaptive Extractability	34
6	Acknowledgements	35
	References	35
A	PIOP Preliminaries	40
A.1	PIOR Product Composition	40
A.2	PIOR trivial extractor	41
A.3	From PIOP to Arguments of Knowledge	42
A.4	Deferred Proofs	43
A.5	PIOP Toolbox	45
B	Deferred proof of Theorem 5.4	49
C	Reductions (and Arguments) of Knowledge from Gen-PIOR + Gen-PCS	54

1 Introduction

Succinct non-interactive arguments of knowledge (SNARKs) are cryptographic proofs that enable an untrusted prover $\mathcal{P}$ to efficiently convince a computationally weak verifier $\mathcal{V}$ of claims of the form “Given a program P and public input x , I know a private input w such that $P(x, w) = 1$ ”.

Arithmetization. Most efficient SNARKs today are designed for NP-complete languages that operate over *finite-field* arithmetic. These include R1CS [Set20; CHMMVW20], CCS [STW23], GR1CS [DMS24], Plonkish systems [GWC19; CBBZ23], AIR [BCKL22], QAP, QSP [GGPR13], SSP [DFGK14], SAP [GM17], etc. Such languages are amenable to succinct and probabilistic checking and admit synthesis from higher-level computation models, such as circuit satisfiability and RAM programs.

Performance Blowups. While these languages are NP-complete and theoretically as expressive as general computation models, the compilation process often incurs substantial overhead when the underlying computation is not naturally expressed over finite fields. This overhead can range from moderate slowdowns—on the order of $2^5 \times$ for basic 32-bit CPU arithmetic—to extreme blowups exceeding $2^{12} \times$ for cryptographic computations such as RSA [CH24; Gar+25]. These costs are particularly pronounced for computations that inherently rely on non-finite-field arithmetic, including (i) integer computations, such as fixed-point arithmetic in real-time systems, digital signal processing, and machine learning; (ii) computations over integer rings, such as k -bit CPU arithmetic over $\mathbb{Z}/2^k\mathbb{Z}$, RSA operations over $\mathbb{Z}/N\mathbb{Z}$, and fully homomorphic computations; and (iii) computations over rational numbers or over fields that are non-native to the proving system.

Soundness Pitfalls. Beyond performance overhead, the arithmetization process is complex and error-prone, particularly for non-native computations such as fixed-width integer arithmetic, comparisons, and bit-level operations. These operations must be simulated using auxiliary constraints (e.g., range checks, bit decompositions, and carry relations), and subtle omissions can lead to under-constrained circuits which are the dominant vulnerability class in deployed ZK systems [CETWJL24; Pai+23]. In particular, missing range constraints or improperly enforced fixed-width arithmetic can introduce modular wraparound or aliasing behaviors that compromise soundness [Bes25; Net25b].

Ideally, we would like SNARKs that can operate directly over integers or rationals, without requiring expensive arithmetic *emulation* within finite fields.

Challenges of building SNARKs over Integers. Building SNARKs over the integers has been significantly less explored than their finite-field counterparts, largely due to the difficulty of constructing suitable underlying building blocks. These challenges include (a) the absence of multiplicative inverses in the integers, which complicates witness extraction; (b) the unbounded magnitude of integers, which hinders efficiency and succinctness; and (c) the scarcity of commitment schemes for integers. Most existing approaches rely on groups of unknown order (GUO), which are neither post-quantum secure nor practical at the moment.

Blueprint: Mod-PIOP + Mod-PC $\rightarrow$ SNARK. Following the widely used “PIOP+PC $\rightarrow$ SNARK” framework for constructing SNARKs over finite fields [GWC19; CHMMVW20], Campanelli et al. [CH24] recently proposed a blueprint for constructing SNARKs over integer arithmetic by combining two novel primitives: *modular polynomial interactive oracle proofs* (Mod-PIOPs) and *modular polynomial commitments* (mod-PCs). Mod-PIOPs extend the standard interactive oracle proof model by allowing polynomial oracles to answer queries modulo a randomly chosen modulus. Mod-PCs extend standard polynomial commitment schemes by enabling commitments to integer polynomials that can later be opened modulo a random modulus.

On the Difficulty of Constructing Mod-PCs. Constructing Mod-PIOPs is relatively straightforward, as existing finite-field PIOPs can often be adapted to the modular setting with only modest modifications [CH24;

Gar+25]. In contrast, building efficient mod-PCs, the core cryptographic component, remains a major technical challenge. Evidence for this gap is striking: while the literature contains numerous standard polynomial commitment schemes [KZG10; BFRW25; BMMS25; BCFRRZ25; EG25], there is, to the best of our knowledge, only a single mod-PC construction, due to Campanelli et al. [CH24]. Their scheme builds on the approach of Block et al. [BHRS21], extending DARK [BFS20] and operating over groups of unknown order (GUO). As a result, it is neither post-quantum secure nor practical in concrete settings. In particular, generating a single proof for moderately sized instances can take on the order of tens of hours [Net25a], and proof sizes are typically several megabytes. This undermines the primary motivation for using SNARKs over integers, namely, enabling *concretely* fast provers by reducing arithmetization overhead.

Relaxed Mod-PC. Subsequently, Garreta et al. [Gar+25] introduced a relaxed notion of mod-PC. This relaxation occurs along two dimensions. First, they fix the maximum bit length supported by the mod-PC (and, consequently, by the resulting SNARK); we denote this bound by $B \in \mathbb{N}$ throughout the paper, and write $\mathbb{Z}_B$ for the ring of integers of bit length at most B . Second, they relax the extractability guarantee to allow rational extractability. Concretely, in honest executions the primitive provides the same API as a mod-PC; however, against a malicious prover, the knowledge extractor is only guaranteed to recover a polynomial with rational—rather than integer—coefficients. We refer to this weaker notion as a relaxed mod-PC.

Garreta et al. further show that this relaxation remains useful in practice: the gap between rational and integer extractability can be compensated at the PIOP level using a lookup argument that enforces the integrality of the coefficients. While this approach enjoys several attractive properties—being transparent, hash-based, post-quantum secure, and supporting a fast prover—it suffers from two main drawbacks in terms of succinctness: 1. The reliance on a Brakedown-style argument [GLSTW23] leads to large proof sizes and high verification costs. In particular, for multilinear polynomials over a boolean hypercube of size $N = 2^\mu$, both the proof size and the verifier timescale as $O(\sqrt{N})$ asymptotically, and in practice exceed one megabyte in proof size and nearly one second in verification time. 2. Both the proof size and the verification time grow linearly with B , the maximum bit length of the integers. This contrasts with the mod-PC of Campanelli et al. [CH24], which is *fully succinct*, i.e., succinct both in the size of the polynomial and in the magnitude of its coefficients. This naturally leads to the following open question:

RQ1: Can we build a transparent, post-quantum secure, and fully succinct relaxed mod-PC?

Lack of general frameworks for relaxed mod-PCs. The landscape of standard polynomial commitment schemes is remarkably rich, spanning a wide range of cryptographic regimes, including elliptic-curve-based, lattice-based, and hash-based constructions. Each regime offers distinct trade-offs: elliptic-curve-based PCs typically achieve very small proof sizes [KZG10; BBBPWM18], lattice-based constructions provide the only post-quantum options with useful homomorphic properties [NS24; ZGX25; CMNW24], and hash-based schemes offer transparency and strong post-quantum security. Even within the hash-based setting, different code choices lead to different performance profiles—for example, Reed–Solomon–based constructions often yield smaller proofs and extremely fast verification [ACFY25], while RAA-code–based approaches typically enable faster provers [BCFRRZ25].

In contrast, the mod-PC landscape lacks this diversity. At present, there is a single integer-extractable construction in the groups-of-unknown-order setting [CH24] and a single rationally extractable construction in the hash-based (JEA code) regime [Net25a]. Achieving a comparable level of diversity would require designing a new mod-PC separately for each cryptographic regime, a task that demands substantial research effort. This raises a natural research question:

RQ2: Can we design a generic framework to bootstrap any polynomial commitment scheme into a mod-PC?

1.1 Our Contributions

We organize our contributions into main and additional contributions. The main contributions, presented in Section 1.1.1, focus on designing a family of SNARKs over integer arithmetic by addressing Research question RQ2 (and automatically resolving RQ1 by instantiating the framework with an appropriate PCS). These results are enabled by a collection of technical innovations, each of which is an additional contribution and of independent interest and is detailed in Section 1.1.2.

1.1.1 Main Contribution

In this work, we introduce a new family of SNARKs for integer arithmetic by proposing a general framework for constructing relaxed mod-PCs. Specifically, we present a bootstrapping compiler, $\mathbb{Q}\bar{a}ren^1$, which is the first generic transformation that turns any standard polynomial commitment scheme over $\mathbb{F}_p$ into a relaxed mod-PC. See Section 2.2 and Section 5 for informal and formal descriptions of the compiler, respectively.

Theorem 1.1 (informal). Let p be a sufficiently large prime, and PCS a multilinear polynomial commitment scheme over $\mathbb{F}_p$. Then there exists a relaxed mod-PC that uses PCS as a black box and inherits its transparency and security assumptions.

Corollary 1.2. Let p be a sufficiently large prime, and let PCS be a multilinear PCS over $\mathbb{F}_p$. Then there exists a SNARK for the Customizable Constraint System (CCS) language over $\mathbb{Z}_B$ (as defined in [Gar+25]), where $B \in \mathbb{N}$ denotes the maximum bit length of the witness vector. Moreover, the resulting SNARK inherits the transparency and security assumptions of the underlying PCS.

1.1.2 Additional Contributions

To achieve our main contribution—constructing a relaxed mod-PC—we reduce the task of *evaluating an integer polynomial with N coefficients of bit length at most B to committing to a constant number of large integers of size $\tilde{O}(NB)$ and proving simpler relations over them*, such as integer addition, multiplication, and Diophantine satisfiability. We now outline these contributions:

Batched Integer Commitment Scheme (ICS). We construct a commitment scheme for bounded-size integers by interpreting each integer as a multilinear polynomial and committing to it using a multilinear PCS over $\mathbb{F}_p$. We extend the construction to support batch commitment to t bounded integers. The resulting ICS inherits the commitment time (for an MLE of size tB), transparency and security assumptions of the underlying PCS. See Section 2.2 and Section 4.1 for informal and formal descriptions of the construction, respectively.

Then, we develop a suite of special-purpose sumcheck-based SNARKs for this family of integer commitment schemes. Leveraging the structure of our ICS, we design dedicated PIOPs over $\mathbb{F}_p$ and compile them into full-fledged arguments:

Integer Addition and Multiplication. We develop efficient succinct arguments (Sections 4.2.1 and 4.2.5) for proving addition and multiplication relations over committed *large* integers (of bit-length at least 2^{10}). Concretely, given commitments cm_a , cm_b , and cm_c to integers a , b , and c , the protocol proves that either $a + b = c$ or $a \cdot b = c$. Further, we extend these protocols to batched settings where we commit to $3t$ integers $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_B^t$ and prove either $\mathbf{a}[i] + \mathbf{b}[i] = \mathbf{c}[i]$ or $\mathbf{a}[i] \times \mathbf{b}[i] = \mathbf{c}[i]$ for $i = 0, \dots, t - 1$. Both of our protocols have $O(tB)$ prover time, $O(\log(tB))$ verifier time and proof size. See Section 2.3 and Section 4.2.5

¹A mythical and righteous Persian gladiator, son of Kāveh the Blacksmith.

for informal and formal descriptions of the multiplication protocol, respectively. Since the addition protocol is not directly used as a building block in the main construction, we only provide a formal description in Section 4.2.1.

NTT Check. Our integer multiplication argument requires proving consistency between a committed vector and its number theoretic transform (NTT). To this end, we develop a protocol (Section 4.2.4) based on the univariate Schwartz–Zippel lemma, even though our vectors are committed in multilinear form. To bridge this mismatch, we introduce an information-theoretic technique that transforms *any* multilinear PCS into a univariate PCS. This approach can be viewed as the conceptual inverse of the well-studied paradigm that compiles multilinear PCS from univariate PCS [ZSCZ25; GPS25; EG25; BCHO22], and may be of independent interest. See Section 2.3.1 and Section 4.2.4 for informal and formal descriptions of the protocol, respectively.

Arguments for Diophantine Equations. A more versatile building block developed in this work is a succinct argument for proving Diophantine relations [Lip03] over committed large (but bounded) integers. More specifically, given a batch commitment cm_a to integers $\mathbf{a} = [a_1, \dots, a_t]$, the protocol proves that these integers satisfy a relation of the form $G(a_1, \dots, a_t) = 0$, where $G \in \mathbb{Z}[X_1, \dots, X_t]$ represents an arithmetic circuit over integers of size $s \in \mathbb{N}$. Our protocol has $O(t B \log B)$ prover time and $O(t\lambda + \log Bt)$ proof size and verifier time. See Section 2.2.2 and Section 4.2.7 for informal and formal descriptions of the protocol, respectively.

Definitional contributions. We introduce a few new definitions that help us with the presentation and security analysis of our protocols.

- *Generalized Polynomial Interactive Oracle Reduction* (Gen-PIOR): a unifying definition that captures all of our idealized protocols. It extends the standard notion of PIOPs to PIORs—analogueous to how IORs generalize IOPs in [BMNW25]—and further generalizes both standard and modular oracle protocols, where oracle queries are answered modulo a random modulus. See Section 3.2.1 for a formal definition. We further extend the definition of round-by-round soundness (originally introduced in [Can+19]) and knowledge soundness to the Gen-PIOR setting in Definitions 3.7 and 3.8.
- *Generalized Polynomial Commitment Scheme* (Gen-PCS): a generalization of the standard notion of polynomial commitment schemes that captures both standard and modular commitment schemes as special cases. See Section 3.5 for a formal definition.

1.2 Related Works

1.2.1 SNARKs over the integers

A growing line of work studies proof systems for computations over the integers and more general rings. Ganesh et al. [GNS23] introduced Rinocchio, the first SNARK for arithmetic circuits over rings; however, the scheme relies on new “linear-only” assumptions, requires a circuit-specific trusted setup, and is not succinct in the bit-length of ring elements. More recently, [WZD25; HMZ25] proposed SNARKs for statements over Galois rings via polynomial commitment schemes defined over such rings. While these constructions are hash-based, post-quantum secure, and offer fast provers, they incur $\sqrt{N}$ -size proofs and verification time, making them impractical for many applications. The closest works to ours, as discussed earlier, are those of Campanelli et al. [CH24] and Garreta et al. [Gar+25]; we provide a detailed comparison in Table 1.

Scheme	Properties				Prover	Verifier	Proof
	PQ	IE	Tr	Assumpt.			
Campanelli et al. [CH24]	✗	✓	✓	GUO	$O(NB \text{ poly}(\lambda, \log B))$	$O(\text{poly}(\lambda, \log B, \mu))$	$O(\text{poly}(\lambda, \log B, \mu))$
Zip [Gar+25]	✓	✗	✓	RO	$N(B + \lambda) + \lambda\sqrt{N}B + \lambda^2\sqrt{N}\log N$	$\sqrt{N}B + \lambda^2\sqrt{N} + \lambda^2\log N$	$\sqrt{N}(B + \lambda) + \lambda^2\log N$
$\mathcal{Q}\bar{a}ren_{TS}$ Instantiated with [BFRW25]	✓	✓	✓	RO	$B \log N + N \log B \log N$	$\log(B + N)$	$\log(B + N)$
$\mathcal{Q}\bar{a}ren_{MC}$ Instantiated with [EG25]	✗	✓	✗	Q-DLOG	$B \log N + N \log B \log N$	$\log(B + N)$	$\log(B + N)$

PQ: Post-Quantum Security, IE: Integer Extractability, Tr: Transparent

N : The size of the multilinear polynomial

B : The maximum bit-length of the coefficients of the polynomial

Table 1: Comparison of *all* mod-pc schemes

1.2.2 Arguments for large Integer Addition and Multiplication

Proving arithmetic relations over large committed integers has received substantial attention in the literature, with constructions based on groups of unknown order (GUO) [CPP17; DF02; Lip03] as well as lattice-based techniques [LLNW18; LNS20; KSSL21]. Such arguments serve as fundamental building blocks for a wide range of cryptographic applications, including range proofs, order relations, set non-membership proofs, and zero-knowledge lists, which in turn underpin systems such as anonymous credentials [CL01], privacy-preserving certificate transparency [EMBB17], and secure electronic voting [Gro05]. We compare our protocols with the state of the art in Table 2.

1.2.3 Arguments for Diophantine equations over the integers

Proving Diophantine equations over committed integers was first studied in early works such as [FO97; DF02; Lip03], which relied on groups of unknown order (GUOs). More recently, Towa and Vergnaud [TV20] gave the first succinct argument for satisfiability of Diophantine equations with proof size logarithmic in the size of the polynomial. The work most closely related to ours is the PoKEMath protocol of [BMSS25], where the proof size, verifier time, and communication grow logarithmically in the magnitude of the solution, while the verifier time scales linearly in the size of the equation and the proof size scales linearly in the number of variables. However, PoKEMath also relies on GUOs, limiting practicality and post-quantum security. Our protocol, Masdak, supports only bounded-size integers, which restricts generality, but can be viewed as a practical version of PoKEMath with similar asymptotic efficiency, while being instantiable from standard polynomial commitment schemes and therefore amenable to post-quantum.

Scheme	Properties				Int Addition			Int Multiplication		
	PQ	Tr	ZK	Assumpt.	$T(\mathcal{P})$	$T(\mathcal{V})$	$ \pi $	$T(\mathcal{P})$	$T(\mathcal{V})$	$ \pi $
Libert et al. [LLNW18]	✓	✓	✓	SIVP	tB	tB	tB	$tB^{\log_2 3}$	$tB^{\log_2 3}$	$tB^{\log_2 3}$
Lyubashevsky et al. [LNS20]	✓	✓	✓	R-SIS R-LWE	tB	tB	tB	$tB \log B$	tB	tB
Kuchta et al. [KSSL21]	✓	✓	✓	M-SIS M-LWE	tB	tB	tB	$tB \log B$	$tB \log B$	$tB \log B$
This work (TS)	✓	✓	✗	RO	tB	$\log tB$	$\log tB$	$tB \log B$	$\log tB$	$\log tB$
This work (MC)	✗	✗	✗	Q-DLOG	tB	$\log tB$	$\log tB$	$tB \log B$	$\log tB$	$\log tB$

PQ: Is Post-Quantum Secure, ZK: Is Zero-Knowledge, Tr: Is Transparent
 t : The number of addition/multiplication instances being proved
 B : The maximum bit-length of the coefficients of the polynomial

Table 2: Comparison of Integer Addition and Multiplication Arguments

2 Techniques

We now outline the techniques underlying our contributions, making several simplifying assumptions for clarity of exposition. We explicitly indicate whenever such a simplification is made.

2.1 Prelude

Before presenting our contributions, we briefly review SNARKs over the integers and summarize the key techniques and concepts drawn from prior work. As in Zinc [Gar+25], our goal is to construct a SNARK for the Customizable Constraint System (CCS) [STW23] over bounded integers $\mathbb{Z}_B$, where $B \in \mathbb{N}$ denotes the bit-length bound.

Mod-PIOP + Mod-PC $\rightarrow$ SNARK over Integers. We obtain our SNARK by instantiating the framework of Zinc [Gar+25], which builds upon the framework introduced by Campanelli et al. [CH24]. At a high level, this framework compiles an information-theoretic component—called a Modular Interactive Oracle Proof (Mod-PIOP)—with a cryptographic component that we refer to as a relaxed Modular Polynomial Commitment (mod-PC). In a (multivariate) Mod-PIOP, in the commitment phase, the prover $\mathcal{P}$ first sends oracles to multilinear polynomials defined over $\mathbb{Z}_B$ to the verifier. Then, in the querying phase, the verifier $\mathcal{V}$ samples a random prime $q \in \mathbb{N}$ and queries these oracles only modulo q . Finally, in the decommitment phase, the verifier checks the validity of the proof using the answers to its queries. We adopt the same PIOP as Zinc, consisting of a PIOP for CCS augmented with a lookup PIOP.

Mod-PC. To realize an idealized Mod-PIOP as a concrete argument, oracle access to polynomials over $\mathbb{Z}_B$ is instantiated using a mod-PC. Concretely, in the commitment phase, the prover sends *commitments* to μ -variate multilinear polynomials defined over $\mathbb{Z}_B$. Later in evaluation phase, The prover can open these commitments at verifier-chosen random points *modulo a randomly sampled prime* q , providing a proof π that the revealed value z satisfies $z \equiv p(\mathbf{y}) \pmod{q}$.

Scheme	Properties					Prover	Verifier	Proof
	PQ	ZK	Tr	∞	Assumpt.			
Towa et al. [TV20]	✗	✓	✓	✓	GUO^*	$sB \log B$	$B + \log s + \lambda$	$B + \log s + \lambda$
PoKEMath Bünz et al. [BMSS25]	✓	✗	✓	✓	GUO^*	$tB \log B$	$\lambda(t + s \log \lambda)$	$t\lambda$
$\mathcal{Q}\bar{a}ren_{TS}$ Instantiated with [BFRW25]	✗	✗	✓	✗	RO	$tB \log B$	$t\lambda + \log tB + \lambda \log \lambda$	$t\lambda + \log tB$
$\mathcal{Q}\bar{a}ren_{MC}$ Instantiated with [EG25]	✗	✗	✗	✗	Q-DLOG	$tB \log B$	$t\lambda + \log tB + \lambda \log \lambda$	$t\lambda + \log tB$

PQ: Is Post-Quantum Secure, ZK: Is Zero-Knowledge

∞ : Supports unbounded integers, Tr: Is Transparent

t : The number of addition/multiplication instances being proved

B : The maximum bit-length of the coefficients of the polynomial

Table 3: Comparison of arguments for Diophantine equations

Observe that this querying/opening step mirrors that of a standard PCS over $\mathbb{F}_q$, where evaluations are verified modulo a fixed prime q . The key difference is that, in our setting, the prime is sampled during the query phase, *after* the polynomial has already been committed. This “commit-then-choose- q ” requirement is therefore incompatible with many mainstream PCS constructions, such as those based on elliptic curves [KZG10; BBBPWM18].

Why Field-Agnostic PCSs Are Not Enough. There exists a class of PCSs known as *field-agnostic*, which can operate over arbitrary fields [BFKTWZ24; GLSTW23; SB25; AGLMS23; BFS20]. While these schemes offer considerable flexibility, they still require the underlying field to be fixed before the commitment is created. In contrast, in a mod-PC the verifier samples the prime field $\mathbb{F}_q$ only after the commitments are produced, making the construction of a mod-PC strictly more challenging.

Relaxed mod-PC vs. Mod-PC. As in *Zinc*, which introduces *Zip*, we focus on constructing a *relaxed* variant of a mod-PC. In an honest-prover execution, a relaxed mod-PC behaves identically to a standard mod-PC; the distinction arises only in the extractability guarantees against malicious provers. In a mod-PC, the extractor is guaranteed to recover polynomials with *integer* coefficients (or evaluations), whereas in a relaxed mod-PC the extractor is only guaranteed to recover polynomials with *rational* coefficients (or evaluations).

2.2 Warmup: A PCS with Known Modulus

Before presenting our framework for constructing a relaxed mod-PC, we begin with a simplified version that yields a standard PCS in the setting where the prime q is known in advance. In Section 2.4, we then lift this construction to obtain a relaxed mod-PC. For clarity of exposition, we focus in this technical overview on commitments to univariate polynomials $f \in \mathbb{Z}_B^{\leq N}[X]$; the framework extends to multilinear polynomials in a straightforward manner, as described in Section 5.

Before committing to integer-valued polynomials, we first construct a much simpler primitive: a commitment scheme for a single bounded integer.

An Integer Commitment Scheme. To commit to an integer $a \in \mathbb{Z}_B$, assuming $B = 2^\beta$, we first decompose it into its bit representation $\mathbf{a} = [a_0, a_1, \dots, a_{B-1}]$, where $a = \sum_{i=0}^{B-1} a_i 2^i$. We then interpret the vector $\mathbf{a}$ as the evaluation vector of a β -variate multilinear polynomial $\tilde{a}$ and commit to a by committing to $\tilde{a}$ using a multilinear PCS over $\mathbb{F}_p$. We denote the resulting integer commitment by cm_a . Notably, this ICS is generic with respect to the choice of the underlying multilinear PCS over $\mathbb{F}_p$, and more importantly, p is completely independent of q and can be different. Also, we extend this ICS to a batched setting which allows us to commit to multiple integers simultaneously (see Section 4.1 for more details). Later, we will see that this choice of integer commitment allows us to treat the verifier as having oracle access to the binary representation of the integer, enabling the reuse and construction of sumcheck-based PIOPs.

We now describe how to commit to an integer-valued polynomial using the integer commitment primitive introduced above. These techniques are also used in prior work [AGLMS23; BMSS25].

PCS Commitment Phase. We commit to a univariate polynomial $f \in \mathbb{Z}_B^{\leq N}[X]$ as follows:

1. We evaluate the polynomial at a publicly known, sufficiently large integer point $\alpha \in \mathbb{Z}$ such that $\alpha \gg B$. This can be viewed as an *encoding* or *Injective map* of the polynomial f into the single very large integer value $f(\alpha) \in \mathbb{Z}$.
2. We then commit to the integer value $f(\alpha)$ using the integer commitment scheme described above.

The binding of the PCS follows from the binding of the integer commitment: if a commitment could be opened to two polynomials $f, g \in \mathbb{Z}_B^{\leq N}[X]$, then we would have $f(\alpha) = g(\alpha)$ over the integers, which is impossible due to the injectivity of the encoding map $f \mapsto f(\alpha)$ for sufficiently large α .

Evaluation Phase. For simplicity, we focus on a non-adaptive evaluation point, i.e., $z \in \mathbb{Z}_B$ sampled uniformly at random, which suffices for our SNARK applications. In Section 5, we extend the construction to adaptive evaluation via a simple repetition of the protocol. To prove that $f(z) = y \bmod q$, we begin with the following observation: Defining $g \in \mathbb{Z}_B^{\leq N}[X]$ as $g(X) := 1 + zX + z^2X^2 + \dots + z^{N-1}X^{N-1}$, and its reverse polynomial $g^{\text{rev}}(X) \in \mathbb{Z}_B^{\leq N}[X]$ as $g^{\text{rev}}(X) := z^{N-1} + z^{N-2}X + \dots + X^{N-1}$. Observe that multiplying the integer encodings of f and g^{rev} yields a highly structured result:

$$\begin{aligned} f(\alpha) \cdot g^{\text{rev}}(\alpha) &= \left(\sum_{i=0}^{N-1} f_i \alpha^i \right) \left(\sum_{i=0}^{N-1} g_{N-1-i} \alpha^i \right) \\ &= \left(\sum_{i=0}^{N-2} c_i \alpha^i \right) + \langle \mathbf{f}, \mathbf{g} \rangle \cdot \alpha^{N-1} + \left(\sum_{i=N}^{2N-2} c_i \alpha^i \right) \end{aligned}$$

which shows that the coefficient of α^{N-1} is the inner product of the coefficient vectors of f and g which is $\langle \mathbf{f}, \mathbf{g} \rangle = f(z) = y$, hence

$$f(\alpha) \cdot g^{\text{rev}}(\alpha) = l + m \cdot \alpha^{N-1} + r \tag{1}$$

We denote by l , m , and r the left, middle, and right parts of the above equation, respectively. The protocol then proceeds by having the prover convince the verifier that (a) Eq. (1) holds over the integers, i.e., the prover knows integers l, m, r satisfying the equation, (b) y is the remainder of m modulo q , i.e., the prover knows an integer h such that $m = y + h \cdot q$, (c) l, m , and r are bounded integers. Since integers l, m , and r satisfying Eq. (1) are large, the prover cannot directly send them to the verifier. Instead, the prover commits to these constant number of integers using the ICS described before (see Section 4.1 for more details) and then proves the required relations as follows: First of all, statement (c) corresponds to a range proof over committed integers, which we will explain next:

2.2.1 Range Proof

The goal of the range proof is to show that a committed integer $a \in \mathbb{Z}_B$, with $B = 2^\beta$, satisfies $a < M$. As we will see in Section 5, we deliberately choose the bound to be a power of two, i.e., $M = 2^m$, which makes the range proof particularly simple. For non-power-of-two bounds, see Section 4.2.2 for a more general construction.

For a power-of-two bound $M = 2^m$, it suffices to verify that the binary decomposition of a , which we denote by $\mathbf{a} \in \{0, 1\}^B$, has its $B - M$ most significant bits equal to zero. By the choice of the ICS, the verifier has oracle access to $\tilde{a}$, the multilinear extension of $\mathbf{a}$. The verifier samples $\mathbf{r} \in \mathbb{F}_q^m$ and queries $\tilde{a}(0^{\beta-m} \parallel \mathbf{r})$. If the result is zero, the verifier accepts; otherwise, it rejects.

Moreover, the verifier must ensure that the provided oracle correspond to a valid binary decomposition, i.e., that its values are boolean. To this end, the prover and verifier run an instance of the standard multilinear ZeroCheck (see Appendix A.5.3) on the polynomial $\tilde{a}(1 - \tilde{a})$.

Continuing with the argument for Eq. (1), Statements (a) and (b) are purely algebraic and require the prover to demonstrate knowledge of integers satisfying certain Diophantine equations, which is more complex than a range proof. To handle this, we introduce a new tool called *MaSDAK*², described next.

2.2.2 MaSDAK

Magnitude Succinct Diophantine Argument of Knowledge (MaSDAK, see the detailed description of the protocol in Section 4.2.7) proves knowledge of large integers $u_1, \dots, u_t \in \mathbb{Z}_B$ (where this B is specific to MaSDAK) satisfying a Diophantine equation $f(u_1, \dots, u_t) = 0$ for an integer polynomial f . The core idea of MaSDAK is to verify the relation modulo a random prime rather than directly over the integers. Concretely, the verifier samples a random λ -bit prime ℓ and asks the prover for the remainders $r_1, \dots, r_t \in \mathbb{Z}_\ell$ of $u_1, \dots, u_t$. The verifier then checks that $f(r_1, \dots, r_t) \equiv 0 \pmod{\ell}$.

It remains to prove consistency between $r_1, \dots, r_t$ and $u_1, \dots, u_t$, i.e., that there exist integers $h_1, \dots, h_t \in \mathbb{Z}_B$ such that $u_i = r_i + h_i \cdot \ell$ for all $i \in [t]$. We introduce a tool called *Division Check* to prove this efficiently.

2.3 Division and Multiplication Check

Before describing the Division Check, we first present a key building block: an argument for integer multiplication. We will later show that a simple modification of this multiplication argument yields the Division Check.

2.3.1 Integer Multiplication Argument

Here, the prover aims to convince the verifier that the committed integers $a, b, c \in \mathbb{Z}_B$ satisfy the multiplication relation $c = a \cdot b$. Recall that, by our ICS design, the verifier receives commitments to the multilinear polynomials $\tilde{a}, \tilde{b}, \tilde{c}$ corresponding to the bit decompositions $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \{0, 1\}^B$ of a, b, c , respectively. For simplicity, we treat these commitments as oracles and refer to openings as oracle queries.

We follow a similar approach to [LNS20; KSSL21] and use the Number Theoretic Transformation (NTT) of $\mathbf{a}, \mathbf{b}, \mathbf{c}$ to handle multiplication. Intuitively, the NTT maps integer multiplication to a Hadamard (component-wise) product over an NTT-friendly subgroup of the field. Thus enforcing

$$\text{NTT}(\mathbf{a}) \circ \text{NTT}(\mathbf{b}) = \text{NTT}(\mathbf{c})$$

²named after Mazdak (d. c. 524 CE), a Persian reformer associated with ideals of justice and equity

captures the relation $c = a \cdot b$, provided there is no overflow. Hence, the multiplication argument starts by $\mathcal{P}$ sending oracles for a', b', c' to $\mathcal{V}$, which are the NTTs of a, b, c , respectively. It then suffices for $\mathcal{V}$ to verify:

- **Booleanity:** a, b, c are bit vectors, using the same Booleanity check described in the aforementioned range proof (Section 2.2.1);
- **Overflow:** the multiplication incurs no overflow, which reduces to checking that a and b have fewer than $B/2$ bits, following the approach outlined in the aforementioned range proof (Section 2.2.1);
- **Hadamard:** $a' \circ b' = c'$, which the verifier checks by sampling a random $r \in \mathbb{F}_p^\beta$ and verifying that $\tilde{a}(r) \cdot \tilde{b}(r) = \tilde{c}(r)$;
- **NTT Check:** a', b' , and c' are the NTTs of a, b , and c , respectively. This step requires more subtle techniques, described next (See the details of the protocol in Section 4.2.4).

By the definition of the NTT, for an integer $m \in \{a, b, c\}$ with binary decomposition vector $\mathbf{m}$, The NTT vector $\mathbf{m}' = \text{NTT}(\mathbf{m})$ corresponds to evaluations of the *univariate* polynomial $\widehat{m}(X) = m_0 + m_1X + \dots + m_{B-1}X^{B-1}$ over the NTT domain; i.e. the set $\mathbb{H} := \{\omega^0, \omega^1, \dots, \omega^{B-1}\}$, where ω is a primitive B -th root of unity in $\mathbb{F}_p$. Then we have $\mathbf{m}' = [\widehat{m}(\omega^0), \widehat{m}(\omega^1), \dots, \widehat{m}(\omega^{B-1})]$.

Thus, $\mathbf{m}$ and $\mathbf{m}'$ represent the same polynomial $\widehat{m}$ in two different bases: the monomial basis and the Lagrange basis over the NTT domain. In particular, $\sum_{i=0}^{B-1} m_i X^i = \sum_{i=0}^{B-1} m'_i L_i(X) = \widehat{m}(X)$, where $L_i(X)$ is the i -th Lagrange basis polynomial corresponding to the NTT evaluation points. To verify this equality, the verifier applies the univariate Schwartz–Zippel lemma (see Lemma 3.1): it samples a random point $r \in \mathbb{F}_p$ and checks $\sum_{i=0}^{B-1} m_i r^i = \sum_{i=0}^{B-1} m'_i L_i(r)$. Equivalently, the prover must establish the following over multilinear polynomials:

$$\sum_{\mathbf{x} \in \{0,1\}^{\log B}} \widetilde{m}(\mathbf{x}) \cdot \widetilde{\text{mon}}_{\mathbf{r}}(\mathbf{x}) = \sum_{\mathbf{x} \in \{0,1\}^{\log B}} \widetilde{m}'(\mathbf{x}) \cdot \widetilde{\text{Lag}}_{\mathbf{r}}(\mathbf{x}) \quad (2)$$

where $\widetilde{\text{mon}}$ and $\widetilde{\text{Lag}}$ are the multilinear extensions of $\mathbf{mon} := [1, r, r^2, \dots, r^{B-1}]$ and $\mathbf{Lag} := [L_0(r), L_1(r), \dots, L_{B-1}(r)]$. Equation Eq. (2) is an instance of the standard multivariate sumcheck problem (see Appendix A.5.4). In the final round of sumcheck, the verifier must evaluate both sides of Eq. (2) at a random point; to keep the verifier succinct (i.e., sublinear in B), the relevant polynomials must either be given as oracles (via commitments) or be succinctly evaluable.

In our setting, both $\widetilde{m}$ and $\widetilde{m}'$ are committed, but $\widetilde{\text{mon}}$ and $\widetilde{\text{Lag}}$ are not. We show in Section 4.2.4 that $\widetilde{\text{mon}}$ can be evaluated succinctly by the verifier. Since $\widetilde{\text{Lag}}$ cannot be computed succinctly, the prover provides a fresh commitment to $\widetilde{\text{Lag}}$ and proves its well-formedness via the Lagrange check described next (see the detailed protocol in Section 4.2.3).

Remark 2.1. One can see the above protocols as the reverse of the techniques used in [BCHO22; ZSCZ25] where the goal is to prove the evaluation of a multilinear polynomial (either in coefficient form or evaluation form) by invoking evaluation protocols on univariate polynomials. Here, the goal is to prove the evaluation of univariate polynomials (either in coefficient form or evaluation form) by invoking evaluation protocols on multilinear polynomials and can be of independent interest.

Lagrange-Check. Given oracle access to $\widetilde{\text{Lag}}$, the verifier must ensure that it is the MLE of $\mathbf{Lag} = [L_0(r), L_1(r), \dots, L_{B-1}(r)]$ for the public point r . By the barycentric form of Lagrange polynomials, for $i \in \{0, 1, \dots, B-1\}$ we have $L_i(X) = (\omega^i Z_{\mathbb{H}}(X)) / (B(X - \omega^i))$, which implies for a fixed r that $B \cdot L_i(r)(r - \omega^i) = \omega^i Z_{\mathbb{H}}(r)$. Equivalently, we have

$$B \cdot \mathbf{Lag}[i](r - \mathbf{mon}_{\omega}[i]) = \mathbf{mon}_{\omega}[i] Z_{\mathbb{H}}(r)$$

for all $i \in \{0, 1, \dots, B-1\}$. This naturally yields the following one-shot verification: The verifier samples a random $\mathbf{s} \in \mathbb{F}_p^\beta$ and checks

$$f(\mathbf{s}) = N \tilde{v}(\mathbf{s}) (r - \widetilde{\text{mon}}_\omega(\mathbf{s})) - Z_{\mathbb{H}}(\omega) \widetilde{\text{mon}}_\omega(\mathbf{s}).$$

Note that $\widetilde{\text{mon}}_\omega$ is succinctly evaluatable by the verifier, and $Z_{\mathbb{H}}(\omega) = \omega^B - 1$ can also be computed in $O(\log B)$ time [GWC19].

Division Check. The Division Check verifies the relation $u = \ell h + r$, where ℓ is a λ -bit prime. As before, we transform the binary decomposition vectors into their NTT representations and perform the required addition and multiplication checks. This is due to the fact that similar to the multiplication check, the addition of two integers can also be verified by checking the element-wise addition of their NTT representations. More generally, We have $u = \ell h + r$ if and only if

$$\text{NTT}(\mathbf{u}) = \text{NTT}(\ell) \circ \text{NTT}(\mathbf{h}) + \text{NTT}(\mathbf{r})$$

The key difference is that ℓ and r are given to the verifier in the clear rather than as commitments. However, since both are succinct (λ -bit integers), the verifier can compute their NTTs in $O(\lambda \log \lambda)$ time and remain succinct.

Conclusion of the Warmup. By combining the above techniques, we can construct a standard field-agnostic PCS over $\mathbb{F}_q$ where the prime $q \in \mathbb{N}$ is known beforehand. But, it's still not clear how to extend this construction to a relaxed mod-PC where q is only known during evaluation. We address this in the next section using the commitment switching technique.

2.4 From PC over $\mathbb{F}_p$ to a relaxed mod-PC

At a first glance, it may seem that the prime q have not been used in anywhere at the time of commitment, so the above construction should already yield a relaxed mod-PC. However, this is not the case.

Recall (as described in Section 2.2) that to commit to a polynomial $f \in \mathbb{Z}_B^{\leq N}[X]$, the prover computes the integer $\hat{f}(\alpha)$ for a sufficiently large public integer α . However, α cannot be chosen arbitrarily – it must also satisfy $q \mid \alpha$. This is crucial during evaluation, where the prover shows that $\hat{f}(z) = y \pmod{q}$ by proving the existence of an integer h such that $\hat{f}(z) = y + h \cdot q \pmod{\alpha}$. As shown in DewTwo [BMSS25], to conclude $\hat{f}(z) = y \pmod{q}$, it is necessary that $\alpha \equiv 0 \pmod{q}$. To ensure both this and the largeness property of α , DewTwo [BMSS25] sets $\alpha = q^{7\mu}$.

In our setting, however, the modulus q is not known at commitment time, so α cannot be fixed in advance and the commitment cannot be computed directly. This constitutes our main technical challenge, which we address using the *Commitment Switching* technique.

Commitment Switching. To address the above roadblock, we propose a technique called commitment switching. This concept is closely related to mechanisms used in several modern PC schemes: it appears implicitly in KZH [KZHB25], and explicitly via code switching in Blaze [BCFRRZ25] and FICS [BMMS25].

The rough idea of commitment switching is to first commit to a polynomial f under a certain commitment scheme (called the source commitment scheme) and get $\text{cm}_{\text{source}}$, and later commit to the same polynomial under a different commitment scheme (called the target commitment scheme) to get $\text{cm}_{\text{target}}$, then argue that both $\text{cm}_{\text{source}}$ and $\text{cm}_{\text{target}}$ correspond to the same polynomial f . There are different reasons to perform commitment switching in different schemes, but in our case, the reason is to first commit to the polynomial f using α_{source} that only satisfies the “largeness” requirement and not the divisibility requirement, because q is not known yet; Then, after receiving the challenge q , switch to a commitment α_{target} that

satisfies both “largeness” and “divisibility” requirements. Hence, in the commitment phase, the prover sends $\text{cm}_{\text{source}} = \text{ICS.Commit}(f(\alpha_{\text{source}}))$. Then, after knowing q at the time of evaluation, the prover sends a *new* commitment with $\alpha_{\text{target}} = q\alpha_{\text{source}}$ that satisfies both the largeness and divisibility requirements. Then we continue with proving the evaluation of the polynomial $f \bmod q$ using $\text{cm}_{\text{target}}$.

Now it remains to prove that both commitments $\text{cm}_{\text{source}}$ and $\text{cm}_{\text{target}}$ correspond to the same polynomial f . To do this, we also run the evaluation protocol for $\text{cm}_{\text{source}}$; although we know that it doesn’t guarantee anything about the evaluation mod q , it still guarantees the evaluation mod α_{source} . In fact, during the extraction, we extract different polynomials f_{source} and f_{target} from each commitment separately, such that

$$f_{\text{source}}(z) = y \bmod \alpha_{\text{source}} \quad \text{and} \quad f_{\text{target}}(z) = y \bmod \alpha_{\text{target}}$$

since $\alpha_{\text{target}} = q\alpha_{\text{source}}$, we have that $f_{\text{target}}(z) = y \bmod \alpha_{\text{source}}$. Hence, $f_{\text{source}}(z) = f_{\text{target}}(z) \bmod \alpha_{\text{source}}$. Since f_{source} and f_{target} were chosen before z is known, by the Schwartz-Zippel lemma (either assuming that α_{source} only has large prime factors or by using the multilinear Schwarz-Zippel lemma for composite moduli [BF23]), we have that with high probability over the choice of z ,

$$f_{\text{source}}(X) = f_{\text{target}}(X) \bmod \alpha_{\text{source}} \tag{3}$$

Note that although the above argument shows that the two polynomials are equal mod α_{source} , the prover could still cheat by adding multiples of α_{source} to the coefficients of f_{target} . We show that this is impossible because during the extraction process, we get the guarantee that the coefficients of f_{target} are bounded rationals, hence Eq. (3) can be lifted to an equality over the rationals: $f_{\text{source}}(X) = f_{\text{target}}(X)$ over the rationals. This concludes the argument for commitment switching and the construction of a relaxed mod-PC.

3 Preliminaries

In this section, we review the basic definitions, notations, and tools that will be used throughout the paper.

Notation. Throughout the paper, we adopt the following notational conventions: ring (and field) elements and indices are denoted by lowercase letters (e.g., a, b, c). Vectors are denoted with lowercase bold letters (e.g., $\mathbf{a} \in \mathbb{Z}^n$), while matrices are denoted by uppercase bold letters (e.g., $\mathbf{A}, \mathbf{B} \in \mathbb{Z}^{m \times n}$).

Indexing and Ranges. For indexing, we write $\mathbf{x}[i]$ to denote the i -th entry of a vector $\mathbf{x}$, and $\mathbf{A}[i, j]$ to denote the entry in the i -th row and j -th column of a matrix $\mathbf{A}$. Unless stated otherwise, indices are assumed to start from 0. For any integer $a \geq 1$, we define $[a] := \{1, 2, \dots, a\}$ and $[a) := \{0, 1, \dots, a-1\}$.

Algebra. We denote by $\mathbb{R}$ an arbitrary ring; by $\mathbb{Z}$ the set of integers; by $\mathbb{Z}_B$ the set of B -bit positive integers. We further denote by $\mathbb{Q}$ the set of rational numbers, and by $\mathbb{Q}_B := \{a/b \mid a, b \in \mathbb{Z}_B, b \neq 0\}$ the set of bounded rational numbers. Finally, we denote by $\mathbb{F}_p$ the finite field of prime order p , and by $\mathbb{F}_p^\times$ the multiplicative subgroup of non-zero elements of $\mathbb{F}_p$. Throughout this paper, all finite fields are prime fields. For an integer $z \in \mathbb{Z}_B$, we denote by $\text{Bits}(z) \in \{0, 1\}^B$ the big-endian bit-decomposition of z ; i.e. MSB bits appear first. Likewise, for a vector $\mathbf{z} \in \{0, 1\}^B$, we denote by $\text{Int}(\mathbf{z}) \in \mathbb{Z}_B$ the integer whose big-endian bit-decomposition is $\mathbf{z}$.

3.1 Polynomials

We write $\mathbb{R}_\mu^{\text{tot} \leq d}$ (resp., $\mathbb{R}_\mu^{\text{ind} \leq d}$) for the set of μ -variate polynomials over $\mathbb{R}$ of total (resp., individual) degree $\leq d$, and $\mathbb{R}^{\leq d}$ for the set of univariate polynomials over $\mathbb{F}_p$ of degree $\leq d$.

Multilinear Extension (MLE). Let $N = 2^\mu$ and $\mathbf{u} \in \mathbb{F}_p^N$. We denote by $\tilde{u} \in \mathbb{F}_{p,\mu}^{\text{ind} \leq 1}$ the multilinear extension (MLE) of $\mathbf{u}$ defined as follows:

$$\tilde{u}(X_1, \dots, X_\mu) := \sum_{\mathbf{b} \in \{0,1\}^\mu} \mathbf{u}[\text{Int}(\mathbf{b})] \cdot \text{eq}_{\mathbf{b}}(X_1, \dots, X_\mu)$$

Lemma 3.1 (Schwartz–Zippel [Sch80; Zip79]). Let $\mathbb{F}$ be a field, let $S \subseteq \mathbb{F}$ be a finite set, and let $f \in \mathbb{F}_{p,\mu}^{\text{tot} \leq d}$ be a nonzero polynomial (for any $\mu \geq 1$). If $\mathbf{r} = (r_1, \dots, r_\mu)$ is sampled uniformly at random from S^μ , then

$$\Pr_{\mathbf{r} \leftarrow S^\mu} [f(\mathbf{r}) = 0] \leq \frac{d}{|S|}.$$

In particular, for $\mu = 1$ (the univariate case), this specializes to $\Pr_{r \leftarrow S} [f(r) = 0] \leq d/|S|$.

Number Theoretic Transform (NTT). The Number Theoretic Transform (NTT) of a vector $\mathbf{u} \in \mathbb{F}_p^n$ with respect to a smooth multiplicative subgroup $\mathbb{H} \subseteq \mathbb{F}_p^\times$ (defined above) evaluates the polynomial defined by the coefficient vector $\mathbf{u}$ over the points of $\mathbb{H} = \{\omega^i\}_{i=0}^{n-1}$. Concretely, letting $u(X) = \sum_{i=0}^{n-1} \mathbf{u}[i]X^i$, the NTT output is $\text{NTT}(\mathbf{u}) := [u(\omega^0), \dots, u(\omega^{n-1})]$. The NTT over $\mathbb{H}$ can be computed in $O(n \log n)$.

3.2 Idealized Oracle Relations and Proof Systems

In this section, we introduce the idealized algebraic objects and proof systems used throughout the paper. The relationships among these objects are summarized in Fig. 1. Since our focus is exclusively on polynomials, all oracles and relations considered are polynomial oracles and polynomial oracle relations.

Polynomial Oracle. Let $\mathbb{R}$ be the ambient ring, $\mathbb{S}$ be evaluation ring, and $\varphi : \mathbb{R} \rightarrow \mathbb{S}$ be an evaluation map. We denote by $\llbracket f \rrbracket_\varphi$ an oracle to $f \in \mathbb{R}_\mu^{\text{ind} \leq d}$ that on input $\mathbf{x} \in \mathbb{R}^\mu$ outputs $\varphi(f(\mathbf{x}))$. Whenever $\mathbb{R} = \mathbb{S}$ and φ is the identity map, we omit the subscript and simply write $\llbracket f \rrbracket$.

Notation. For ease of presentation, we use **colored text** to denote index-related components that are present in the general definition but typically omitted in the relations considered in this paper.

Definition 3.2 (Indexed Polynomial Oracle Relations). An **indexed** polynomial oracle relation $\mathfrak{R}$ is a set of triples $(\mathbf{i}, \mathbf{x}, \mathbf{y}, \mathbf{w})$, where **$\mathbf{i}$ is the index**, $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ is the implicit instance which includes a set of polynomial oracles, and $\mathbf{w}$ is the witness.

Definition 3.3 (Cartesian Product of Relations). Consider ternary relations $\mathfrak{R}_1$ and $\mathfrak{R}_2$ over instance, oracle-instance, witness tuples. We define

$$\mathfrak{R}_1 \times \mathfrak{R}_2 = \left\{ ((\mathbf{i}_1, \mathbf{i}_2), (\mathbf{x}_1, \mathbf{x}_2), (\mathbf{y}_1, \mathbf{y}_2), (\mathbf{w}_1, \mathbf{w}_2)) \mid \begin{array}{l} (\mathbf{i}_1, \mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1) \in \mathfrak{R}_1 \\ (\mathbf{i}_2, \mathbf{x}_2, \mathbf{y}_2, \mathbf{w}_2) \in \mathfrak{R}_2 \end{array} \right\}.$$

We let $\mathfrak{R}^\ell$ denote $\mathfrak{R} \times \dots \times \mathfrak{R}$ for ℓ times.

3.2.1 Generalized Polynomial Interactive Oracle Reduction

Here, we define Generalized Polynomial Interactive Oracle Reductions (Gen-PIORs). Our definition is inspired by the interactive oracle reductions (IORs) of Bünz et al. [BMNW25], and extends that framework in one key dimension while restricting it in another. Specifically: (a) We generalize IORs by allowing the ambient ring of a polynomial oracle to differ from its evaluation ring, capturing settings in which polynomials

are defined over one ring but evaluated over another via an evaluation map. (b) We restrict attention to polynomial oracles and polynomial oracle relations, rather than arbitrary oracles and relations. This is a deliberate scoping choice—tailored to the needs of our work—rather than a technical limitation of the framework.

Definition 3.4 (Generalized Polynomial Interactive Oracle Reduction (Gen-PIOR)). Let $\mathfrak{R}$ and $\mathfrak{R}'$ be indexed oracle relations. A (public-coin, holographic) interactive oracle reduction from $\mathfrak{R}$ to $\mathfrak{R}'$ is a tuple of the form $\text{Gen-PIOR} = (\text{Algs}, \text{Eval-Conf}, \text{Metrics})$ defined as follows:

- **Algs** consists of a tuple of polynomial time algorithms $(\mathcal{I}, \mathcal{P}, \mathcal{V})$ participating in the protocol, namely an indexer $\mathcal{I}$, a prover $\mathcal{P}$, and a verifier $\mathcal{V}$. At the beginning of the protocol, the indexer $\mathcal{I}$ receives as input an index i (for a relation $\mathfrak{R}$) and outputs a short index ι , an implicit index l , and a new index i' (for a relation $\mathfrak{R}'$). The prover $\mathcal{P}$ and verifier $\mathcal{V}$ are interactive algorithms. The prover receives as input the index i , an explicit instance x , an implicit instance y , and a witness w , while the verifier receives as input the short index ι , the explicit instance x , oracle access to the implicit index l , and oracle access to the implicit instance y . The prover and verifier engage in (potentially multiple) rounds of interaction. At the end of the protocol, the prover outputs a new witness w' , and the verifier outputs a new explicit instance x' and a new implicit instance y' .
- **Eval-Conf** is a tuple $(\mathbb{R}, \mathbb{S}, \Phi)$ defining the evaluation configuration of the oracles in the protocol, where $\mathbb{R}$ denotes the ambient ring, $\mathbb{S}$ denotes the evaluation ring, and Φ is a set of evaluation mappings of the form $\varphi : \mathbb{R} \rightarrow \mathbb{S}$.
- **Metrics** is a tuple $(k, n_\Pi, n_\varrho, s, v)$ defining the efficiency metrics of the protocol, where k denotes the number of rounds, $n_\Pi : [k] \rightarrow \mathbb{N}$ specifies the number of polynomials sent by the prover in each round, $n_\varrho : [k] \rightarrow \mathbb{N}$ specifies the number of challenges sent by the verifier in each round, $s : \mathbb{N} \rightarrow \mathbb{N}$ specifies the number of oracle queries made by the verifier, and $v : [k] \times \mathbb{N} \rightarrow \mathbb{N}$ specifies the number of variables of the polynomials sent by the prover in each round.

We describe the indexing phase in more detail below.

Indexer in the 0-th round. The indexer $\mathcal{I}$ runs on input i and outputs a short index ι , an implicit index l , and a new index i' . Concretely, it computes the round-0 oracles to multilinear polynomials

$$\Pi_0 = \{p_{0,j}\}_{j=1}^{n_\Pi(0)} \subseteq \mathbb{R}_{v(0,j)}^{\text{ind} \leq 1},$$

and encodes them as an oracle description in the implicit index l . The short index ι contains any public auxiliary data needed by the verifier. The indexer outputs (ι, l, i') .

We describe the prover-verifier interaction in more detail below.

$\mathcal{P}$ in i -th round. The prover $\mathcal{P}$ sends a collection of polynomial oracles $\Pi_i \in \{p_{i,j}\}_{j=1}^{n_\Pi(i)}$. Each polynomial $p_{i,j} \in \mathbb{R}_{v(i,j)}^{\text{ind} \leq 1}$ is a multilinear polynomial over $\mathbb{R}$ in $v(i, j)$ variables.

$\mathcal{V}$ in i -th round. $\mathcal{V}$ receives evaluation oracle access to polynomials in Π_i , then replies with a collection of uniformly random challenges $\varrho_i \in \{\rho_{i,j}\}_{j=1}^{n_\varrho(i)}$. Each challenge $\rho_{i,j} \in \mathbb{R}$ is a ring element in $\mathbb{R}$.

Verifier phases. Without loss of generality, the verifier is split into two phases. In the interaction phase, it samples challenges and sends them to the prover. In the query phase, first it samples an evaluation map $\varphi \in \Phi$, then it makes s queries to the polynomial oracles in the implicit index, implicit instance, and prover's messages. All oracle queries are evaluated via evaluation map $\varphi : \mathbb{R} \rightarrow \mathbb{S}$ where $\mathbb{S}$ is the evaluation ring. The verifier's output is a deterministic function of its input and the transcript, which we denote

$$(x', y') := \mathcal{V}^{\Pi, l, y}(\iota, x, \varrho)$$

Now, we define the security properties of Gen-PIORs:

Definition 3.5 (Completeness). Gen-PIOR is complete if the following holds. For any $(i, x, y, w) \in \mathfrak{R}$,

$$\Pr \left[(i', x', y', w') \in \mathfrak{R}' \mid \begin{array}{l} (\iota, l, i') \leftarrow \mathcal{I}(i) \\ (w', (x', y')) \leftarrow \langle \mathcal{P}(i, x, y, w), \mathcal{V}^{l, y}(\iota, x) \rangle \end{array} \right] = 1 .$$

We define the notion of round-by-round soundness and knowledge-soundness for our PIORs, which are stronger than the standard notions of soundness and knowledge-soundness, and imply soundness and knowledge-soundness in the non-interactive setting after the Fiat–Shamir transformation [Can+19; CY24]. We first define the state function for PIORs as follows:

Definition 3.6 (State function for PIORs). A *state function* for Gen-PIOR is a function State for which the following holds:

- *Empty transcript*: $\text{State}(i, x, y, \emptyset) = 0$ unconditionally, where $\emptyset$ is the empty transcript.
- *Prover moves*: If $\text{State}(i, x, y, \text{tr}) = 0$ for a partial transcript $\text{tr} = (\Pi_1, \varrho_1, \dots, \Pi_{i-1}, \varrho_{i-1})$, then for every possible next prover message Π_{i+1} , where the prover is about to move, then for any prover message Π_i , $\text{State}(i, x, y, \text{tr} \parallel \Pi_i) = 0$
- *Full transcript*: If $\text{State}(i, x, y, \text{tr}) = 0$ for a full transcript $\text{tr} = (\Pi_1, \varrho_1, \dots, \Pi_k, \varrho_k)$, then the verifier outputs $(x', y') := \mathcal{V}^{\Pi, l, y}(\iota, x, \varrho)$ such that $(i', x', y') \notin \mathcal{L}(\mathfrak{R}')$ (where $(\iota, l, i') := \mathcal{I}(i)$).

Definition 3.7 (Round-by-round Soundness). Gen-PIOR has round-by-round (rbr) soundness error ϵ_{RBR} if there exists a state function State such that the following holds. If $\text{State}(i, x, y, \text{tr}) = 0$ for $(i, x, y) \notin \mathcal{L}(\mathfrak{R})$ and a partial transcript $\text{tr} = (\Pi_1, \varrho, \dots, \Pi_i)$, $i \in [k]$, where the verifier is about to move, then

$$\Pr_{\varrho_i \leftarrow \mathbb{R}^*} [\text{State}(i, x, y, \text{tr} \parallel \varrho_i) = 1] \leq \epsilon_{\text{RBR}}(i, x) \quad (4)$$

Definition 3.8 (Round-by-round Knowledge Soundness). Gen-PIOR has round-by-round knowledge error κ_{RBR} if there exists a state function State and polynomial-time extractor $\mathcal{E}$ such that the following holds. If $\text{State}(i, x, y, \text{tr}) = 0$ for a partial transcript $\text{tr} = (\Pi_1, \varrho_1, \dots, \Pi_i)$, $i \in [k]$ where the verifier is about to move, then

$$\Pr_{\varrho_i \leftarrow \mathbb{R}^*} [\text{State}(i, x, y, \text{tr} \parallel \varrho_i) = 1] > \kappa_{\text{RBR}}(i, x)$$

implies the following. For any transcript continuation $(\Pi_{i+1}, \dots, \Pi_k)$, the extractor outputs $w \leftarrow \mathcal{E}(i, x, y, \Pi, w')$ such that $(i, x, y, w) \in \mathfrak{R}$.

3.2.2 Gen-PIOR Composition

Theorem 3.9 (Product Composition of Gen-PIORs). Let $\mathfrak{R}$ and $\{\mathfrak{R}_i\}_{i=1}^m$ be indexed oracle relations such that

$$\mathfrak{R} = \bigcap_{i=1}^m \mathfrak{R}_i \quad \text{i.e.,} \quad (i, x, y, w) \in \mathfrak{R} \iff (i, x, y, w) \in \mathfrak{R}_i \quad \text{for all } i \in [m].$$

For each $i \in [m]$, let $\text{Gen-PIOR}_i : \mathfrak{R}_i \rightarrow \mathfrak{R}'_i$ be a Gen-PIOR with round-by-round soundness error $\epsilon_{\text{RBR}}^{(i)}$. Define $\mathfrak{R}' := \mathfrak{R}'_1 \times \dots \times \mathfrak{R}'_m$ to be the *Cartesian product of relations* as in Definition 3.3. Let $\text{PIOR} : \mathfrak{R} \rightarrow \mathfrak{R}'$ be the protocol that executes the PIOR_i on the same input index-instance-witness tuple and outputs the resulting tuple of index-instance-oracle-witness triples. Then PIOR is a PIOR with round-by-round soundness error

$$\epsilon_{\text{RBR}} \leq \max_{i \in [m]} \epsilon_{\text{RBR}}^{(i)}.$$

Proof. See Appendix A.1. □

3.2.3 Generalized Polynomial interactive Oracle Proofs

Generalized Polynomial Interactive Oracle Proofs (Gen-PIOPs) are special cases of Gen-PIORs that reduce arbitrary relations to decision relations defined as follows.

Definition 3.10 (Decision Relation). An indexed oracle relation $\mathfrak{R}$ is a decision relation $\mathfrak{R}_{\text{Decision}}$ is the set of triples of the form $(i, x, y, w) = (\emptyset, \text{accept}, \emptyset, \emptyset)$.

Compared to standard PIOPs as defined in [GWC19; CHMMVW20], Gen-PIOPs generalize the model by allowing polynomial oracles to be defined over one ring and evaluated over another, and thereby subsume Mod-PIOPs as introduced in [CH24].

Definition 3.11 (Generalized Polynomial Interactive Oracle Proof (Gen-PIOP)). We denote by $\text{Gen-PIOP} = (\text{Algs}, \text{Eval-Conf}, \text{Metrics})$ a polynomial interactive oracle proof for a relation $\mathfrak{R}$ if $\text{Gen-PIOR} = (\text{Algs}, \text{Eval-Conf}, \text{Metrics})$ is a Gen-PIOR of the form $\text{Gen-PIOR} : \mathfrak{R} \rightarrow \mathfrak{R}_{\text{Decision}}$.

3.2.4 Special Cases of Gen-PIORs and Gen-PIOPs

Here, we define several special cases of Gen-PIORs and Gen-PIOPs that are commonly used in practice.

Definition 3.12 (Standard PIOR/PIOP). A standard PIOR (resp., PIOP) is a Gen-PIOR (resp., Gen-PIOP) where the ambient ring and evaluation ring are the same and equal to a field $\mathbb{F}$, that is $\mathbb{R} = \mathbb{S} = \mathbb{F}$. In this case, the evaluation map is the identity map, i.e. $\varphi : \mathbb{F} \rightarrow \mathbb{F}$, $\varphi(x) = x$.

Next, we define modular PIORs and PIOPs over the integers. The notion of a modular PIOP (Mod-PIOP) was first introduced by Campanelli et al. [CH24, Definition 10].

Definition 3.13 (Modular PIOR/PIOP [CH24]). A modular PIOR (resp., PIOP) is a Gen-PIOR (resp., Gen-PIOP) where the ambient ring is the integers and the evaluation ring is a finite field $\mathbb{F}_p$, that is $\mathbb{R} = \mathbb{Z}$ and $\mathbb{S} = \mathbb{F}_p$. In this case, the evaluation map is the natural modulo map $\varphi : \mathbb{Z} \rightarrow \mathbb{F}_p$, $\varphi(x) = x \bmod p$.

Definition 3.14 (Relaxed Modular PIOR/PIOP). A relaxed modular PIOR (resp., PIOP) is a mod-PIOR (resp., Mod-PIOP) in which the extractor is allowed to output a rational witness $w \in \mathbb{Q}_B$.

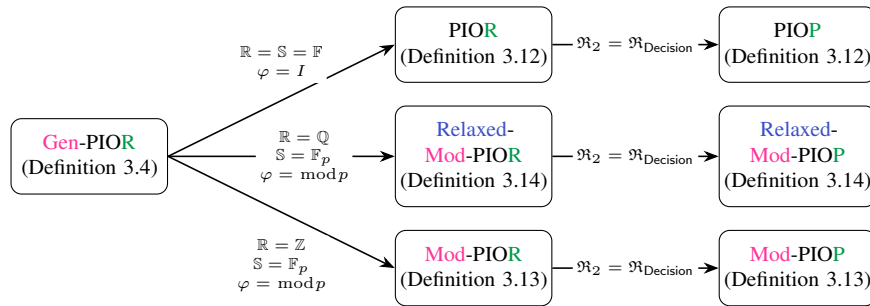


Figure 1: Relationship between the idealized proof systems defined in Section 3.2.

3.3 NP Relations and Arguments Systems

Definition 3.15 (Indexed Relation). An **indexed** relation $\mathfrak{R}$ is a set of triples (i, x, y, w) , where i is the index, x is the explicit instance, y , and w is the witness.

The explicit and implicit instance separation is useful for in decoupling the committed and non-committed information in the argument system, as was proposed in Campanelli et al.[CFQ19] with a different terminology.

3.3.1 Reduction of Knowledge

Since we invoke arguments in a nested manner, we prefer to work with the notion of reduction of knowledge (RoK) as defined by Kothapalli et al. [KP23], which generalizes the notion of argument of knowledge (AoK).

Definition 3.16 (Reduction of Knowledge(RoK, [KP23])). Consider indexed relations $\mathfrak{R}$ and $\mathfrak{R}'$. A reduction of knowledge (RoK) from $\mathfrak{R}$ to $\mathfrak{R}'$ is defined by PPT algorithms $\text{ROK} = (\mathcal{S}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ defined as follows:

- $\mathcal{S}(1^\lambda, N) \rightarrow \text{pp}$: The setup algorithm takes as input a security parameter 1^λ and a maximum size parameter N , and outputs public parameters pp .
- $\mathcal{I}(\text{pp}, i) \rightarrow (\text{pk}, \text{vk}, i')$: The indexer algorithm takes as input the public parameters pp and an index i for relation $\mathfrak{R}$, and outputs a proving key pk , a verification key vk , and a new index i' for relation $\mathfrak{R}'$.
- $\langle \mathcal{P}(\text{pk}, i, x, y, w), \mathcal{V}(\text{vk}, x, y, w') \rangle \rightarrow (w', x', y)$: The prover and verifier are interactive algorithms. The prover takes as input the proving key pk , an index i , an explicit instance x , an implicit instance y , and a witness w , while the verifier takes as input the verification key vk , the explicit instance x , and a new explicit instance x' . At the end of the protocol, the prover outputs a new witness w' , and the verifier outputs a new instance x' .

Definition 3.17 (Completeness). For all size bounds $N \in \mathbb{N}$ and efficient adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{c|c} (i, x, y, w) \in \mathfrak{R} & \text{pp} \leftarrow \mathcal{S}(1^\lambda, N) \\ \vee & (i, x, y, w) \leftarrow \mathcal{A}(\text{pp}) \\ (i', x', y', w') \notin \mathfrak{R}' & (\text{pk}, \text{vk}, i') \leftarrow \mathcal{I}(\text{pp}, i) \end{array} \right] = 1$$

Definition 3.18 (Adaptive Knowledge Soundness). We say RoK is an adaptive knowledge-sound reduction of knowledge, if for all size bounds $N \in \mathbb{N}$ and for all expected poly-time stateful adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists an expected poly-time extractor $\mathcal{E}$ such that

$$\Pr \left[\begin{array}{c|c} (i, x, y, w^*) \notin \mathfrak{R} & \text{pp} \leftarrow \mathcal{S}(1^\lambda, N) \\ \wedge & i \leftarrow \mathcal{A}_1(\text{pp}) \\ (i', x', y', w') \in \mathfrak{R}' & (\text{pk}, \text{vk}, i') \leftarrow \mathcal{I}(\text{pp}, i) \\ & (x, y) \leftarrow \mathcal{A}_2(\text{pk}, \text{vk}, i) \\ & w^* \leftarrow \mathcal{E}(\text{pk}, \text{vk}, i, x, y) \\ \hline & (w', x', y) \leftarrow \langle \mathcal{A}_3(\text{pk}, x, y), \mathcal{V}(\text{vk}, x, y) \rangle \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 3.19 (Semi-Adaptive knowledge soundness). Given a partitioning of the explicit instance $x = (x_1, x_2, x_3)$, we say RoK satisfies semi-adaptive knowledge soundness if for every polynomial-time public-coin sampler Sampler , and all expected poly-time adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{A}_4)$, there exists an

expected poly-time extractor $\mathcal{E}$ such that:

$$\Pr \left[\begin{array}{c} (i, x, y, w^*) \notin \mathfrak{R} \\ \wedge \\ (i', x', y', w') \in \mathfrak{R}' \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \mathcal{S}(1^\lambda, N) \\ i \leftarrow \mathcal{A}_1(\text{pp}) \\ (\text{pk}, \text{vk}, i') \leftarrow \mathcal{I}(\text{pp}, i) \\ x_1 \leftarrow \mathcal{A}_2(\text{pp}) \\ x_2 \leftarrow \text{Sampler} \\ x_3 \leftarrow \mathcal{A}_3(\text{pk}, \text{vk}, i, x_1, x_2) \\ \hline x := (x_1, x_2, x_3) \\ w^* \leftarrow \mathcal{E}(\text{pk}, \text{vk}, i, x, y) \\ \hline (w', x', y) \leftarrow \langle \mathcal{A}_4(\text{pk}, x, y), \mathcal{V}(\text{vk}, x, y) \rangle \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 3.20. We say ROK is succinct if the size of the communication transcript between the prover and the verifier is $\text{poly}(\log(|w|), \lambda)$ and the running time of the verifier is $\text{poly}(\log(|w|), |x|, \lambda)$.

3.3.2 Arguments of Knowledge

Definition 3.21 (Argument of Knowledge (ARK)). We denote by $\text{ARK} = (\mathcal{S}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ an argument of knowledge for a relation $\mathfrak{R}$ if $\text{ROK} = (\mathcal{S}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ is a reduction of knowledge from $\mathfrak{R}$ to the decision relation (Definition 3.10).

3.4 Commitment Schemes

A commitment scheme CS over a message space $\mathfrak{D}$ is a tuple of PPT algorithms $\text{CS} = (\text{Setup}, \text{Commit}, \text{Open})$ defined as follows:

- $\text{Setup}(1^\lambda, \mathfrak{D}) \rightarrow \text{pp}$: On input a security parameter 1^λ and a description $\mathfrak{D}$ of the message space generates the public parameters pp.
- $\text{Commit}(\text{pp}; m) \rightarrow (\text{cm}; \text{opn})$: On input the public parameters pp and a message $m \in \mathfrak{D}$, outputs a commitment cm and an opening opn.
- $\text{Open}(\text{pp}, \text{cm}, m, \text{opn}) \rightarrow \{0, 1\}$: On input the public parameter pp, a commitment cm, a message m and an opening opn, verifies the opening of cm to m using the opening opn.

Binding. A commitment scheme is binding if for all PPT adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{c} \text{Open}(\text{pp}, \text{cm}, m, \text{opn}) = 1 \wedge \\ \text{Open}(\text{pp}, \text{cm}, m', \text{opn}') = 1 \wedge \\ m \neq m' \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ (\text{cm}, m, m', \text{opn}, \text{opn}') \leftarrow \mathcal{A}(\text{pp}) \end{array} \right] \leq \text{negl}(\lambda)$$

3.5 Generalized Polynomial Commitment Scheme

We now define a Generalized Polynomial Commitment Scheme (Gen-PCS) for multilinear polynomials, extending standard polynomial commitment schemes [KZG10] and modular polynomial commitment schemes (Mod-PC) [CH24]. The key distinction is that a Gen-PCS permits the evaluation ring to differ from the polynomial's ambient ring, thereby subsuming both standard PCS (Definition 3.26) and Mod-PC as special cases.

Definition 3.22. A Generalized Polynomial Commitment Scheme (Gen-PCS) for multilinear polynomials is a tuple of the form $\text{Gen-PCS} = (\text{Eval-Conf}, \text{Setup}, \text{Commit}, \text{Open}, \text{ROK}_{\text{Eval}})$ where

- Eval-Conf is a tuple $(\mathbb{R}, \mathbb{S}, \varphi)$ defining the evaluation configuration of the oracles in the protocol, where $\mathbb{R}$ denotes the ambient ring, $\mathbb{S}$ denotes the evaluation ring, and φ is an evaluation map of the form $\varphi : \mathbb{R} \rightarrow \mathbb{S}$.
- the tuple $\text{CS} = (\text{Setup}, \text{Commit}, \text{Open})$ separately constitutes a binding commitment scheme (see Section 3.4) for multilinear polynomials over $\mathbb{R}$, i.e. the message space $\mathcal{D}_{\text{CS}} = \mathbb{R}_{\mu}^{\text{ind} \leq 1}$.
- ROK_{Eval} is an interactive argument system for the relations $\mathfrak{R}_{\text{Eval}}$ (Def. 3.23).

Following the approach in [BFS20; BMSS25] we express the evaluation correctness of the Gen-PCS via an NP relation defined as follows. For maximum generality, we define the evaluation relation

Definition 3.23 (Evaluation relation). Given a commitment scheme $\text{CS} = (\text{Setup}, \text{Commit}, \text{Open})$, we define the NP relation $\mathfrak{R}_{\text{Eval}}$ to be the set of triples

$$\begin{pmatrix} i, \\ x, \\ w \end{pmatrix} = \begin{pmatrix} (\text{ck}, \text{vk}) \\ (\ell, \varphi, [\text{cm}_i, \mathbf{y}_i, z_i]_{i \in (\ell)}), \\ ([p_i, \text{opn}_i]_{i \in (\ell)}) \end{pmatrix}$$

where (ck, vk) is a valid committing and verification key for CS and contains a description of the message domain $\mathcal{D}_{\text{CS}} = \mathbb{R}_{\mu}^{\text{ind} \leq 1}$, and $\varphi : \mathbb{R} \rightarrow \mathbb{S}$ is a mapping from the coefficient ring to the evaluation ring, such that for all $i \in [N]$,

$$z_i = \varphi(p_i(\mathbf{y}_i)) \quad \text{and} \quad \text{PCS.Open}(\text{ck}, \text{cm}_i, p_i, \text{opn}_i) = 1$$

Special Cases of Gen-PCS. Depending on whether adversaries are permitted to adaptively select the evaluation point $\mathbf{y}$, we distinguish between two notions of extractability: non-adaptive and adaptive extractability.

Definition 3.24 (Adaptive Extractability). A generalized polynomial commitment scheme $\text{Gen-PCS} = (\text{Eval-Conf}, \text{Setup}, \text{Commit}, \text{Open}, \text{ROK}_{\text{Eval}})$ has adaptive extractability if ROK_{Eval} is a succinct *semi-adaptive* argument of knowledge (see Section 3.3.2) for the relation $\mathfrak{R}_{\text{Eval}}$ (see Definition 3.23) with respect to the following instance partitioning:

$$\mathbf{x}_1 = (\ell, [\text{cm}]_{i \in (\ell)}), \quad \mathbf{x}_2 = \varphi, \quad \mathbf{x}_3 = ([\mathbf{y}_i, z_i]_{i \in (\ell)})$$

Definition 3.25 (Non-Adaptive Extractability). A polynomial commitment scheme $\text{Gen-PCS} = (\text{Eval-Conf}, \text{Setup}, \text{Commit}, \text{Open}, \text{ROK}_{\text{Eval}})$ has adaptive extractability if Eval is a succinct *semi-adaptive* argument of knowledge (see Section 3.3.2) for the relation $\mathfrak{R}_{\text{Eval}}$ (see Definition 3.23).

$$\mathbf{x}_1 = (\ell, [\text{cm}]_{i \in (\ell)}), \quad \mathbf{x}_2 = ([\varphi, \mathbf{y}]_{i \in (\ell)}), \quad \mathbf{x}_3 = ([z]_{i \in (\ell)})$$

Definition 3.26 (Standard Polynomial Commitment Scheme [KZG10]). A standard polynomial commitment scheme for multilinear polynomials is a Gen-PCS in which the ambient ring and the evaluation ring coincide and are equal to a field $\mathbb{F}$, that is, $\mathbb{R} = \mathbb{S} = \mathbb{F}$. In this case, the evaluation map φ is the identity.

Definition 3.27 (Relaxed Modular Polynomial Commitment Scheme). A relaxed modular polynomial commitment scheme (Mod-PC) for multilinear polynomials is a mod-PC in which the extractor is allowed to output a rational witness polynomials $p_i \in \mathbb{Q}_{\mu}^{\text{ind} \leq 1}$.

4 Toolbox: Integer commitment and arguments

In this section, we define integer commitment schemes (ICS) in Section 4.1 and present a generic construction based on polynomial commitment schemes (PCS). Building on this ICS, we then develop a collection of useful arguments over integers in Section 4.2. These arguments serve as key building blocks for our relaxed mod-PC (see Section 5) construction and may be of independent interest.

4.1 Integer Commitment Schemes (ICS)

We begin by defining ICS, which are commitment schemes whose message space consists of t positive B -bit integers, i.e., $\mathcal{M} = \mathbb{Z}_B^{+t}$. We then present a construction of a family of ICS based on polynomial commitment schemes (PCS). For concreteness and comparison, we instantiate our construction using the state-of-the-art PCS TensorSwitch (Blaze) [BFRW25]; however, the construction is compatible with any standard PCS.

Definition 4.1. An integer commitment scheme ICS is a tuple of PPT algorithms $\text{ICS} = (\text{Setup}, \text{Commit}, \text{Open})$

1. $\text{Setup}(1^\lambda, B, t) \rightarrow \text{pp}$: On input a security parameter 1^λ , arity $t := 2^\tau$ and a the maximum bit length supported $B := 2^\beta$ generates public parameters pp .
2. $\text{Commit}(\text{pp}; z) \rightarrow (\text{cm}; \text{opn})$: On input the public parameters pp and a message $z \in \mathbb{Z}_B^{+t}$, outputs a commitment cm and an opening opn .
3. $\text{Open}(\text{pp}, \text{cm}, z, \text{opn}) \rightarrow \{0, 1\}$: On input the public parameter pp , a commitment cm , a message z and an opening opn , verifies the opening of cm to z using the opening opn .

Next, we present our construction of ICS based on PCS. The main idea is to represent the integer bits as evaluations of a multilinear polynomial on the boolean hypercube and then commit to the polynomial using a multilinear PCS.

Construction 1: Integer Commitment

$\text{ICS.Setup}(1^\lambda, t, B) \rightarrow (\text{ck}, \text{vk})$

1. Update B and t to their next powers of two, i.e., $B := 2^\beta$ and $t := 2^\tau$.
 2. Invoke $(\text{ck}_{\text{PCS}}, \text{vk}_{\text{PCS}}) := \text{PCS.Setup}(1^\lambda, B, t)$.
 3. Output $\text{ck}_{\text{ICS}} := (B, t, \text{ck}_{\text{PCS}})$ and $\text{vk}_{\text{ICS}} := (B, t, \text{vk}_{\text{PCS}})$.
-

$\text{ICS.Commit}(z, \text{ck}) \rightarrow (\text{cm}; \text{opn})$

1. Parse ck as ck_{PCS} and extract $\mathbb{F}_p$ from ck_{PCS} .
 2. For $i \in [t]$, Expand the integer $z[i]$ into its binary representation $\hat{z}_i = [\hat{z}_{i,0}, \hat{z}_{i,1}, \dots, \hat{z}_{i,B-1}] \in \{0, 1\}^B$.
 3. Concatenate the binary representations $\hat{z} := [\hat{z}_1 || \hat{z}_2 || \dots || \hat{z}_t] \in \{0, 1\}^{tB}$.
 4. Compute the multilinear extension $\tilde{z} \in \mathbb{F}_{p, \beta+\tau}^{\text{ind} \leq 1}$ of the vector $\hat{z}$.
 5. Compute the polynomial commitment $(\text{cm}_{\text{PCS}}; \text{opn}_{\text{PCS}}) := \text{PCS.Commit}(\text{ck}_{\text{PCS}}, \tilde{z})$.
 6. Set $\text{cm} := \text{cm}_{\text{PCS}}$ and $\text{opn} := \text{opn}_{\text{PCS}}$, then output $(\text{cm}; \text{opn})$.
-

$\text{ICS.Open}(\text{ck}, \text{cm}, z, \text{opn}) \rightarrow b$

1. Parse ck as ck_{PCS} , opn as opn_{PCS} , and cm as cm_{PCS} .
2. For $i \in [t]$, Expand the integer $z[i]$ into its binary representation $\hat{z}_i = [\hat{z}_{i,0}, \hat{z}_{i,1}, \dots, \hat{z}_{i,B-1}] \in \{0, 1\}^B$.
3. Concatenate the binary representations $\hat{z} := [\hat{z}_1 || \hat{z}_2 || \dots || \hat{z}_t] \in \{0, 1\}^{tB}$.

4. Compute the multilinear extension $\tilde{z} \in \mathbb{F}_{p,\beta+\tau}^{\text{ind} \leq 1}$ of the vector $\hat{z}$.
5. Invoke $b_{\text{PCS}} := \text{PCS.Open}(\text{ck}_{\text{PCS}}, \text{cm}_{\text{PCS}}, \tilde{z}, \text{opn}_{\text{PCS}})$ and output b_{PCS} .

Theorem 4.2. Let PCS be a standard polynomial commitment scheme (see Definition 3.26), then Construction 2 is an integer commitment scheme according to Definition 4.1 with the following properties:

Setup Time	Ck Size	Vk Size	Com. Time	Com. Size	Opn. Time
$T(\text{PCS.Setup})$	$ \text{ck}_{\text{PCS}} $	$ \text{vk}_{\text{PCS}} $	$T(\text{PCS.Commit})$	$ \text{cm}_{\text{PCS}} $	$T(\text{PCS.Open})$

Moreover, instantiating Construction 2 with the state-of-the-art PCS TensorSwitch (Blaze) [BFRW25] yields a transparent integer commitment scheme under collision-resistant hashing with the following optimal properties:

Setup Time	Ck Size	Vk Size	Com. Time	Com. Size	Opn. Time
$O(1)$	$O(1)$	$O(1)$	$O(Bt)$	$O(1)$	$O(Bt)$

4.2 A Suite of Primitive Integer Arguments

In this section, we develop a suite of arguments over integers that serve as building blocks for our relaxed mod-PC (Section 5) construction.

Idealized presentation of the protocols. Our protocols are tailored to the family of integer commitment schemes defined in Construction 2, which represent committed integers using multilinear polynomials. For clarity and ease of exposition, rather than presenting the protocols as full arguments of knowledge over committed integers for a relation $\mathfrak{R}_X$, we describe them for a dual relation $\mathfrak{R}_{X\text{-Oracle}}$ defined as follows:

Definition 4.3 (Dual Integer Indexed Relations). Let ICS be the integer commitment scheme for $\mathbb{Z}_B$ as defined in Construction 2. Let $\mathfrak{R}_X^{\text{ICS}}$ be an indexed relation (defined in Definition 3.15) consisting of tuples (i, x, y, w) and $\mathfrak{R}_{X\text{-Oracle}}^{\text{ICS}}$ be an indexed oracle relation (defined in Definition 3.2) consisting of tuples (i_o, x_o, y_o, w_o) . We say that $\mathfrak{R}_{X\text{-Oracle}}^{\text{ICS}}$ and $\mathfrak{R}_X^{\text{ICS}}$ are *dual* relations if

- $i = \emptyset$ and $i_o = (\text{ck}, \text{vk})$.
- For all $x \in \mathbb{Z}_B$, if $x \in x$ (respectively, w), then $\tilde{x} \in x_o$ (respectively, w_o) where $\tilde{x}$ is the multilinear extension of x , the binary decomposition of x .
- For all $x \in \mathbb{Z}_B$, let $(\text{cm}_x; \text{opn}_x) = \text{ICS.Commit}(x, \text{ck})$, then if $\text{cm}_x \in y$, then $[\tilde{x}] \in y_o$ where $\tilde{x}$ is the multilinear extension of x , the binary decomposition of x .
- For all $x \notin \mathbb{Z}_B$, if $x \in x$ (respectively, y and w), then $x \in x_o$ (respectively, y_o and w_o).

in an idealized model where the prover has oracle access to the multilinear polynomials representing the committed integers. These idealized protocols can be compiled into arguments of knowledge for the committed integers using the general recipe described in Construction 8.

Protocols as reductions. Since our protocols invoke several subprotocols, we present them as reductions to other relations. Concretely, instead of using Polynomial Interactive Oracle Proofs (PIOPs, see Definition 3.11), we work with Polynomial Interactive Oracle Reductions (PIORs, see Definition 3.4).

Purpose of the protocols. Ultimately, we require two main protocols from this section: (a) the range-check protocol (Section 4.2.2), which verifies that a committed integer lies within a specified range, and (b) the integer division-check protocol (Section 4.2.6), which verifies the correctness of the quotient and remainder for a committed dividend and divisor. To construct these protocols, we introduce several intermediate building blocks.

Batched Representation. We present all our protocols in a batched form, where the prover proves the correctness of t instances of the relation in parallel. For example, in the integer addition protocol, the prover proves the correctness of t addition instances in parallel.

4.2.1 Integer Addition

Although integer addition is not a building block of our relaxed Mod-PC construction, we include it as part of our additional contributions to provide a complete suite of integer arguments.

Definition 4.4 (Integer Addition Oracle Relation). The oracle relation $\mathfrak{R}_{\text{Add-Oracle}}$ is the set of all triples of the form $(x, y, w) = (\emptyset, ([\tilde{a}], [\tilde{b}], [\tilde{c}]), (\tilde{a}, \tilde{b}, \tilde{c}))$, where $\tilde{a}, \tilde{b}, \tilde{c} \in \mathbb{F}_{p, \beta + \tau}^{\text{ind} \leq 1}$ have evaluation vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \{0, 1\}^{Bt}$ such that $B = 2^\beta$ and $t = 2^\tau$, and they are of the form

$$\mathbf{a} = [a_0 || \dots || a_{t-1}], \quad \mathbf{b} = [b_0 || \dots || b_{t-1}], \quad \mathbf{c} = [c_0 || \dots || c_{t-1}]$$

Moreover, for $i \in [t-1]$, each of the vectors $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \{0, 1\}^{2B}$ represent the bit decomposition of integers a_i, b_i, c_i respectively as

$$a_i = \sum_{j \in [B]} a_i[j] 2^j, \quad b_i = \sum_{j \in [B]} b_i[j] 2^j, \quad c_i = \sum_{j \in [2B]} c_i[j] 2^j$$

and they satisfy the addition relation; i.e. $c_i = a_i + b_i$

Moreover, we denote by $\mathfrak{R}_{\text{Add}}$ as the dual relation of $\mathfrak{R}_{\text{Add-Oracle}}$ according to Definition 4.3.

Next, we define the corresponding PIOR for the integer addition relation. The key idea is that if $c_i = a_i + b_i$, then there exists a binary vector $\mathbf{k}_i$ (the carry vector) such that the bit decompositions of $\mathbf{a}_i, \mathbf{b}_i$, and $\mathbf{c}_i$ satisfy the relation $a_i[j] + b_i[j] + k_i[j] = c_i[j] + 2k_i[j+1]$ over $\mathbb{F}_p$.

Hence, the prover can send an oracle for the carry MLE $\tilde{k}$ to the verifier, and the verifier can check the above relation at a random point. The main technical challenge is enabling the verifier to evaluate $\tilde{k}$ at a rotated point while having only oracle access to $\tilde{k}$. A naïve approach would be to recommit to the rotated carry MLE and verify consistency via a prescribed permutation check [CBBZ23]; however, this would incur an additional commitment, as well as extra rounds and queries. Instead, we exploit the fact that rotation is a highly structured prescribed permutation. In particular, rotation by one position corresponds to adding 1 to the input index in the integer domain, and can therefore be expressed as a low-degree polynomial map over $\mathbb{F}_p$. Consequently, the verifier can obtain the value of $\tilde{k}$ at the rotated point by computing the rotated query point via this map and then querying $\tilde{k}$ at the resulting point.

Definition 4.5 (Rotation-by- σ Index Map). Let $\sigma \in \{0, 1\}^\mu$ be the binary expansion of the rotation amount $\sigma \in [2^\mu]$. Given a point $\mathbf{x} \in \mathbb{F}_p^\mu$, define the carry-chain vector $\mathbf{c} \in \mathbb{F}_p^\mu$ by setting $c[0] = 0$ and, for $j \in [\mu-1]$, defining $c[j]$ recursively as

$$c[j] = \mathbf{x}[j-1]\sigma[j-1] + \mathbf{x}[j-1]c[j-1] + \sigma[j-1]c[j-1] - 2\mathbf{x}[j-1]\sigma[j-1]c[j-1].$$

This recurrence is the multilinear extension of the ripple-carry rule for binary addition. The *rotation-by- σ index map* $\text{Rot}_\sigma : \mathbb{F}^\mu \rightarrow \mathbb{F}^\mu$ is the multilinear extension of the function $x \mapsto x + \sigma \pmod{2^\mu}$ where x and σ have binary expansions $\mathbf{x}$ and σ , respectively. For each $j \in [\mu-1]$, the j -th coordinate of $\text{Rot}_\sigma(\mathbf{x})$ is defined by

$$\text{Rot}_\sigma(\mathbf{x})_j = \mathbf{x}[j] + \sigma[j] + c[j] - 2\mathbf{x}[j]\sigma[j] - 2\mathbf{x}[j]c[j] - 2\sigma[j]c[j] + 4\mathbf{x}[j]\sigma[j]c[j].$$

Lemma 4.6 (MLEs are closed under fixed rotations). Let $\mu \in \mathbb{N}$ and $N = 2^\mu$. Let $\tilde{m} : \mathbb{F}^\mu \rightarrow \mathbb{F}$ be the multilinear extension (MLE) of a vector $\mathbf{m} \in \mathbb{F}_p^N$. For any $\sigma \in [N]$, $\tilde{m}(\text{Rot}_\sigma(\mathbf{x}))$ is the multilinear extension of the rotated vector $\mathbf{m}' \in \mathbb{F}_p^N$ defined by $\mathbf{m}'[i] = \mathbf{m}[i + \sigma \pmod{N}]$

PIOR 1: $\mathfrak{R}_{\text{Add-Oracle}} \rightarrow \mathfrak{R}_{\text{Decision}} \times \mathfrak{R}_{\text{Bin}}^3$

Parse: $\mathbf{x} = \emptyset, \mathbf{y} = (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket), \mathbf{w} = (\tilde{a}, \tilde{b}, \tilde{c})$

1. $\mathcal{P}$ computes the carry vector $\mathbf{k}$ which is the concatenation of $\mathbf{k}_1, \dots, \mathbf{k}_t$ where each $\mathbf{k}_i$ is the carry vector for the i -th addition instance. Concretely, for $i \in [t]$ and $j \in [n-1]$, the carry bit $k_i[j]$ is defined by the following recursive relation:

$$k_i[j+1] = (a_i[j] + b_i[j] + k_i[j] - c_i[j])/2$$

where we set $k_i[0] = 0$ for all $i \in [t]$. $\mathcal{P}$ then sends the oracle $\llbracket \tilde{k} \rrbracket$ to $\mathcal{V}$.

2. $\mathcal{V}$ samples a random vector $\mathbf{r} \in \mathbb{F}_p^\mu$ and queries the oracles $\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket$, and $\llbracket \tilde{k} \rrbracket$ at the point $\mathbf{r}$ to obtain the values $\tilde{a}(\mathbf{r}), \tilde{b}(\mathbf{r}), \tilde{c}(\mathbf{r}), \tilde{k}(\mathbf{r})$.
3. $\mathcal{V}$ computes the rotated point $\text{Rot}_1(\mathbf{r})$ using the rotation-by-one index map defined in Definition 4.5 and queries $\llbracket \tilde{k} \rrbracket$ at the rotated point to obtain $\tilde{k}(\text{Rot}_1(\mathbf{r}))$.
4. To check if the addition relation holds for the queried point, $\mathcal{V}$ checks:

$$\tilde{a}(\mathbf{r}) + \tilde{b}(\mathbf{r}) + \tilde{k}(\mathbf{r}) \stackrel{?}{=} \tilde{c}(\mathbf{r}) + 2 \cdot \tilde{k}(\text{Rot}_1(\mathbf{r}))$$

$\mathcal{P}$ outputs $\mathbf{w}'_{\text{Decision}} := \emptyset$ and $\mathcal{V}$ outputs $(\mathbf{x}'_{\text{Decision}}, \mathbf{y}'_{\text{Decision}}) := (\text{accept}, \emptyset)$ for the decision relation $\mathfrak{R}_{\text{Decision}}$ which checks the above polynomial identity.

5. To verify that the committed vectors are binary, $\mathcal{P}$ outputs the following witnesses for the binary relation (Definition A.2):

$$\mathbf{w}'_{\text{Bin},a} := \tilde{a}, \quad \mathbf{w}'_{\text{Bin},b} := \tilde{b}, \quad \mathbf{w}'_{\text{Bin},c} := \tilde{c}$$

Correspondingly, $\mathcal{V}$ outputs the associated instances and oracle instances for the binary relation as

$$(\mathbf{x}'_{\text{Bin},a}, \mathbf{y}'_{\text{Bin},a}) := (\emptyset, \llbracket \tilde{a} \rrbracket), \quad (\mathbf{x}'_{\text{Bin},b}, \mathbf{y}'_{\text{Bin},b}) := (\emptyset, \llbracket \tilde{b} \rrbracket), \quad (\mathbf{x}'_{\text{Bin},c}, \mathbf{y}'_{\text{Bin},c}) := (\emptyset, \llbracket \tilde{c} \rrbracket)$$

Theorem 4.7. PIOR 2 has perfect completeness and round-by-round soundness error at most μ/p with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
1	5	2^μ	$O(2^\mu)$	$O(\mu)$

Proof. Deferred to Appendix A.4.1. □

4.2.2 Integer Range

Definition 4.8 (Integer Range Oracle Relation). The oracle relation $\mathfrak{R}_{\text{Range-Oracle}}$ is the set of all triples of the form $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = ((m), (\llbracket \tilde{a} \rrbracket), (\tilde{a}))$, where $m \in \mathbb{Z}_B^+$ is the range upper bound and $\tilde{a} \in \mathbb{F}_{p,\beta+\tau}^{\text{ind} \leq 1}$ has an evaluation vector $\mathbf{a} \in \{0, 1\}^{Bt}$ such that $B = 2^\beta$ and $t = 2^\tau$, and it's of the form $\mathbf{a} = [\mathbf{a}_0 || \dots || \mathbf{a}_{t-1}]$. Moreover, for $i \in [t-1]$, the vector $\mathbf{a}_i \in \{0, 1\}^{2B}$ represents the bit decomposition of an integer a_i as $a_i = \sum_{j \in [B]} \mathbf{a}_i[j] 2^j$ and it satisfied the range bound; i.e. $0 \leq a_i \leq m$.

Moreover, we denote by $\mathfrak{R}_{\text{Range}}$ as the dual relation of $\mathfrak{R}_{\text{Range-Oracle}}$ according to Definition 4.3.

PIOR 2: $\mathfrak{R}_{\text{Range-Oracle}} \rightarrow \mathfrak{R}_{\text{Add-Oracle}}$

Parse: $x = m, y = \llbracket \tilde{a} \rrbracket, w = \tilde{a}$

1. $\mathcal{P}$ computes $\tilde{b}$ such that the underlying integers satisfy $a_i + b_i = m$ for all i , and sends the oracle $\llbracket \tilde{b} \rrbracket$ to $\mathcal{V}$.
2. $\mathcal{P}$ outputs a witness for $\mathfrak{R}_{\text{Add-Oracle}}$ corresponding to the tuple (a, b, c) with $c_i = m$ for all i , namely $w'_{\text{Add}} := (\tilde{a}, \tilde{b}, \tilde{c})$, where $\tilde{c}$ is the multilinear extension of the constant vector $(m, \dots, m)$.
3. $\mathcal{V}$ outputs the corresponding instance and oracle instance for $\mathfrak{R}_{\text{Add-Oracle}}$, namely $(x'_{\text{Add}}, y'_{\text{Add}}) := (\emptyset, (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket))$, using the received oracle $\llbracket \tilde{b} \rrbracket$.

Theorem 4.9. PIOR 3 has perfect completeness and round-by-round soundness error 0 with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
1	5	2	$O(2^\mu)$	$O(1)$

Proof. Deferred to Appendix A.4.2. □

Next, we present three seemingly unrelated PIORs which will be used as building blocks for the integer multiplication argument: the Lagrange-Check PIOR, and the NTT-Check PIOR.

4.2.3 Lagrange

In this section, we formalize the Lagrange check relation which checks if a given multilinear polynomial is the evaluation of the univariate Lagrange polynomial at a publicly known point, then present a PIOR to check this relation by reducing it to the zero-check relation (see Def. A.6). We will use this Lagrange-Check PIOR as a building block for the NTT-Check PIOR (see PIOR 5).

Definition 4.10 (Lagrange Oracle Relation). The oracle relation $\mathfrak{R}_{\text{Lag-Oracle}}$ is the set of all triples of the form $(x, y, w) = ((r, \omega), (\llbracket \tilde{v} \rrbracket), (\tilde{v}))$, where $r \in \mathbb{F}_p$, $\omega \in \mathbb{F}_p$ is a primitive B -th root of unity, and $\tilde{v} \in \mathbb{F}_{p, \beta+\tau}^{\text{ind} \leq 1}$ has an evaluation vector $v \in \{0, 1\}^{Bt}$ such that $B = 2^\beta$ and $t = 2^\tau$, and it's of the form $v = [\mathbf{Lag} \parallel \dots \parallel \mathbf{Lag}]$. Moreover the vector $\mathbf{Lag} \in \{0, 1\}^{2B}$ is of the form $\mathbf{Lag}[i] = L_i(r)$ for $i \in [B-1]$ where L_i is the i -th Lagrange basis polynomial over the FFT-friendly domain $\mathbb{H} = [\omega^k]_{k \in [B]}$.

Moreover, we denote by $\mathfrak{R}_{\text{Lag}}$ as the dual relation of $\mathfrak{R}_{\text{Lag-Oracle}}$ according to Definition 4.3.

PIOR 3: $\mathfrak{R}_{\text{Lag-Oracle}} \rightarrow \mathfrak{R}_{\text{Zero}}$

Parse: $x = (r, \omega), y = \llbracket \tilde{v} \rrbracket, w = \tilde{v}$

1. $\mathcal{P}$ and $\mathcal{V}$ define the succinctly representable MLE of the vector $\mathbf{mon}_\omega = [\omega^0, \omega^1, \dots, \omega^{B-1}]$ as $\widetilde{\mathbf{mon}}_\omega(x_0, \dots, x_{\beta-1}) = \prod_{j=0}^{\mu-1} ((1 - x_j) + x_j \omega^{2^j})$.
2. $\mathcal{V}$ samples $r = [r_1, \dots, r_\tau] \in \mathbb{F}_p^\tau$ and sends it to the prover.
3. $\mathcal{P}$ and $\mathcal{V}$ define the virtual polynomial $f \in \mathbb{F}_{p_\beta}^{\text{ind} \leq 2}$ as

$$f(x_0, \dots, x_{\beta-1}) = N \tilde{v}(x_0, \dots, x_{\beta-1}, r_1, \dots, r_\tau) (r - \widetilde{\mathbf{mon}}_\omega(x_0, \dots, x_{\beta-1})) - Z_{\mathbb{H}(\omega)} \widetilde{\mathbf{mon}}_\omega(x_0, \dots, x_{\beta-1})$$

4. To verify that the virtual polynomial f is zero on the boolean hypercube, $\mathcal{P}$ outputs the witnesses for the Zero-Check relation (Definition A.6) as $w'_{\text{Zero}} := f$ while $\mathcal{V}$ outputs the corresponding explicit and implicit

instance for the Zero-Check relation as $(x'_{\text{Zero}}, y'_{\text{Zero}}) := (\emptyset, (\llbracket f \rrbracket))$.

Theorem 4.11. PIOR 4 has perfect completeness and round-by-round soundness error at most $\max\left\{\frac{\tau}{p}, \frac{\beta}{p}\right\}$ with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
1	0	1	$O(2^\mu)$	$O(1)$

Proof. Deferred to Appendix A.4.3. □

4.2.4 NTT Oracle Check

In this section, we formalize the NTT relation between two vectors and their corresponding MLEs, which checks if one vector is indeed the NTT of the other vector, and then present a PIOR to reduce this relation to the Lagrange-Check relation (Definition 4.10) and the Sumcheck relation (Definition A.9). We will use this NTT-Check PIOR as a key building block for the integer multiplication (see PIOR 6) and division check (see PIOR 7) arguments.

Definition 4.12 (NTT Relation). The oracle relation $\mathfrak{R}_{\text{NTT-Oracle}}$ is the set of all triples of the form $(x, w) = ((\omega, \llbracket \tilde{m} \rrbracket, \llbracket \tilde{m}' \rrbracket), (\mathbb{H}, \tilde{m}, \tilde{m}'))$ where $\omega \in \mathbb{F}_p$ is the B 'th root of unity, $\mathbb{H} := [\omega^i]_{i=0}^{B-1}$ is the NTT-friendly domain associated with ω , and $\tilde{m}, \tilde{m}' \in \mathbb{F}_{p,B}^{\text{ind} \leq 1}$ have evaluation vectors $\mathbf{m}, \mathbf{m}'$ that satisfy the NTT relation; i.e. $\mathbf{m}' = \text{NTT}_{\mathbb{H}}(\mathbf{m})$.

Moreover, we denote by $\mathfrak{R}_{\text{NTT}}$ as the dual relation of $\mathfrak{R}_{\text{NTT-Oracle}}$ according to Definition 4.3.

Next, we present a PIOP to check if a vector is indeed the NTT of another vector. The core idea is to interpret the original vector $\mathbf{m}$ as the coefficients of a *univariate* polynomial and the NTT vector $\mathbf{m}'$ as *evaluations* of the same polynomial on the NTT-friendly domain $\mathbb{H}$, then use the fact the two vectors represent the same univariate polynomial, hence they should be equal on a random point s sampled by the verifier.

PIOR 4: $\mathfrak{R}_{\text{NTT-Oracle}} \rightarrow \mathfrak{R}_{\text{Lag-Oracle}} \times \mathfrak{R}_{\text{sum}}$

Parse: $x = (\omega, \llbracket \tilde{m} \rrbracket, \llbracket \tilde{m}' \rrbracket), y = \phi, w = (\mathbb{H}, \tilde{m}, \tilde{m}')$

1. $\mathcal{V}$ samples a random element $r \xleftarrow{\$} \mathbb{F}_p$ and sends it to $\mathcal{P}$.
2. $\mathcal{P}$ and $\mathcal{V}$ define the succinctly representable MLE of $\mathbf{u} := [\mathbf{mon}_r \parallel \dots \parallel \mathbf{mon}_r] \in \mathbb{F}_p^{Bt}$ where $\mathbf{mon}_r = [r^0, r^1, \dots, r^{B-1}]$ as

$$\tilde{u}(x_0, \dots, x_{\beta-1}, y_0, \dots, y_{\tau-1}) = \prod_{j=0}^{\beta-1} ((1 - x_j) + x_j r^{2^j}) \in p_{\beta+\tau}^{\text{ind} \leq 1}$$

3. $\mathcal{P}$ computes $\tilde{v} \in \mathbb{F}_{p,\beta+\tau}^{\text{ind} \leq 1}$ as the multilinear extension of the vector $\mathbf{v} := [\mathbf{Lag} \parallel \dots \parallel \mathbf{Lag}]$, where $\mathbf{Lag} := [L_0(r), \dots, L_{B-1}(r)] \in \mathbb{F}_p^B$ and $L_i(X) \in \mathbb{F}_p^{\leq B}$ denotes the i -th univariate Lagrange polynomial over the NTT-friendly domain $\mathbb{H}$. The $\mathcal{P}$ then sends the oracle $\llbracket \tilde{v} \rrbracket$ to the verifier.
4. To verify that the multilinear polynomial $\tilde{v}$ has a correct form, $\mathcal{P}$ outputs the witnesses for the Lagrange Check relation (Definition 4.10) as $w'_{\text{Zero}} := f$ while $\mathcal{V}$ outputs the corresponding explicit and implicit instance for the Lagrange Check relation as $(x'_{\text{Zero}}, y'_{\text{Zero}}) := (r, \omega, \llbracket \tilde{v} \rrbracket, \tilde{v})$.
5. To verify that $\mathbf{m}$ and $\mathbf{m}'$ have the same evaluation on the randomly chosen point r , $\mathcal{P}$ outputs the witnesses for the sumcheck relation (Definition A.9) as $w'_{\text{Sum}} := f$ while $\mathcal{V}$ outputs the corresponding explicit and implicit

instance for the Lagrange Check relation as $(x'_{\text{Sum}}, y'_{\text{Sum}}) := (0, \llbracket f \rrbracket)$ where f is the virtual polynomial defined as $f(x) := \tilde{m}(x) \cdot \tilde{u}(x) - \tilde{m}'(x) \cdot \tilde{v}(x)$.

Theorem 4.13. PIOR 5 has perfect completeness and round-by-round soundness error at most tB/p with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
1	0	1	$O(B)$	$O(\beta)$

Proof. Deferred to Appendix A.4.4. □

4.2.5 Integer Multiplication

In this section, we formalize the integer multiplication relation, which checks if three committed integers satisfy the multiplication relation, and then present a PIOR for this relation by reducing it to the Hadamard relation (Definition A.4), the NTT relation (Definition 4.12), and the binary relation (Definition A.2). We will use this Integer Multiplication PIOR as a building block for the Division-Check PIOP (see PIOR 7) and Masdak (see PIOR 8).

Definition 4.14 (Integer Multiplication Oracle Relation). The oracle relation $\mathfrak{R}_{\text{Mul-Oracle}}$ is the set of all triples of the form $(x, y, w) = (\emptyset, (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket), (\tilde{a}, \tilde{b}, \tilde{c}))$ where $\tilde{a}, \tilde{b}, \tilde{c} \in \mathbb{F}_{p, \beta+\tau}^{\text{ind} \leq 1}$ have evaluation vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \{0, 1\}^{Bt}$ such that $B = 2^\beta$ and $t = 2^\tau$, and they are of the form

$$\mathbf{a} = [a_0 || \dots || a_{t-1}], \quad \mathbf{b} = [b_0 || \dots || b_{t-1}], \quad \mathbf{c} = [c_0 || \dots || c_{t-1}]$$

Moreover, for $i \in [t-1]$, each of the vectors $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \{0, 1\}^{2B}$ represent the bit decomposition of integers a_i, b_i, c_i respectively as

$$a_i = \sum_{j \in [B]} a_i[j] 2^j, \quad b_i = \sum_{j \in [B]} b_i[j] 2^j, \quad c_i = \sum_{j \in [2B]} c_i[j] 2^j$$

and they satisfy the multiplication relation; i.e. $c_i = a_i b_i$

Moreover, we denote by $\mathfrak{R}_{\text{Mul}}$ as the dual relation of $\mathfrak{R}_{\text{Mul-Oracle}}$ according to Definition 4.3.

A PIOR for the batched integer multiplication oracle relation is presented below. Refer to Section 2.3.1 for an informal overview of the main ideas.

PIOR 5: $\mathfrak{R}_{\text{Mul-Oracle}} \rightarrow \mathfrak{R}_{\text{Had}} \times \mathfrak{R}_{\text{Bin}}^3 \times \mathfrak{R}_{\text{NTT-Oracle}}^3$

Parse: $x = \emptyset, y = (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket), w = (\tilde{a}, \tilde{b}, \tilde{c})$

1. $\mathcal{P}$ and $\mathcal{V}$ define a mask polynomial $\tilde{m} \in \mathbb{F}_{p, \beta+\tau}^{\text{ind} \leq 1}$ with succinct representation that has an evaluation vector of the form $\mathbf{m} = [\mathbf{m}' || \dots || \mathbf{m}']$ where $\mathbf{m}' = [1^{B/2} || 0^{B/2}]$ as follows:

$$\tilde{m}(x_0, \dots, x_{\beta-1}, y_0, \dots, y_{\tau-1}) = 1 - x_{\beta-1}$$

2. To verify that there is no overflow in the multiplication; i.e. the integers encoded in $\mathbf{a}$ and $\mathbf{b}$ have bit-length at most $B/2$, $\mathcal{P}$ outputs two witnesses for the Zero Check relation (Definition A.6) as $w'_{\text{Zero}, a} := f_a$, $w'_{\text{Zero}, b} := f_b$ while $\mathcal{V}$ outputs the corresponding explicit and implicit instances for the Zero Check relation as $(x'_{\text{Zero}, a}, y'_{\text{Zero}, a}) := (\emptyset, \llbracket f_a \rrbracket)$ and $(x'_{\text{Zero}, b}, y'_{\text{Zero}, b}) := (\emptyset, \llbracket f_b \rrbracket)$ where $f_a \in \mathbb{F}_{p, \beta+\tau}^{\text{ind} \leq 2}$ and $f_b \in \mathbb{F}_{p, \beta+\tau}^{\text{ind} \leq 2}$ are virtual polynomials defined as

$$f_a(x, y) := \tilde{m}(x, y) \cdot \tilde{a}(x, y), \quad f_b(x, y) := \tilde{m}(x, y) \cdot \tilde{b}(x, y)$$

3. $\mathcal{P}$ computes the NTT of the bit decomposition of a , b , and c :

$$\mathbf{a}' := \text{NTT}(\mathbf{a}), \quad \mathbf{b}' := \text{NTT}(\mathbf{b}), \quad \mathbf{c}' := \text{NTT}(\mathbf{c})$$

and sends the oracles $\llbracket \tilde{a}' \rrbracket$, $\llbracket \tilde{b}' \rrbracket$, and $\llbracket \tilde{c}' \rrbracket$ to $\mathcal{V}$. To verify the correctness of the NTT computations, $\mathcal{P}$ outputs the witnesses for the NTT relation (Definition 4.12): $\mathbf{w}'_{\text{NTT},a} := (\tilde{a}, \tilde{a}')$, $\mathbf{w}'_{\text{NTT},b} := (\tilde{b}, \tilde{b}')$, $\mathbf{w}'_{\text{NTT},c} := (\tilde{c}, \tilde{c}')$. Correspondingly, $\mathcal{V}$ outputs the associated instances and oracle instances for the NTT relation as

$$\begin{aligned} (\mathbf{x}'_{\text{NTT},a}, \mathbf{y}_{\text{NTT},a}) &:= (\emptyset, (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{a}' \rrbracket)), & (\mathbf{x}'_{\text{NTT},b}, \mathbf{y}_{\text{NTT},b}) &:= (\emptyset, (\llbracket \tilde{b} \rrbracket, \llbracket \tilde{b}' \rrbracket)), \\ (\mathbf{x}'_{\text{NTT},c}, \mathbf{y}_{\text{NTT},c}) &:= (\emptyset, (\llbracket \tilde{c} \rrbracket, \llbracket \tilde{c}' \rrbracket)) \end{aligned}$$

4. To verify that the Hadamard relation holds among the NTT vectors, $\mathcal{P}$ outputs a witness for the Hadamard relation (Definition A.4), as $\mathbf{w}'_{\text{Had}} := (\tilde{a}, \tilde{b}, \tilde{c}, 0)$, where the linear term is the zero polynomial. $\mathcal{V}$ outputs the corresponding instance and oracle instance for the Hadamard relation as $(\mathbf{x}'_{\text{Had}}, \mathbf{y}'_{\text{Had}}) := (\emptyset, (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket, \llbracket 0 \rrbracket))$.
5. To verify that the committed vectors are binary and therefore valid bit decompositions of integers, $\mathcal{P}$ outputs the following witnesses for the binary relation (Definition A.2): $\mathbf{w}'_{\text{Bin},a} := \tilde{a}$, $\mathbf{w}'_{\text{Bin},b} := \tilde{b}$, $\mathbf{w}'_{\text{Bin},c} := \tilde{c}$. Correspondingly, $\mathcal{V}$ outputs the associated instances and oracle instances for the binary relation as

$$\begin{aligned} (\mathbf{x}'_{\text{Bin},a}, \mathbf{y}'_{\text{Bin},a}) &:= (\emptyset, \llbracket \tilde{a} \rrbracket), & (\mathbf{x}'_{\text{Bin},b}, \mathbf{y}'_{\text{Bin},b}) &:= (\emptyset, \llbracket \tilde{b} \rrbracket), \\ (\mathbf{x}'_{\text{Bin},c}, \mathbf{y}'_{\text{Bin},c}) &:= (\emptyset, \llbracket \tilde{c} \rrbracket) \end{aligned}$$

Theorem 4.15. PIOR 6 has perfect completeness and round-by-round soundness error 0 with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
0	0	3	$O(Bt)$	$O(1)$

Proof. Deferred to Appendix A.4.5. □

4.2.6 Division Check

In this section, we define the integer division relation, which checks if four integers satisfy the division relation, and then present a PIOR for this which is very similar to the Integer Multiplication PIOR (see PIOR 6).

Definition 4.16 (Division Oracle Relation). The oracle relation $\mathfrak{R}_{\text{Div-Oracle}}$ is the set of all tuples of the form $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = (\gamma, (\llbracket \tilde{v} \rrbracket, \llbracket \tilde{q} \rrbracket, \tilde{\ell}, \tilde{r}), (\tilde{v}, \tilde{q}))$ where $\tilde{v}, \tilde{q}, \tilde{\ell}, \tilde{r} \in \mathbb{F}_{p,\beta+\tau}^{\text{ind} \leq 1}$ ($B = 2^\beta, t = 2^\tau$) have evaluation vectors $\mathbf{v}, \mathbf{q}, \mathbf{\ell}, \mathbf{r}$ respectively of the form

$$\mathbf{v} = [\mathbf{v}_0 || \dots || \mathbf{v}_{t-1}], \quad \mathbf{q} = [\mathbf{q}_0 || \dots || \mathbf{q}_{t-1}], \quad \mathbf{\ell} = [\mathbf{\ell}_0 || \dots || \mathbf{\ell}_{t-1}], \quad \mathbf{r} = [\mathbf{r}_0 || \dots || \mathbf{r}_{t-1}]$$

where $\gamma \in \mathbb{N}$ is the number of trailing nonzero entries of each divisor and remainder, $\mathbf{\ell}_i = [0^{\beta-\gamma} || \mathbf{\ell}'_i]$ and $\mathbf{\ell}_i = [0^{\beta-\gamma} || \mathbf{\ell}'_i]$ for all $i \in [t-1]$. Moreover, for $i \in [t-1]$, each of the vectors $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \{0, 1\}^B$ represent the bit decomposition of integers v_i, q_i, ℓ_i, r_i respectively as

$$v_i = \sum_{j \in [B]} v_i[j] 2^j, \quad q_i = \sum_{j \in [B]} q_i[j] 2^j, \quad \ell_i = \sum_{j \in [2B]} \ell_i[j] 2^j, \quad r_i = \sum_{j \in [2B]} r_i[j] 2^j$$

and they satisfy the division relation; i.e. $v_i = q_i \cdot \ell_i + r_i$

Moreover, we denote by $\mathfrak{R}_{\text{Div}}$ as the dual relation of $\mathfrak{R}_{\text{Div-Oracle}}$ according to Definition 4.3.

PIOR 6: $\mathfrak{R}_{\text{Div}} \rightarrow \mathfrak{R}_{\text{Had}} \times \mathfrak{R}_{\text{Bin}}^3 \times \mathfrak{R}_{\text{NTT-Oracle}}^3$

Parse: $x=a, y=b, w=b$

1. $\mathcal{P}$ and $\mathcal{V}$ define a mask polynomial $\tilde{m} \in p_{\beta+\tau}^{\text{ind} \leq 1}$ with succinct representation that has an evaluation vector of the form $\mathbf{m} = [\mathbf{m}' || \dots || \mathbf{m}']$ where $\mathbf{m}' = [1^{B/2} || 0^{B/2}]$ as follows:

$$\tilde{m}(x_0, \dots, x_{\beta-1}, y_0, \dots, y_{\tau-1}) = 1 - x_{\beta-1}$$

2. To verify that the integers encoded in $\mathbf{v}$ and $\mathbf{q}$ have bit-length at most $B/2$, $\mathcal{P}$ outputs two witnesses for the Zero Check relation (Definition A.6) as $w'_{\text{Zero},v} := f_v, w'_{\text{Zero},q} := f_q$ while $\mathcal{V}$ outputs the corresponding explicit and implicit instances for the Zero Check relation as $(x'_{\text{Zero},v}, y'_{\text{Zero},v}) := (\emptyset, \llbracket f_v \rrbracket)$ and $(x'_{\text{Zero},q}, y'_{\text{Zero},q}) := (\emptyset, \llbracket f_q \rrbracket)$ where $f_v \in \mathbb{F}_{p\beta+\tau}^{\text{ind} \leq 2}$ and $f_q \in \mathbb{F}_{p\beta+\tau}^{\text{ind} \leq 2}$ are virtual polynomials defined as

$$f_v(\mathbf{x}, \mathbf{y}) := \tilde{m}(\mathbf{x}, \mathbf{y}) \cdot \tilde{v}(\mathbf{x}, \mathbf{y}), \quad f_q(\mathbf{x}, \mathbf{y}) := \tilde{m}(\mathbf{x}, \mathbf{y}) \cdot \tilde{q}(\mathbf{x}, \mathbf{y})$$

3. $\mathcal{P}$ computes the NTT of $\mathbf{v}$, and $\mathbf{q}$:

$$\mathbf{v}' := \text{NTT}(\mathbf{v}), \quad \mathbf{q}' := \text{NTT}(\mathbf{q})$$

and sends the oracles $\llbracket \tilde{v}' \rrbracket$, and $\llbracket \tilde{q}' \rrbracket$ to $\mathcal{V}$. To verify the correctness of the NTT computations, $\mathcal{P}$ outputs the witnesses for the NTT relation (Definition 4.12):

$$w'_{\text{NTT},v} := (\tilde{v}, \tilde{v}'), \quad w'_{\text{NTT},q} := (\tilde{q}, \tilde{q}')$$

Correspondingly, $\mathcal{V}$ outputs the associated instances and oracle instances for the NTT relation as

$$(x'_{\text{NTT},v}, y_{\text{NTT},v}) := (\emptyset, (\tilde{v}, \llbracket \tilde{v}' \rrbracket)), \quad (x'_{\text{NTT},q}, y_{\text{NTT},q}) := (\emptyset, (\tilde{q}, \llbracket \tilde{q}' \rrbracket))$$

4. To verify that the committed vectors are binary and therefore valid bit decompositions of integers, $\mathcal{P}$ outputs the following witnesses for the binary relation (Definition A.2):

$$w'_{\text{Bin},v} := \tilde{v}, \quad w'_{\text{Bin},q} := \tilde{q},$$

Correspondingly, $\mathcal{V}$ outputs the associated instances and oracle instances for the binary relation as

$$(x'_{\text{Bin},v}, y'_{\text{Bin},v}) := (\emptyset, \llbracket \tilde{v} \rrbracket), \quad (x'_{\text{Bin},q}, y'_{\text{Bin},q}) := (\emptyset, \llbracket \tilde{q} \rrbracket)$$

5. To verify the division relation in the NTT domain, $\mathcal{P}$ outputs a witness for the Hadamard relation (Definition A.4) as

$$w'_{\text{Had}} := (\tilde{q}', \tilde{\ell}', \tilde{v}', \tilde{r}')$$

while $\mathcal{V}$ outputs the corresponding instance and oracle instance as

$$(x'_{\text{Had}}, y'_{\text{Had}}) := (\emptyset, (\llbracket \tilde{q}' \rrbracket, \llbracket \tilde{\ell}' \rrbracket, \llbracket \tilde{v}' \rrbracket, \llbracket \tilde{r}' \rrbracket)).$$

Theorem 4.17. PIOR 7 has perfect completeness and round-by-round soundness error 0 with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
0	0	2	$O(Bt)$	$O(\gamma \log \gamma)$

Proof. Deferred to Appendix A.4.6. □

4.2.7 MaSDAK: Magnitude-Succinct Diophantine Argument of Knowledge

We present Masdak, a public-coin argument of knowledge for proving knowledge of bounded, positive integer solutions to Diophantine equations. This protocol serves as a core component of our relaxed mod-PC and mod-PC constructions. Masdak is magnitude-succinct, meaning that the proof size and verification time scale sublinearly in the bit-length of the integer solutions being proven. While Masdak is not succinct in the length of the full witness, we only invoke it for Diophantine equations of constant arity, ensuring that the overall proof size and verification time remain sublinear in the bit-length of the integer solutions.

Definition 4.18. We define $\mathfrak{R}_{\text{Masdak-Oracle}}$ as the set of all triples of the form $(x, y, w) = ((t, f), \llbracket \tilde{u} \rrbracket, \tilde{u})$ where $f : \mathbb{Z}_B^t \rightarrow \mathbb{Z}_B$ is a Diophantine function of arity $t \in \mathbb{N}$. The witness contains a multilinear polynomial $\tilde{u}$ with a binary evaluation vector $\mathbf{u} \in \{0, 1\}^{tB}$ such that $f(\mathbf{v}) = 0$, where each integer input is reconstructed from its bit decomposition as $v[i] = \sum_{j=0}^{B-1} 2^j \mathbf{u}[in + j]$. I.e., the relation asserts that the reconstructed integer vector $\mathbf{v}$ satisfies the Diophantine equation $f(\mathbf{v}) = 0$.

Moreover, we denote by $\mathfrak{R}_{\text{Masdak}}$ as the dual relation of $\mathfrak{R}_{\text{Masdak-Oracle}}$ according to Definition 4.3.

PIOR 7: $\mathfrak{R}_{\text{Masdak-Oracle}} \rightarrow \mathfrak{R}_{\text{Div}} \times \mathfrak{R}_{\text{Decision}}$

Parse: $x = (t, f), y = \llbracket \tilde{u} \rrbracket, w = \tilde{u} \langle \mathcal{P}(i, x, w), \mathcal{V}(x) \rangle$

1. $\mathcal{V}$ samples a random prime modulus $\ell \in \text{Primes}[\gamma]$ and sends it to the prover.
2. $\mathcal{P}$ computes the quotients $\mathbf{q} \in \mathbb{Z}_{B-\lambda}^t$ and remainders $\mathbf{r} \in \mathbb{Z}_\lambda^t$ s.t. the division equations hold; i.e. $\mathbf{u} = \ell \cdot \mathbf{q} + \mathbf{r}$.
3. $\mathcal{P}$ sends the plaintext $\mathbf{r}$ and the oracle $\llbracket \mathbf{q} \rrbracket$ to $\mathcal{V}$.
4. $\mathcal{V}$ checks if the Diophantine equation holds mod ℓ ; i.e. $f(\mathbf{r}) \stackrel{?}{=} 0 \pmod{\ell}$.
5. $\mathcal{P}$ outputs a witness for the decision relation (Definition 3.10) as $w'_{\text{Decision}} := \emptyset$ while $\mathcal{V}$ outputs the corresponding instance for the decision relation as $(x'_{\text{Decision}}, y'_{\text{Decision}}) := (b, \emptyset)$ where b is accept if the check in Step 4 passes and b is reject otherwise.
6. Finally, $\mathcal{P}$ outputs the witness for the division relation as $w'_{\text{Div}} = (\tilde{v}, \tilde{q})$ while $\mathcal{V}$ outputs the corresponding instance and oracle instance for the division relation as $(x'_{\text{Div}}, y'_{\text{Div}}) := (\gamma, (\llbracket \tilde{v} \rrbracket, \llbracket \tilde{q} \rrbracket, \tilde{\ell}, \tilde{r}))$.

Theorem 4.19. Let γ be the bit-length of the prime modulus sampled by the verifier. PIOR 8 has perfect completeness and round-by-round soundness error at most $2^{-\gamma}$. Moreover, let $M(B)$ be the complexity of performing B -bit integer multiplication, then PIOR 8 has the following efficiency measures:

rounds	Message size	queries	oracle size	prover time	verifier time
1	$t\gamma$	0	$2Bt$	$O(t M(B))$	$O(t\gamma)$

Proof. Deferred to Appendix A.4.7. □

Remark 4.20. Masdak is closely related in spirit to the PoKEMath protocol introduced in DewTwo [BMSS25] and can be viewed as a generalization of DewTwo: the latter operates in groups of unknown order and supports arbitrary-length integer solutions, whereas Masdak works over a generic ICS and leverages the division checks supported by that ICS.

PIOP	rounds	queries	oracle size	prover time	verifier time
Addition (Section 4.2.1)	$\beta + \tau + 2$	$3\beta + 3\tau + 8$	Bt	Bt	$\beta + \tau$
Range (Section 4.2.2)	$\beta + \tau + 3$	$3\beta + 3\tau + 8$	$2Bt$	Bt	$\beta + \tau$
Int. Mul. (Section 4.2.5)	$\beta + \tau + 1$	$11\beta + 8\tau + 15$	$3Bt$	$tB \log B$	$\beta + \tau$
MaSDAK (Section 4.2.7)	$\beta + \tau + 2$	$8\beta + 6\tau + 12$	$3Bt$	$tB \log B$	$t\gamma + \beta + \tau + \gamma \log \gamma$

$B = 2^\beta$: The maximum bit-length of the integers

$t = 2^\tau$: The number of integers in the witness (assumption $\tau \geq 2$)

γ : The bit-length of the prime modulus sampled by the verifier in Masdak

Table 4: Summary of the efficiency measures for the assembled PIOPs by the PIORs presented in Section 4. Big-O notation is omitted for clarity.

SNARK	proof size	prover time	verifier time
Addition	$\beta + \tau$	Bt	$\beta + \tau$
Range	$\beta + \tau$	Bt	$\beta + \tau$
Int. Mul.	$\beta + \tau$	$tB \log B$	$\beta + \tau$
MaSDAK	$t\lambda + \beta + \tau$	$tB \log B$	$t\lambda + \beta + \tau + \lambda \log \lambda$

λ : The security parameter, $B = 2^\beta$: The maximum bit-length of the integers

$t = 2^\tau$: The number of integers in the witness (assumption $\tau \geq 2$)

Table 5: Summary of the efficiency of the assembled SNARKs obtained by instantiating the PIORs from Section 4 using the compiler of Construction 8. Big-O notation is omitted for clarity.

4.3 Summary of the costs

In this section, we assemble the efficiency measures of the end-to-end PIOPs and compiled arguments for all the PIORs presented in this section. To demonstrate the efficiency of the final SNARK constructions, we instantiate the costs with two PCS: 1. TensorSwitch [BFRW25] a state-of-the-art hash-based PCS with $O(\lambda N)$ prover time, $O(\lambda \log N)$ proof size, and $O(\lambda \log N)$ verifier time, and, 2. Mercury [EG25], the state of the art elliptic-curve based PCS with $O(N)$ prover time, $O(1)$ proof size, and $O(\log N)$ verifier time (Table 5).

5 $\mathbb{Q}\bar{a}ren$: A framework for building relaxed mod-pc

In this section, we present $\mathbb{Q}\bar{a}ren$, our generic framework for building relaxed mod-pcs (defined in Definition 3.27). For an intuitive overview of the construction, see Section 2.2.

Some notations. We denote by $\text{rev}(v)$ the vector obtained by reversing the elements of v . We also define $\lceil x \rceil_2$ for an integer x which represents the closest power of 2 larger than or equal to x (we use this as our range proofs support bounds that are powers of 2).

5.1 Committing to an integer multilinear polynomial

According to Definition 3.22, we first need to construct a commitment scheme for integer multilinear polynomials. Our commitment scheme relies on encoding a μ -variate polynomial $p \in \mathbb{Z}_B[X_1, \dots, X_\mu]$ as an integer $e_{p,\alpha} \in \mathbb{Z}$ which is described next:

Algorithm 1: Integer Encoding and Decoding

- IntegerEncode(p, α) $\rightarrow e_{p,\alpha}$ where $p \in \mathbb{Z}_B[X_1, \dots, X_\mu]$, $e_{p,\alpha} \in \mathbb{Z}$ and $\alpha \in \mathbb{Z}^+$
1. Convert the polynomial p into a vector of coefficients $\mathbf{p} = [p_0, \dots, p_{N-1}] \in \mathbb{Z}_B^N$.
 2. Assert $\|\mathbf{p}\|_\infty < \alpha$ to ensure injectivity of the encoding.
 3. Output $e_{p,\alpha} := \sum_{i=0}^{N-1} p_i \alpha^i$.
- IntegerDecode($e_{p,\alpha}, \alpha$) $\rightarrow p$ where $p \in \mathbb{Z}_B[X_1, \dots, X_\mu]$, $e_{p,\alpha} \in \mathbb{Z}$ and $\alpha \in \mathbb{Z}^+$
1. Initialise an empty vector $\mathbf{p}$ to hold the digits.
 2. Loop while $e_{p,\alpha} \neq 0$:
 - (a) Compute a pair (q, r) such that $e_{p,\alpha} = q\alpha + r$ and $0 \leq r < |\alpha|$.
 - (b) Append r to $\mathbf{p}$ and update $e_{p,\alpha} := q$.
 3. Output $\mathbf{p}$.
-

Next, We describe our commitment scheme for integer multilinear polynomials which first encodes the polynomial as an integer using the above encoding and then commits to the resulting integer using an integer commitment scheme ICS (see Definition 4.1).

Construction 2: Q̄aren commitment scheme

- Setup($1^\lambda, N, B$) $\rightarrow (\text{ck}, \text{vk})$
1. Invoke $(\text{ck}_{\text{ICS}}, \text{vk}_{\text{ICS}}) := \text{ICS.Setup}(1^\lambda, B, t)$
 2. Set the integer encoding base $\alpha > 2^{7B\mu}$ such that it has no prime factors smaller than 2^B .
 3. Output $\text{ck} := (\text{ck}_{\text{ICS}}, \alpha)$ and $\text{vk} := (\text{vk}_{\text{ICS}}, \alpha)$
-
- Commit(p, ck) $\rightarrow (\text{cm}; \text{opn})$
1. Compute the integer encoding of the polynomial as $e := \text{IntegerEncode}(p, \alpha)$.
 2. Compute the integer commitment $(\text{cm}_{\text{ICS}}, \text{opn}_{\text{ICS}}) := \text{ICS.Commit}(\text{ck}_{\text{ICS}}, e)$.
 3. Output $\text{cm}_{\text{PCS}} = \text{cm}_{\text{ICS}}$ and $\text{opn}_{\text{PCS}} = \text{opn}_{\text{ICS}}$.
-
- Open($\text{ck}, \text{cm}, p, \text{opn}$) $\rightarrow b$
1. Parse ck as ck_{ICS} , cm as cm_{ICS} , and opn as opn_{ICS} .
 2. Regenerate the integer encoding of the polynomial $e := \text{IntegerEncode}(p)$.
 3. Perform the opening for ICS and output $b := \text{ICS.Open}(\text{ck}_{\text{ICS}}, \text{cm}_{\text{ICS}}, e, \text{opn}_{\text{ICS}})$.
-

Remark 5.1 (Comparison with KZG). Construction 3 is reminiscent of the classic approach of KZG [KZG10], where a polynomial is first evaluated at a point and the resulting value is then committed using a standard commitment scheme. Nevertheless, there are several key differences: (a) In KZG, all computations take

place over a finite field, whereas our construction operates over bounded integers $\mathbb{Z}_B$, which form a finite ring. (b) In KZG, the scalar encoding of a polynomial is not injective. In contrast, we ensure injectivity by choosing the encoding base α to exceed the bound on the coefficients. (c) In KZG, the encoding base is secret and must be discarded as *toxic waste*. In our setting, the encoding base is public and forms part of the commitment key.

Next, we lift the commitment scheme to relaxed modular polynomial commitment scheme by augmenting it with an evaluation argument which we describe next.

5.2 A None-Adaptive Relaxed Mod-PC

Before presenting the constructions for evaluation argument, which is an argument for the evaluation relation defined in Definition 3.23, we introduce two helper algorithms that we use in the argument: First, Alg 3 which expands a μ -variate evaluation point $\mathbf{y} \in \mathbb{F}_q^\mu$ into a N -dimensional vector $\mathbf{y} \in \mathbb{Z}_B^N$ representing all possible monomials of a multilinear polynomial evaluated at this point.

Algorithm 2: Point to Coefficient Vector

PointToCoefficients($\mathbf{y}$) $\rightarrow \mathbf{y}$

1. For all $i \in \{0, \dots, N-1\}$: compute $\mathbf{y}_i = \prod_{j=0}^{\mu-1} \mathbf{y}_j^{i_j}$ where i_j is the j -th bit of the binary representation of i and the product is over integers.
 2. Output the monomial expansion $\mathbf{y} = [\mathbf{y}_i]_{i=0}^{N-1}$.
-

Second, Alg 4, which splits an integer w at index N into three integers l, m, r in base α .

Algorithm 3: Three-way Decomposition

ThreeWayDecomp(w, α, N) $\rightarrow (l, m, r)$

1. Let $\mathbf{c} = (c_0, \dots, c_m)$ represent w in base α where $m := \lceil \log_\alpha |w| \rceil$.
2. Output (l, m, r) where

$$w = \sum_{j=0}^m c_j \alpha^j = \sum_{j=0}^{N-2} c_j \alpha^j + c_{N-1} \alpha^{N-1} + \sum_{j=N}^m c_j \alpha^j = l + m \alpha^{N-1} + r \alpha^N$$

Our evaluation argument, both for adaptive and non-adaptive extractability, relies on repeating a core sub-protocol. Intuitively, this core protocol only guarantees that the extracted polynomial satisfies the evaluation relation modulo α which is the encoding base. Obviously, this guarantee is not sufficient for a relaxed mod-PC. Later, we show that by running the core protocol twice with different encoding bases, we can ensure that the extracted polynomial satisfies the evaluation relation over the random modulus q , which is sufficient for a relaxed mod-PC.

Moreover, we present the construction only for a single evaluation claim (i.e., $\ell = 1$), and the evaluation map φ set to mod q for a randomly sampled modulo q and therefore simplify the notation accordingly compared to Definition 3.23.

Definition 5.2. We define $\mathcal{R}_{CoreEval}$ as the set of all tuples of the form $(i, \mathbf{x}, \mathbf{w}) = (\text{ck}, \text{vk}, (q, \text{cm}, \mathbf{y}, z), (p, \text{opn}))$, where ck, vk contains commitment and verification keys for the PCS, and $\mathbf{x}, \mathbf{w}$ satisfy $z = p(\mathbf{y}) \bmod \alpha$ and $\text{PCS.Open}(\text{ck}, \text{cm}, p, \text{opn}) = 1$.

Construction 3: $\text{ROK}_{\text{CoreEval}} : \mathfrak{R}_{\text{CoreEval}} \rightarrow \mathfrak{R}_{\text{Masdak}} \times \mathfrak{R}_{\text{Range}}$

Parse: $i = (\text{ck}, \text{vk}), x = (q, \text{cm}, \mathbf{y}, z), w = (p, \text{opn})$

1. The prover parses $\text{pk} := \text{ck}_{\text{PCS}}$ where $\text{ck}_{\text{PCS}} := (\text{ck}_{\text{ICS}}, \alpha)$, $w := (p, \text{opn})$, verifier parses $\text{vk} := \text{vk}_{\text{PCS}}$ where $\text{vk}_{\text{PCS}} := (\text{vk}_{\text{ICS}}, \alpha)$, and both parse $x := (q, \text{cm}, \mathbf{y}, z)$.
2. $\mathcal{P}$ and $\mathcal{V}$ compute $\mathbf{y} := \text{PointToCoefficients}(\mathbf{y})$ where $\mathbf{y} \in \mathbb{Z}_B^N$.
3. $\mathcal{P}$ computes the integer encoding of the polynomial and the coefficient vector as follows

$$e_{p,\alpha} := \text{IntegerEncode}(p, \alpha), \quad e_{\mathbf{y}} := \text{IntegerEncode}(\text{rev}(\mathbf{y}), \alpha)$$

and computes the integer $w := e_{p,\alpha} \cdot e_{\mathbf{y}}$.

4. $\mathcal{P}$ decomposes the base- $e_{\mathbf{y}}$ representation of integer w into left, middle and right integers at the index N ; i.e.

$$(l, m, r) := \text{ThreeWayDecomp}(w, \alpha, N)$$

and computes $m = z + hq$, where $z < q$. Then it forms the vector of integers $\mathbf{u} = [l, m, r, h]$ which we prove some properties about in the next steps.

5. $\mathcal{P}$ commits to the integers $\mathbf{u}$ by computing $\text{cm}_{\mathbf{u}} = \text{ICS.Commit}(\mathbf{u}, \text{ck}_{\text{ICS}})$.
6. To check that the committed integers satisfy the diophantine equations f_1 and f_2 which are defined as follows

$$\begin{aligned} f_1(l, m, r, h) &= l + m\alpha^N + r\alpha^{N+1} - e_{p,\alpha} \cdot e_{\mathbf{y}} \\ f_2(l, m, r, h) &= m - z + hq \end{aligned}$$

$\mathcal{P}$ outputs the witness $w'_{\text{Masdak}} := \mathbf{u}$ and $\mathcal{V}$ outputs the instance $(x, y) = ((4, [f_1, f_2]), \text{cm}_{\mathbf{u}})$.

7. To check that the committed integers are in the appropriate ranges, $\mathcal{P}$ outputs the witness $w'_{\text{Range}} := \mathbf{u}$ and $\mathcal{V}$ outputs the instance

$$(x'_{\text{Range}}, y'_{\text{Range}}) = ((\lceil N2^{B(\mu+1)}\alpha^{N-2} \rceil_2, \lceil N2^{B(\mu+1)} \rceil_2, \lceil 2^{B(\mu+1)}\alpha^{N-2} \rceil_2, \perp), \text{cm}_{\mathbf{u}})$$

Next, we construct the full non-adaptive evaluation argument which relies on “Commitment Switching” technique as intuitively described in Section 2.4. The construction simply invokes the core evaluation argument twice, first with the original commitment and encoding base α , and then with a switched commitment and encoding base β which is a random multiple of α .

We define the non-adaptive evaluation relation first, and reduce it to two instances of the core evaluation relation. Note that q and $\mathbf{y}$ are randomly sampled inside the construction (steps 2 and 4), causing the non-adaptive nature of the argument.

Definition 5.3. We define $\mathfrak{R}_{\text{EvalNonAdaptive}}$ as the set of all tuples of the form $(i, x, w) = (\text{ck}, \text{vk}, (\text{cm}), (p, \text{opn}))$, where ck, vk contains commitment and verification keys for the PCS, and x, w satisfy $\text{PCS.Open}(\text{ck}, \text{cm}, p, \text{opn}) = 1$.

Construction 4: $\text{ROK}_{\text{EvalNonAdaptive}} : \mathfrak{R}_{\text{EvalNonAdaptive}} \rightarrow \mathfrak{R}_{\text{CoreEval}}^2$

Parse: $i = (\text{ck}, \text{vk}), x = (\text{cm}), w = (p, \text{opn})$

1. The prover parses $\text{pk} := \text{ck}_{\text{PCS}}$ where $\text{ck}_{\text{PCS}} := (\text{ck}_{\text{ICS}}, \alpha)$ and the verifier parses $\text{vk} := \text{vk}_{\text{PCS}}$ where $\text{vk}_{\text{PCS}} := (\text{vk}_{\text{ICS}}, \alpha)$.
2. $\mathcal{P}$ and $\mathcal{V}$ sample a random prime $q > 2^B$ and compute $\beta := q\alpha$ and $\text{pk}' := (\text{ck}_{\text{ICS}}, \beta)$ and $\text{vk}' := (\text{vk}_{\text{ICS}}, \beta)$.
3. $\mathcal{P}$ computes $\text{cm}' := \text{PCS.Commit}(p, \text{pk}')$.

4. $\mathcal{P}$ and $\mathcal{V}$ sample a random $\mathbf{y} \in \mathbb{Z}_B$.
5. $\mathcal{P}$ computes $z := p(\mathbf{y}) \bmod q$.
6. $\mathcal{P}$ outputs the witness $w'_\alpha := (p, \text{opn})$ and $\mathcal{V}$ outputs the instance $(x'_\alpha, y'_\alpha) = ((q, \text{cm}, \mathbf{y}, z), \perp)$ for $\mathfrak{R}_{\text{CoreEval}}$ with keys (pk, vk) .
7. $\mathcal{P}$ outputs the witness $w'_\beta := (p, \text{opn})$ and $\mathcal{V}$ outputs the instance $(x'_\beta, y'_\beta) = ((q, \text{cm}', \mathbf{y}, z), \perp)$ for $\mathfrak{R}_{\text{CoreEval}}$ with keys (pk', vk') .

Theorem 5.4. Construction 5 is a reduction of knowledge from $\mathfrak{R}_{\text{EvalNonAdaptive}}$ to $\mathfrak{R}_{\text{CoreEval}}$ ². Consequently, composing it with $\text{ROK}_{\text{CoreEval}}$ (Construction 4) yields a relaxed mod-PC (see Def 3.27) with non-adaptive extractability (see Def. 3.25). See Section 5.3 for strengthening this to adaptive extractability. Additionally, we have the following asymptotic efficiency measures (as a function of the sub-protocols used);

- Prover time is bounded as $O(N) + T_P(\text{CS.Commit}(N2^{B(\mu+1)}\alpha^{N-2})) + T_P(\text{Masdak}(4, [f_1, f_2], \text{cm}_u)) + T_P(\text{Range}(N2^{B(\mu+1)}\alpha^{N-2}))$
- Verifier time is $O(\mu) + T_V(\text{Masdak}(4, [f_1, f_2], \text{cm}_u)) + T_V(\text{Range}(N2^{B(\mu+1)}\alpha^{N-2}))$,
- Proof size is $O(1) + \pi(\text{Masdak}(4, [f_1, f_2], \text{cm}_u)) + \pi(\text{Range}(N2^{B(\mu+1)}\alpha^{N-2}))$

where $T_P(\text{CS.Commit}(n))$ denotes the time taken to commit to an integer of size n , $T_P(\text{Masdak}(t, \mathbf{f}, \text{cm}))$ denotes the prover time for the Masdak protocol (see Theorem 4.19) on t integers with Diophantine equations $\mathbf{f}$ and commitment cm , and $T_P(\text{Range}(t, n))$ denotes the time taken to produce a range proof of t integers of size at most n . The other quantities are similarly defined with the subscript V denoting verifier time and $\pi(\cdot)$ denoting proof sizes.

We defer the proof to Appendix B. By instantiating the polynomial commitment schemes with (Tensor-switch) TS and (Mercury) MC, we get the following concrete efficiency metrics for our construction.

Variant	Prover time	Verifier time	Proof size
$\text{ROK}_{\text{EvalTS}}$	$B \log N + N \log B \log N$	$\log(B + N)$	$\log(B + N)$
$\text{ROK}_{\text{EvalMC}}$	$B \log N + N \log B \log N$	$\log(B + N)$	$\log(B + N)$

Figure 2: Efficiency metrics for ROK_{Eval} when instantiated with different polynomial commitment schemes.

5.3 Lifting to Adaptive Extractability

Obtaining an adaptive PCS from a non-adaptive PCS just requires running the protocol again with an arbitrary evaluation point, as described in [AGLMS23; BMSS25]. This is because the non-adaptive PCS constructed in Theorem 5.4 proves that the underlying committed polynomial satisfies some structure, i.e., all the coefficients are bounded rationals. Now, the third call to the core protocol with the externally provided evaluation point will prove that the underlying polynomial evaluates to the claimed value at that point (without any soundness issues, since the first two calls already established the structural properties of the polynomial).

We define the adaptive evaluation relation, which additionally takes an externally chosen evaluation point $\hat{y}$ and claimed evaluation $\hat{z}$ as part of the instance.

Definition 5.5. We define $\mathfrak{R}_{\text{EvalAdaptive}}$ as the set of all tuples of the form $(i, x, w) = (\text{ck}, \text{vk}, (\text{cm}, \hat{y}, \hat{z}), (p, \text{opn}))$, where ck, vk contains commitment and verification keys for the PCS, and x, w satisfy $\hat{z} = p(\hat{y}) \bmod q$ and $\text{PCS.Open}(\text{ck}, \text{cm}, p, \text{opn}) = 1$.

Construction 5: $\text{ROK}_{\text{EvalAdaptive}} : \mathfrak{R}_{\text{EvalAdaptive}} \rightarrow \mathfrak{R}_{\text{CoreEval}}^3$

Parse: $i = (\text{ck}, \text{vk})$, $x = (\text{cm}, \hat{y}, \hat{z})$, $w = (p, \text{opn})$

1. The prover parses $\text{pk} := \text{ck}_{\text{PCS}}$ where $\text{ck}_{\text{PCS}} := (\text{ck}_{\text{ICS}}, \alpha)$ and the verifier parses $\text{vk} := \text{vk}_{\text{PCS}}$ where $\text{vk}_{\text{PCS}} := (\text{vk}_{\text{ICS}}, \alpha)$.
2. $\mathcal{P}$ and $\mathcal{V}$ sample a random prime $q > 2^B$ and compute $\beta := q\alpha$ and $\text{pk}' := (\text{ck}_{\text{ICS}}, \beta)$ and $\text{vk}' := (\text{vk}_{\text{ICS}}, \beta)$.
3. $\mathcal{P}$ computes $\text{cm}' := \text{PCS.Commit}(p, \text{pk}')$.
4. $\mathcal{P}$ and $\mathcal{V}$ sample a random $y \in \mathbb{Z}_B$.
5. $\mathcal{P}$ computes $z := p(y) \bmod q$.
6. $\mathcal{P}$ outputs the witness $w'_\alpha := (p, \text{opn})$ and $\mathcal{V}$ outputs the instance $(x'_\alpha, y'_\alpha) = ((q, \text{cm}, y, z), \perp)$ for $\mathfrak{R}_{\text{CoreEval}}$ with keys (pk, vk) .
7. $\mathcal{P}$ outputs the witness $w'_\beta := (p, \text{opn})$ and $\mathcal{V}$ outputs the instance $(x'_\beta, y'_\beta) = ((q, \text{cm}', y, z), \perp)$ for $\mathfrak{R}_{\text{CoreEval}}$ with keys (pk', vk') .
8. $\mathcal{P}$ outputs the witness $w'_y := (p, \text{opn})$ and $\mathcal{V}$ outputs the instance $(x'_y, y'_y) = ((q, \text{cm}, \hat{y}, \hat{z}), \perp)$ for $\mathfrak{R}_{\text{CoreEval}}$ with keys (pk, vk) .

Corollary 5.6. Construction 6 is a reduction of knowledge from $\mathfrak{R}_{\text{EvalAdaptive}}$ to $\mathfrak{R}_{\text{CoreEval}}^3$. Consequently, composing it with $\text{ROK}_{\text{CoreEval}}$ (Construction 4) yields a relaxed mod-PC (see Def 3.27) with adaptive extractability. The asymptotic efficiency measures are the same as in Theorem 5.4.

The proof of this corollary follows from the same arguments as those in Theorem 5.7 in [BMSS25].

6 Acknowledgements

We thank Pratyush Mishra, Benedikt Bünz, Shravan Srinivasan, Christodoulos Pappas, Tushar Mopuri, and Anubhav Baweja for helpful discussions.

References

- [ACFY25] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev. “WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification”. In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*. Ed. by S. Fehr and P. Fouque. Vol. 15604. Lecture Notes in Computer Science. Springer, 2025, pp. 214–243. DOI: 10.1007/978-3-031-91134-7_8.
- [AGLMS23] A. Arun, C. Ganesh, S. V. Lokam, T. Mopuri, and S. Sridhar. “Dew: A Transparent Constant-Sized Polynomial Commitment Scheme”. In: *Public-Key Cryptography - PKC 2023 - 26th IACR International Conference on Practice and Theory of Public-Key Cryptography, Atlanta, GA, USA, May 7-10, 2023, Proceedings, Part II*. Ed. by A. Boldyreva and V. Kolesnikov. Vol. 13941. Lecture Notes in Computer Science. Springer, 2023, pp. 542–571. DOI: 10.1007/978-3-031-31371-4_19.
- [BBBPWM18] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. “Bulletproofs: Short Proofs for Confidential Transactions and More”. In: *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*. IEEE Computer Society, 2018, pp. 315–334. DOI: 10.1109/SP.2018.00020.

- [BCFRRZ25] M. Brehm, B. Chen, B. Fisch, N. Resch, R. D. Rothblum, and H. Zeilberger. “Blaze: Fast SNARKs from Interleaved RAA Codes”. In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*. Ed. by S. Fehr and P. Fouque. Vol. 15604. Lecture Notes in Computer Science. Springer, 2025, pp. 123–152. DOI: 10.1007/978-3-031-91134-7_5.
- [BCHO22] J. Bootle, A. Chiesa, Y. Hu, and M. Orrù. “Gemini: Elastic SNARKs for Diverse Environments”. In: *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II*. Ed. by O. Dunkelman and S. Dziembowski. Vol. 13276. Lecture Notes in Computer Science. Springer, 2022, pp. 427–457. DOI: 10.1007/978-3-031-07085-3_15.
- [BCKL22] E. Ben-Sasson, D. Carmon, S. Kopparty, and D. Levit. “Scalable and Transparent Proofs over All Large Fields, via Elliptic Curves - (ECFFT Part II)”. In: *Theory of Cryptography - 20th International Conference, TCC 2022, Chicago, IL, USA, November 7-10, 2022, Proceedings, Part I*. Ed. by E. Kiltz and V. Vaikuntanathan. Vol. 13747. Lecture Notes in Computer Science. Springer, 2022, pp. 467–496. DOI: 10.1007/978-3-031-22318-1_17.
- [Bes25] M. Besier. “Common Circom Pitfalls and How to Dodge Them — Part 1”. Blog post, zkSecurity. Discusses common pitfalls such as aliasing bugs and under-constrained signals in Circom circuits. 2025.
- [BF23] B. Bünz and B. Fisch. “Multilinear Schwartz-Zippel Mod N and Lattice-Based Succinct Arguments”. In: *Theory of Cryptography - 21st International Conference, TCC 2023, Taipei, Taiwan, November 29 - December 2, 2023, Proceedings, Part III*. Ed. by G. N. Rothblum and H. Wee. Vol. 14371. Lecture Notes in Computer Science. Springer, 2023, pp. 394–423. DOI: 10.1007/978-3-031-48621-0_14.
- [BFKWTWZ24] A. R. Block, Z. Fang, J. Katz, J. Thaler, H. Waldner, and Y. Zhang. “Field-Agnostic SNARKs from Expand-Accumulate Codes”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*. Ed. by L. Reyzin and D. Stebila. Vol. 14929. Lecture Notes in Computer Science. Springer, 2024, pp. 276–307. DOI: 10.1007/978-3-031-68403-6_9.
- [BFRW25] B. Bünz, G. Fenzi, R. D. Rothblum, and W. Wang. “TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 2065.
- [BFS20] B. Bünz, B. Fisch, and A. Szeponiec. “Transparent SNARKs from DARK Compilers”. In: *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*. Ed. by A. Canteaut and Y. Ishai. Vol. 12105. Lecture Notes in Computer Science. Springer, 2020, pp. 677–706. DOI: 10.1007/978-3-030-45721-1_24.
- [BHRRS21] A. R. Block, J. Holmgren, A. Rosen, R. D. Rothblum, and P. Soni. “Time- and Space-Efficient Arguments from Groups of Unknown Order”. In: *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part IV*. Ed. by T. Malkin and C. Peikert. Vol. 12828. Lecture Notes in Computer Science. Springer, 2021, pp. 123–152. DOI: 10.1007/978-3-030-84259-8_5.
- [BMMS25] A. Baweja, P. Mishra, T. Mopuri, and M. Shtepel. “FICS and FACS: Fast IOPPs and Accumulation via Code-Switching”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 737.
- [BMNW25] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. “Arc: Accumulation for Reed-Solomon Codes”. In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VII*. Ed. by Y. T. Kalai and S. F. Kamara. Vol. 16006. Lecture Notes in Computer Science. Springer, 2025, pp. 128–160. DOI: 10.1007/978-3-032-01907-3_5.

- [BMSS25] B. Bünz, T. Mopuri, A. Shirzad, and S. Sridhar. “DewTwo: A Transparent PCS with Quasi-Linear Prover, Logarithmic Verifier and 4.5KB Proofs from Falsifiable Assumptions”. In: *Advances in Cryptology – CRYPTO 2025*. Ed. by Y. Tauman Kalai and S. F. Kamara. Cham: Springer Nature Switzerland, 2025, pp. 549–583. ISBN: 978-3-032-01887-8.
- [Can+19] R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, R. D. Rothblum, and D. Wichs. “Fiat-Shamir: from practice to theory”. In: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019*. Ed. by M. Charikar and E. Cohen. ACM, 2019, pp. 1082–1090. DOI: 10.1145/3313276.3316380.
- [CBBZ23] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. “HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates”. In: *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II*. Ed. by C. Hazay and M. Stam. Vol. 14005. Lecture Notes in Computer Science. Springer, 2023, pp. 499–530. DOI: 10.1007/978-3-031-30617-4_17.
- [CETWJL24] S. Chaliasos, J. Ernstberger, D. Theodore, D. Wong, M. Jahanara, and B. Livshits. “SoK: what don’t we know? understanding security vulnerabilities in SNARKs”. In: *Proceedings of the 33rd USENIX Conference on Security Symposium. SEC ’24*. Philadelphia, PA, USA: USENIX Association, 2024. ISBN: 978-1-939133-44-1.
- [CFQ19] M. Campanelli, D. Fiore, and A. Querol. “LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*. Ed. by L. Cavallaro, J. Kinder, X. Wang, and J. Katz. ACM, 2019, pp. 2075–2092. DOI: 10.1145/3319535.3339820.
- [CH24] M. Campanelli and M. Hall-Andersen. “Fully-Succinct Arguments over the Integers from First Principles”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 1548.
- [CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. P. Ward. “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS”. In: *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*. Ed. by A. Canteaut and Y. Ishai. Vol. 12105. Lecture Notes in Computer Science. Springer, 2020, pp. 738–768. DOI: 10.1007/978-3-030-45721-1_26.
- [CL01] J. Camenisch and A. Lysyanskaya. “An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation”. In: *Advances in Cryptology - EUROCRYPT 2001, International Conference on the Theory and Application of Cryptographic Techniques, Innsbruck, Austria, May 6-10, 2001, Proceeding*. Ed. by B. Pfitzmann. Vol. 2045. Lecture Notes in Computer Science. Springer, 2001, pp. 93–118. DOI: 10.1007/3-540-44987-6_7.
- [CMNW24] V. Cini, G. Malavolta, N. K. Nguyen, and H. Wee. “Polynomial Commitments from Lattices: Post-quantum Security, Fast Verification and Transparent Setup”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*. Ed. by L. Reyzin and D. Stebila. Vol. 14929. Lecture Notes in Computer Science. Springer, 2024, pp. 207–242. DOI: 10.1007/978-3-031-68403-6_7.
- [CPP17] G. Couteau, T. Peters, and D. Pointcheval. “Removing the Strong RSA Assumption from Arguments over the Integers”. In: *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part II*. Ed. by J. Coron and J. B. Nielsen. Vol. 10211. Lecture Notes in Computer Science. 2017, pp. 321–350. DOI: 10.1007/978-3-319-56614-6_11.
- [CY24] A. Chiesa and E. Yogev. “Building Cryptographic Proofs from Hash Functions”. Online book. Available at <https://snargsbook.org/>. 2024.

- [DF02] I. Damgård and E. Fujisaki. “A Statistically-Hiding Integer Commitment Scheme Based on Groups with Hidden Order”. In: *Advances in Cryptology — ASIACRYPT 2002*. Ed. by Y. Zheng. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 125–142. ISBN: 978-3-540-36178-7.
- [DFGK14] G. Danezis, C. Fournet, J. Groth, and M. Kohlweiss. “Square Span Programs with Applications to Succinct NIZK Arguments”. In: *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014. Proceedings, Part I*. Ed. by P. Sarkar and T. Iwata. Vol. 8873. Lecture Notes in Computer Science. Springer, 2014, pp. 532–550. DOI: 10.1007/978-3-662-45611-8_28.
- [DMS24] M. Dellepère, P. Mishra, and A. Shirzad. “Garuda and Pari: Faster and Smaller SNARKs via Equiefficient Polynomial Commitments”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 1245.
- [EG25] L. Eagen and A. Gabizon. “MERCURY: A multilinear Polynomial Commitment Scheme with constant proof size and no prover FFTs”. *Cryptology ePrint Archive*, Paper 2025/385. 2025.
- [EMBB17] S. Eskandarian, E. Messeri, J. Bonneau, and D. Boneh. “Certificate Transparency with Privacy”. In: *Proc. Priv. Enhancing Technol.* 2017.4 (2017), pp. 329–344. DOI: 10.1515/POPETS-2017-0052.
- [FO97] E. Fujisaki and T. Okamoto. “Statistical Zero Knowledge Protocols to Prove Modular Polynomial Relations”. In: *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*. Ed. by B. S. K. Jr. Vol. 1294. Lecture Notes in Computer Science. Springer, 1997, pp. 16–30. DOI: 10.1007/BFB0052225.
- [Gar+25] A. Garreta, H. Waldner, I. Vlasov, K. Hristova, L. Dall’Ava, M. Čupić, and M. Klein. “Zinc: Succinct Arguments with Small Arithmetization Overheads from IOPs of Proximity to the Integers”. In: *Advances in Cryptology – CRYPTO 2025*. Ed. by Y. Tauman Kalai and S. F. Kamara. Cham: Springer Nature Switzerland, 2025, pp. 259–291. ISBN: 978-3-032-01907-3.
- [GGPR13] R. Gennaro, C. Gentry, B. Parno, and M. Raykova. “Quadratic Span Programs and Succinct NIZKs without PCPs”. In: *Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings*. Ed. by T. Johansson and P. Q. Nguyen. Vol. 7881. Lecture Notes in Computer Science. Springer, 2013, pp. 626–645. DOI: 10.1007/978-3-642-38348-9_37.
- [GKR08] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum. “Delegating Computation: Interactive Proofs for Muggles”. In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing, STOC 2008, Victoria, British Columbia, Canada, May 17-20, 2008*. STOC ’08. ACM, 2008, pp. 113–122. DOI: 10.1145/1374376.1374396.
- [GLSTW23] A. Golovnev, J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby. “Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS”. In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part II*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14082. Lecture Notes in Computer Science. Springer, 2023, pp. 193–226. DOI: 10.1007/978-3-031-38545-2_7.
- [GM17] J. Groth and M. Maller. “Snarky Signatures: Minimal Signatures of Knowledge from Simulation-Extractable SNARKs”. In: *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part II*. Ed. by J. Katz and H. Shacham. Vol. 10402. Lecture Notes in Computer Science. Springer, 2017, pp. 581–612. DOI: 10.1007/978-3-319-63715-0_20.
- [GNS23] C. Ganesh, A. Nitulescu, and E. Soria-Vazquez. “Rinocchio: SNARKs for Ring Arithmetic”. In: *J. Cryptol.* 36.4 (2023), p. 41. DOI: 10.1007/S00145-023-09481-3.
- [GPS25] C. Ganesh, S. Patranabis, and N. Singh. “Samaritan: Linear-time Prover SNARK from New Multilinear Polynomial Commitments”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 419.

- [Gro05] J. Groth. “Non-interactive Zero-Knowledge Arguments for Voting”. In: *Applied Cryptography and Network Security, Third International Conference, ACNS 2005, New York, NY, USA, June 7-10, 2005, Proceedings*. Ed. by J. Ioannidis, A. D. Keromytis, and M. Yung. Vol. 3531. Lecture Notes in Computer Science. 2005, pp. 467–482. DOI: 10.1007/11496137_32.
- [GWC19] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge”. In: *IACR Cryptol. ePrint Arch.* (2019), p. 953.
- [HMZ25] M. M. Huang, X. Mao, and J. Zhang. “Sublinear Proofs over Polynomial Rings”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 199.
- [KP23] A. Kothapalli and B. Parno. “Algebraic Reductions of Knowledge”. In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part IV*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14084. Lecture Notes in Computer Science. Springer, 2023, pp. 669–701. DOI: 10.1007/978-3-031-38551-3_21.
- [KSSL21] V. Kuchta, A. Sakzad, R. Steinfeld, and J. K. Liu. “Lattice-based zero-knowledge arguments for additive and multiplicative relations”. In: *Des. Codes Cryptogr.* 89.5 (2021), pp. 925–963. DOI: 10.1007/S10623-021-00851-1.
- [KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings*. Ed. by M. Abe. Vol. 6477. Lecture Notes in Computer Science. Springer, 2010, pp. 177–194. DOI: 10.1007/978-3-642-17373-8_11.
- [KZHB25] G. Kadianakis, A. Zapico, H. Hafezi, and B. Bünz. “KZH-Fold: Accountable Voting from Sublinear Accumulation”. In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*. Ed. by C. Huang, J. Chen, S. Shieh, D. Lie, and V. Cortier. ACM, 2025, pp. 409–422. DOI: 10.1145/3719027.3744796.
- [LFKN92] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan. “Algebraic Methods for Interactive Proof Systems”. In: *J. ACM* 39.4 (1992), pp. 859–868. DOI: 10.1145/146585.146605.
- [Lip03] H. Lipmaa. “On Diophantine Complexity and Statistical Zero-Knowledge Arguments”. In: *Advances in Cryptology - ASIACRYPT 2003, 9th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, November 30 - December 4, 2003, Proceedings*. Ed. by C. Lai. Vol. 2894. Lecture Notes in Computer Science. Springer, 2003, pp. 398–415. DOI: 10.1007/978-3-540-40061-5_26.
- [LLNW18] B. Libert, S. Ling, K. Nguyen, and H. Wang. “Lattice-Based Zero-Knowledge Arguments for Integer Relations”. In: *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part II*. Ed. by H. Shacham and A. Boldyreva. Vol. 10992. Lecture Notes in Computer Science. Springer, 2018, pp. 700–732. DOI: 10.1007/978-3-319-96881-0_24.
- [LNS20] V. Lyubashevsky, N. K. Nguyen, and G. Seiler. “Practical Lattice-Based Zero-Knowledge Proofs for Integer Relations”. In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by J. Ligatti, X. Ou, J. Katz, and G. Vigna. ACM, 2020, pp. 1051–1070. DOI: 10.1145/3372297.3417894.
- [Net25a] Nethermind. “Zinc Concrete: Costs and Experimental Results”. Online Notion page. Accessed: 2025-01-03. 2025.
- [Net25b] Nethermind Security Audits Team. “ZK Circuit Security: A Guide for Engineers and Architects”. Nethermind Blog. Accessed 2026-03-12. 2025.

- [NS24] N. K. Nguyen and G. Seiler. “Greyhound: Fast Polynomial Commitments from Lattices”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*. Ed. by L. Reyzin and D. Stebila. Vol. 14929. Lecture Notes in Computer Science. Springer, 2024, pp. 243–275. DOI: 10.1007/978-3-031-68403-6_8.
- [Pai+23] S. Pailoor, Y. Chen, F. Wang, C. Rodríguez, J. Van Geffen, J. Morton, M. Chu, B. Gu, Y. Feng, and I. Dillig. “Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs”. In: *Proc. ACM Program. Lang.* 7.PLDI (June 2023). DOI: 10.1145/3591282.
- [SB25] I. A. Seres and P. Burcsi. “Behemoth: Transparent Polynomial Commitment Scheme with Constant Opening Proof Size and Verifier Time”. In: *Progress in Cryptology - AFRICACRYPT 2025 - 16th International Conference on Cryptology in Africa, Rabat, Morocco, July 21-23, 2025, Proceedings*. Ed. by A. Nitaj, S. Petkova-Nikova, and V. Rijmen. Vol. 15651. Lecture Notes in Computer Science. Springer, 2025, pp. 343–369. DOI: 10.1007/978-3-031-97260-7_16.
- [Sch80] J. T. Schwartz. “Fast Probabilistic Algorithms for Verification of Polynomial Identities”. In: *Journal of the ACM* 27.4 (1980), pp. 701–717.
- [Set20] S. T. V. Setty. “Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup”. In: *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part III*. Ed. by D. Micciancio and T. Ristenpart. Vol. 12172. Lecture Notes in Computer Science. Springer, 2020, pp. 704–737. DOI: 10.1007/978-3-030-56877-1_25.
- [STW23] S. T. V. Setty, J. Thaler, and R. S. Wahby. “Customizable constraint systems for succinct arguments”. In: *IACR Cryptol. ePrint Arch.* (2023), p. 552.
- [TV20] P. Towa and D. Vergnaud. “Succinct Diophantine-Satisfiability Arguments”. In: *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part III*. Ed. by S. Moriai and H. Wang. Vol. 12493. Lecture Notes in Computer Science. Springer, 2020, pp. 774–804. DOI: 10.1007/978-3-030-64840-4_26.
- [WZD25] Y. Wei, X. Zhang, and Y. Deng. “Transparent SNARKs over Galois Rings”. In: *Public-Key Cryptography – PKC 2025*. Ed. by T. Jager and J. Pan. Cham: Springer Nature Switzerland, 2025, pp. 418–451. ISBN: 978-3-031-91820-9.
- [ZGX25] L. Zhang, S. Gao, and B. Xiao. “HyperWolf: Efficient Polynomial Commitment Schemes from Lattices”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 922.
- [Zip79] R. Zippel. “Probabilistic algorithms for sparse polynomials”. In: *Proceedings of the 1979 International Symposium on Symbolic and Algebraic Computation*. EUROSAM ’79. 1979, pp. 216–226.
- [ZSCZ25] J. Zhao, S. T. V. Setty, W. Cui, and G. Zaverucha. “MicroNova: Folding-Based Arguments with Efficient (On-Chain) Verification”. In: *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*. Ed. by M. Blanton, W. Enck, and C. Nita-Rotaru. IEEE, 2025, pp. 1964–1982. DOI: 10.1109/SP61157.2025.00168.

A PIOP Preliminaries

A.1 PIOR Product Composition

Here we provide the deferred proof for Theorem 3.9.

Proof.

$$\text{State}(x, y, \text{tr}) = \begin{cases} 0 & \text{if } \text{tr} = \emptyset \\ \text{State}_1(x, y, \text{tr}) & \text{if } |\text{tr}| \leq k_1 \\ \text{State}_j(x, y, \text{tr}) & \text{if } \sum_{k=1}^{j-1} k_k < |\text{tr}| \leq \sum_{k=1}^j k_k \end{cases}$$

First of all note that the above state function is a valid state function according to Definition 3.6; Because (a) It is zero on empty transcript unconditionally. (b) The prover moves condition holds because each sub-PIOR's state function satisfies the prover moves condition. (c) The full transcript condition holds. We prove this by contradiction. Assume that the full transcript condition does not hold. Then, there exists a full transcript tr such that $\text{State}(x, y, \text{tr}) = 0$ but the verifier outputs (x', y') such that $(x', y') \in \mathcal{L}(\mathfrak{R}')$. This means that for all $i \in (m]$, $(x', y') \in \mathcal{L}(\mathfrak{R}'_i)$. This means that for all $i \in (m]$, $\text{State}_i(x, y, \text{tr}_i) = 1$ where tr_i is the sub-transcript corresponding to the i -th sub-PIOR. This contradicts the assumption that $\text{State}(x, y, \text{tr}) = 0$.

Now, we analyze the round-by-round soundness error: If $\text{State}(x, y, \text{tr}) = 0$ for $(x, y) \notin \mathcal{L}(\mathfrak{R})$ and a partial transcript $\text{tr} = (\Pi_1, \varrho, \dots, \Pi_i)$, $i \in (k]$, where $k = \sum_{i \in (m_i]} k_i$ the verifier is about to move, then

$$\begin{aligned} \Pr_{\varrho_i \leftarrow \mathbb{R}^*} [\text{State}(x, y, \text{tr} || \varrho_i) = 1] &= \begin{cases} \epsilon_{\text{RBR}}^{(1)}(x) & \text{if } i \leq k_1 \\ \epsilon_{\text{RBR}}^{(j)}(x) & \text{if } \sum_{k=1}^{j-1} k_k < i \leq \sum_{k=1}^j k_k \end{cases} \\ &\leq \max_{i \in (m]} \epsilon_{\text{RBR}}^{(i)}(x) \end{aligned}$$

□

A.2 PIOR trivial extractor

We now state a useful lemma, adapted from [CBBZ23, Lemma 2.3], establishing that for all protocols considered in this paper, round-by-round knowledge soundness follows directly from round-by-round soundness.

Lemma A.1. Let PIOR be a generalized polynomial interactive oracle reduction (PIOR) for an indexed oracle relation $\mathfrak{R}$ with round-by-round soundness error ϵ_{RBR} and an identity evaluation map φ . Assume that for every $(x, y, w) \in \mathfrak{R}$, the witness w consists solely of polynomials and that the oracle instance y provides oracle access to these polynomials. Then PIOR satisfies round-by-round knowledge soundness with $\kappa_{\text{RBR}} = \epsilon_{\text{RBR}}$ and there exists an extractor that runs in time $O(|w|)$. We refer to relations satisfying the above property as *trivially extractable relations*.

Proof. We follow the same extraction idea as in the proof of [CBBZ23, Lemma 2.3], adapted to the round-by-round notion. [CBBZ23] (see also the proof sketch in [CBBZ23, pp. L10–L31]).

Extractor. Define an extractor $\mathcal{E}$ that, on input (x, y, Π, w') , ignores the transcript and reconstructs the witness polynomials directly from the oracle instance y . Concretely, for each witness polynomial f (which is promised to have an associated evaluation oracle $\llbracket f \rrbracket$ inside y), $\mathcal{E}$ queries $\llbracket f \rrbracket$ on sufficiently many points to recover a full description of f (e.g., its coefficient table in the standard monomial basis). Since φ is the identity, oracle answers are exact field evaluations of f .

If f is μ -variate with individual degree at most d , then f is uniquely determined by its evaluations on any set of $(d+1)^\mu$ points in $\mathbb{F}^\mu$ that forms a full interpolation grid (e.g., a Cartesian product of $(d+1)$ distinct values per coordinate). Thus the extractor can recover each f by multivariate interpolation from $(d+1)^\mu$ oracle queries. In particular, for multilinear f (the case used throughout this paper), it suffices to query all Boolean points, i.e., 2^μ points. Overall, the extractor runs in time linear in the size of the explicit representation of the witness polynomials, i.e., $O(|w|)$.

Why knowledge error equals soundness error. Fix any round i and any partial transcript tr ending right before the verifier samples ϱ_i , such that $\text{State}(x, y, \text{tr}) = 0$. Suppose

$$\Pr_{\varrho_i} [\text{State}(x, y, \text{tr} \parallel \varrho_i) = 1] > \epsilon_{\text{RBR}}(x).$$

By definition of round-by-round soundness, the above strict inequality can only occur when $(x, y) \in \mathfrak{L}(\mathfrak{R})$: otherwise it would contradict the soundness bound at this point.

Now assume $(x, y) \in \mathfrak{L}(\mathfrak{R})$. By the lemma's hypothesis, *every* valid witness for $\mathfrak{R}$ consists only of polynomials that already appear as oracles in y . Hence the extractor described above can reconstruct a candidate witness w^* *solely from* y , independently of how the transcript is continued. By construction, w^* is the unique tuple of polynomials consistent with the oracles in y (two distinct μ -variate individual-degree- d polynomials cannot agree on a full $(d+1)^\mu$ interpolation grid). Therefore, whenever the instance is in the language, w^* is a valid witness and satisfies $(x, y, w^*) \in \mathfrak{R}$.

This exactly matches the round-by-round knowledge-soundness requirement: whenever a prover can drive the state to 1 with probability exceeding $\epsilon_{\text{RBR}}(x)$ at some verifier move, the extractor outputs a valid witness for the original relation, and it does so for *any* transcript continuation. Thus $\kappa_{\text{RBR}} = \epsilon_{\text{RBR}}$.

Finally, we call relations satisfying the lemma's hypothesis *trivially extractable relations*. $\square$

A.3 From PIOP to Arguments of Knowledge

Construction 6: Argument of Knowledge from PIOP + PC

$(pk, vk) \leftarrow \text{Setup}(1^\lambda, i)$

1. Infer the maximum degree N of the polynomial oracles sent by the prover in the PIOP from i .
2. Sample the public parameters for the $(ck_{\text{PCS}}, vk_{\text{PCS}}) \leftarrow \text{PCS.Setup}(1^\lambda, N)$.
3. Output $pk := ck_{\text{PCS}}$ and $vk := vk_{\text{PCS}}$.

$b \leftarrow \langle \mathcal{P}(pk, x, w), \mathcal{V}(vk, x) \rangle$

1. $\mathcal{P}$ parses the proving key $pk = ck_{\text{PCS}}$ and $\mathcal{V}$ parses the verification key $vk = vk_{\text{PCS}}$.
2. For each round $i \in k(i)$ of the interactive phase of PIOP :
 - (a) $\mathcal{V}$ invokes $\mathcal{V}$ and receives public random challenges ϱ_i , then forwards them to $\mathcal{P}$.
 - (b) $\mathcal{P}$ forwards ϱ_i to $\mathcal{P}$ and receives oracle messages Π_i .
 - (c) $\mathcal{P}$ commits to polynomials in Π_i using the PCS:

$$cm_{\Pi_i} \leftarrow \text{PCS.Commit}(ck_{\text{PCS}}, \Pi_i)$$

and sends it to $\mathcal{V}$.

3. For the querying phase of PIOP:
 - (a) $\mathcal{V}$ invokes $\mathcal{V}$ and receives query randomness ϱ_{k+1} , then forwards it to $\mathcal{P}$.
 - (b) $\mathcal{P}$ uses the query algorithm of $\mathcal{V}$ to compute the query set $Q := \mathcal{V}_Q(x; \varrho_1, \dots, \varrho_{k+1})$

- (c) $\mathcal{P}$ replies to the queries by the evaluation $\nu := \Pi(Q)$.
- (d) $\mathcal{P}$ and $\mathcal{V}$ invoke the PCS evaluation argument for the relation

A.4 Deferred Proofs

A.4.1 Addition PIOR

Proof. PIOR 2 outputs four instance–witness pairs: one decision check and three binary checks. We analyze each induced PIOR and then apply product composition.

Decision check. Let

$$g(\mathbf{x}) := \tilde{a}(\mathbf{x}) + \tilde{b}(\mathbf{x}) + \tilde{k}(\mathbf{x}) - \tilde{c}(\mathbf{x}) - 2\tilde{k}(\text{Rot}_1(\mathbf{x}))$$

be the tested polynomial. Define the one-round state function

$$\text{State}_{\text{Decision}}(\text{tr}) = \begin{cases} 0 & \text{if } \text{tr} = \emptyset \\ 1 & \text{if } \text{tr} = (\mathbf{r}) \text{ and } g(\mathbf{r}) = 0 \\ 0 & \text{otherwise.} \end{cases}$$

Validity is immediate, and the escape probability equals $\Pr[g(\mathbf{r}) = 0]$. If the relation is false, then $g \neq 0$ and, by Lemma 4.6, g is multilinear, hence $\Pr[g(\mathbf{r}) = 0] \leq \mu/p$ by Lemma 3.1.

Binary checks. Each binary check is the (non-interactive) PIOR of Theorem A.3; the state functions are trivial and the escape probability is 0.

Product composition. By Theorem 3.9, the overall round-by-round soundness error is the maximum of the four errors, i.e. μ/p . □

A.4.2 Range PIOR

Proof. PIOR 3 is deterministic and outputs a fixed $\mathfrak{R}_{\text{Add-Oracle}}$ instance. There is no verifier randomness in the reduction, so a false range instance always maps to a false addition instance. Hence the state function is trivial and the probability of escaping the doomed state is 0. □

A.4.3 Lagrange PIOR

Proof. **Round-by-round soundness.** Let $f(\mathbf{x}, \mathbf{r})$ be the virtual polynomial defined in PIOR 4. If the claimed relation is false, then $f(\mathbf{x}, \mathbf{r})$ is not identically zero as a polynomial in $(\mathbf{x}, \mathbf{r})$. There exists a monomial in $\mathbf{x}$ whose coefficient is a nonzero polynomial in $\mathbf{r}$ of degree at most τ , since $\tilde{v}$ is multilinear in the $\mathbf{r}$ -variables. By Lemma 3.1, the probability over $\mathbf{r}$ that this coefficient vanishes (and hence $f(\cdot, \mathbf{r})$ is identically zero) is at most τ/p . Conditioned on $f(\cdot, \mathbf{r}) \neq 0$, the ZeroCheck instance is false and is accepted with probability at most β/p by Theorem A.7 (the individual degree in the $\mathbf{x}$ variables is at most 2). Thus the overall round-by-round soundness error is at most $\max\left\{\frac{\tau}{p}, \frac{\beta}{p}\right\}$. □

A.4.4 NTT PIOR

Proof. PIOR 5 outputs two instance–witness pairs: one for the Lagrange relation and one for the SumCheck relation. We analyze each induced PIOR and then apply product composition.

Lagrange check. This component is deterministic: the reduction outputs a fixed Lagrange instance built from the prover's $\tilde{v}$. Thus, if the Lagrange relation is false, the output instance is false with certainty, and the escape probability is 0 (state function is trivial).

SumCheck reduction. Let $t := t$. The reduction checks t univariate identities (one per batch). For each $i \in [t]$, the two sides are

$$\hat{p}_{\text{mono}}(r) = \sum_{\mathbf{x} \in \{0,1\}^\beta} \tilde{m}(\mathbf{x})\tilde{u}(\mathbf{x}) \quad \text{and} \quad \hat{p}_{\text{lag}}(r) = \sum_{\mathbf{x} \in \{0,1\}^\beta} \tilde{m}'(\mathbf{x})\tilde{v}(\mathbf{x}),$$

i.e., both sides evaluate the same underlying univariate polynomial at a random point r . Let $q_i(r) := \hat{p}_{\text{mono},i}(r) - \hat{p}_{\text{lag},i}(r)$. If the NTT relation is false, then at least one q_i is a nonzero univariate polynomial of degree $< B$. By Lemma 3.1 and a union bound,

$$\Pr_r [\forall i \in [t] : q_i(r) = 0] \leq t \cdot B/p,$$

which is the escape probability (and round-by-round soundness error) for the SumCheck component.

Product composition. By Theorem 3.9, the overall round-by-round soundness error is the maximum of the two errors, i.e. tB/p . $\square$

A.4.5 Integer Multiplication PIOR

Proof. The reduction in PIOR 6 is deterministic: given the input instance and witness, it produces a fixed tuple of output instances (ZeroCheck, NTT-check, Hadamard-check, and BinaryCheck) without any verifier randomness. Hence, a false multiplication instance always maps to a false tuple, the state function is trivial, and the probability of escaping the doomed state is 0. $\square$

A.4.6 Integer Division PIOR

Proof. PIOR 7 outputs a tuple of instances for ZeroCheck (overflow), NTT-check, Hadamard-check, and BinaryCheck. Each component is produced deterministically from the input instance and the prover's messages (there is no verifier randomness in the reduction), so for any false component instance the escape probability is 0 and the corresponding state function is trivial. By Theorem 3.9, the overall round-by-round soundness error is the maximum of these (zero), hence 0. $\square$

A.4.7 MaSDAK

Proof. Round-by-round soundness. PIOR 8 outputs two instance–witness pairs: one for the decision relation (the modular equation check) and one for the division relation. We analyze each induced PIOR and then apply product composition.

Decision check. Let $v := f(\mathbf{v}) \in \mathbb{Z}$ be the value of the Diophantine function on the true integer witness. If the statement is false then $v \neq 0$. The verifier samples a random prime $\ell \in \text{Primes}[\lambda]$ and accepts the decision instance iff $v \equiv 0 \pmod{\ell}$. Thus the escape probability is

$$\Pr_{\ell \leftarrow \text{Primes}[\lambda]} [v \equiv 0 \pmod{\ell}] \leq \frac{1}{|\text{Primes}[\lambda]|} = 2^{-\lambda},$$

by definition $|\text{Primes}[\lambda]| = 2^\lambda$.

Division check. This component is deterministic and reduces to PIOR 7 with round-by-round soundness error 0, so the escape probability is 0.

Product composition. By Theorem 3.9, the overall round-by-round soundness error is the maximum of the two errors, i.e. $2^{-\lambda}$. $\square$

A.5 PIOP Toolbox

A.5.1 BinaryCheck

Definition A.2. The oracle relation $\mathfrak{R}_{\text{Bin}}$ is the set of all triples of the form $(x, y, w) = (\emptyset, \llbracket f \rrbracket, f)$ where $f \in \mathbb{F}_{p,\mu}^{\text{ind} \leq d}[X]$ such that for every $x \in \{0, 1\}^\mu$, $f(x) \in \{0, 1\}$.

PIOR 8: $\mathfrak{R}_{\text{Bin}} \rightarrow \mathfrak{R}_{\text{Zero}}$

Parse: $x = \emptyset, y = \llbracket f \rrbracket, w = f$

1. $\mathcal{P}$ computes $f' := f^2 - f$ where $f' \in \mathbb{F}_{p,\mu}^{\text{ind} \leq 2}$ and $\mathcal{V}$ virtually computes the oracle $\llbracket f' \rrbracket$.
2. $\mathcal{P}$ outputs $w'_{\text{Zero}} := f'$ and $\mathcal{V}$ outputs $(x'_{\text{Zero}}, y'_{\text{Zero}}) := (\emptyset, \llbracket f' \rrbracket)$.

Theorem A.3. PIOR 9 from $\mathfrak{R}_{\text{Bin}}$ (defined in Definition A.2) to $\mathfrak{R}_{\text{Zero}}$ (defined in Definition A.6) has perfect completeness and round-by-round soundness error 0 with the following efficiency measures:

rounds	message size	queries	oracle size	prover time	verifier time
0	0	0	1	$O(N)$	$O(1)$

Proof. The perfect completeness and efficiency measures follow directly from the construction. We now prove the round-by-round soundness for the PIOR.

Round-by-round soundness. The reduction in PIOR 9 is non-interactive: it deterministically maps f to $f' := f^2 - f$ and outputs the ZeroCheck instance $(\emptyset, \llbracket f' \rrbracket)$. Therefore, for any false binary claim, the produced ZeroCheck instance is false and there is no verifier randomness that could turn it into a true instance. Hence the round-by-round soundness error is 0. $\square$

A.5.2 HadamardCheck

Definition A.4. The oracle relation $\mathfrak{R}_{\text{Had}}$ is the set of all triples of the form $(x, y, w) = (\emptyset, (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket, \llbracket \tilde{d} \rrbracket), (\tilde{a}, \tilde{b}, \tilde{c}, \tilde{d}))$ where $\tilde{a}, \tilde{b}, \tilde{c}, \tilde{d} \in \mathbb{F}_{p,\mu}^{\text{ind} \leq 1}$, such that

$$d = a \circ b + c$$

where a, b, d , and c are the evaluation of $\tilde{a}, \tilde{b}, \tilde{d}$, and $\tilde{c}$ on the boolean hypercube $\{0, 1\}^\mu$ respectively.

PIOR 9: $\mathfrak{R}_{\text{Had}} \rightarrow \mathfrak{R}_{\text{Decision}}[\text{Set20}]$

Parse: $x = \emptyset, y = (\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket, \llbracket \tilde{d} \rrbracket), w = \tilde{a}, \tilde{b}, \tilde{c}, \tilde{d}$

1. $\mathcal{V}$ samples a random $r \in \mathbb{F}_p^\mu$ and queries the oracles $\llbracket \tilde{a} \rrbracket, \llbracket \tilde{b} \rrbracket, \llbracket \tilde{c} \rrbracket$, and $\llbracket \tilde{d} \rrbracket$ at r to obtain $a := \tilde{a}(r), b := \tilde{b}(r), c := \tilde{c}(r)$, and $d := \tilde{d}(r)$.
2. $\mathcal{P}$ outputs $w'_{\text{Decision}} := \emptyset$. If $a \cdot b + c = d$, then $\mathcal{V}$ outputs $(x'_{\text{Decision}}, y'_{\text{Decision}}) := (\text{accept}, \emptyset)$; otherwise it

outputs (reject, $\emptyset$).

Theorem A.5. PIOR 10 has perfect completeness and round-by-round soundness error at most μ/p with the following efficiency measures:

rounds	#queries	oracle size	prover time	verifier time
1	4	0	$O(1)$	$O(\mu)$

Proof. The perfect completeness and efficiency measures follow directly from the construction. We now prove the round-by-round soundness.

Round-by-round soundness. Let $h(\mathbf{x}) := \tilde{a}(\mathbf{x})\tilde{b}(\mathbf{x}) + \tilde{c}(\mathbf{x}) - \tilde{d}(\mathbf{x})$. If the relation is false, then h is a nonzero multilinear polynomial, so $\Pr_{\mathbf{r} \leftarrow \mathbb{F}_p^\mu} [h(\mathbf{r}) = 0] \leq \mu/p$ by Lemma 3.1. $\square$

A.5.3 ZeroCheck

For completeness, we restate the zerocheck relation, protocol, and its properties here. The zerocheck protocol is a fundamental building block for all of our protocols.

Definition A.6. The oracle relation $\mathfrak{R}_{\text{zero}}$ is the set of all tuples of the form $(\mathbf{x}, w) = (\llbracket p \rrbracket, p)$ where $p \in \mathbb{F}_{p,\mu}^{\text{ind} \leq d}[\mathbf{X}]$ such that $\forall \mathbf{x} \in \{0, 1\}^\mu, p(\mathbf{x}) = 0$.

PIOR 10: $\mathfrak{R}_{\text{zero}} \rightarrow \mathfrak{R}_{\text{sum}}[\text{Set20}]$

Parse: $\mathbf{x} = \emptyset, \mathbf{y} = \llbracket f \rrbracket, w = f$

1. $\mathcal{V}$ samples a random point $\mathbf{r} \in \mathbb{F}_p^\mu$ and sends it to $\mathcal{P}$.
2. To verify that p is zero on the boolean hypercube, $\mathcal{P}$ outputs the witness for the sumcheck relation (Definition A.9) as $w'_{\text{sum}} := f$ while $\mathcal{V}$ outputs the corresponding explicit and implicit instance as $(\mathbf{x}'_{\text{sum}}, \mathbf{y}'_{\text{sum}}) := (0, \llbracket f \rrbracket)$, where f is the virtual polynomial defined as $f(\mathbf{x}) := p(\mathbf{x}) \cdot \text{eq}(\mathbf{x}, \mathbf{r})$.

Theorem A.7. PIOR 11 has perfect completeness and round-by-round soundness error at most $\mu/|\mathbb{F}|$ with the following efficiency measures:

rounds	queries	oracle size	prover time	verifier time
1	0	0	$O(N)$	$O(\mu)$

Proof. See [CBBZ23]. Since PIOR 11 is a PIOR to SumCheck, we only analyze the verifier round that samples $\mathbf{r}$ and produces a SumCheck instance. To fit the round-by-round definition, we consider a functionally equivalent variant with an initial empty prover message $\Pi_1 = \emptyset$. We define the state function as

$$\text{State}(\mathbf{x} = \llbracket p \rrbracket, \mathbf{y} = \emptyset, \text{tr}) = \begin{cases} 0 & \text{tr} = \emptyset \\ 0 & \text{tr} = (\Pi_1) \\ 0 & \text{tr} = (\Pi_1, \varrho_1 = \{\mathbf{r}\}) \wedge \sum_{\mathbf{x} \in \{0,1\}^\mu} p(\mathbf{x})\text{eq}(\mathbf{x}, \mathbf{r}) \neq 0 \\ 1 & \text{tr} = (\Pi_1, \varrho_1 = \{\mathbf{r}\}) \wedge \sum_{\mathbf{x} \in \{0,1\}^\mu} p(\mathbf{x})\text{eq}(\mathbf{x}, \mathbf{r}) = 0. \end{cases}$$

This state function satisfies Definition 3.6. The round-by-round soundness error is the probability that a false zero-check claim is reduced to a true SumCheck instance:

$$\begin{aligned} \epsilon_{\text{PIOP}}^{\text{rbr}} &= \Pr_{\mathbf{r} \xleftarrow{\$} \mathbb{F}_p^\mu} \left[\sum_{\mathbf{x} \in \{0,1\}^\mu} p(\mathbf{x}) \text{eq}(\mathbf{x}, \mathbf{r}) = 0 \mid \exists \mathbf{x} \in \{0,1\}^\mu : p(\mathbf{x}) \neq 0 \right] \\ &\leq \frac{\mu}{|\mathbb{F}|} \quad \text{Schwartz-Zippel (Lemma 3.1)} \end{aligned}$$

□

Remark A.8 (Batching ZeroCheck claims). To prove multiple ZeroCheck claims $\{(\llbracket p_i \rrbracket, p_i)\}_{i=1}^t$ where each p_i is a μ -variate polynomial with individual degree d , $\mathcal{V}$ can sample t random field elements $\{c_i\}_{i=1}^t$ and run a single ZeroCheck on the polynomial $p^* = \sum_{i=1}^t c_i p_i$.

A.5.4 SumCheck

For completeness, we restate the sumcheck relation, protocol, and its properties here. The sumcheck protocol is a fundamental building block for all of our protocols.

Definition A.9. The oracle relation $\mathfrak{R}_{\text{sum}}$ is the set of all tuples of the form $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = (v, \llbracket f \rrbracket, f)$ where $v \in \mathbb{F}_p$ is the sum and $f \in \mathbb{F}_{p,\mu}^{\text{ind} \leq d}[\mathbf{X}]$ is such that $\sum_{\mathbf{x} \in \{0,1\}^\mu} f(\mathbf{x}) = v$.

PIOR 11: $\mathfrak{R}_{\text{sum}} \rightarrow \emptyset$ ([LFKN92])

Parse: $\mathbf{x} = v, \mathbf{y} = \llbracket f \rrbracket, \mathbf{w} = f$

1. $\mathcal{P}$ sends to $\mathcal{V}$ a univariate polynomial f_1 such that

$$f_1(X_1) = \sum_{\{x_2, \dots, x_\mu\} \in \{0,1\}^{\mu-1}} f(X_1, x_2, \dots, x_\mu)$$

2. $\mathcal{V}$ checks that the degree of $f_1(x_1)$ is at most the degree of f in x_1 . It computes $v_1 = f_1(0) + f_1(1)$ and checks that $v_1 = v$. If not, $\mathcal{V}$ rejects. It then picks a random $r_1 \in \mathbb{F}$ and sends it to the Prover ($\mathcal{P}$).
3. For each i from 2 to μ (subsequent rounds):
 - (a) $\mathcal{P}$ sends to $\mathcal{V}$ the univariate polynomial

$$f_i(x_i) = \sum_{\{x_{i+1}, \dots, x_\mu\} \in \{0,1\}^{\mu-i}} f(r_1, \dots, r_{i-1}, x_i, \dots, x_\mu)$$

- (b) $\mathcal{V}$ checks that the degree of $f_i(x_i)$ is at most the degree of f in x_i . It computes $v_i = f_i(0) + f_i(1)$ and checks that $v_i = f_{i-1}(r_{i-1})$. If not, $\mathcal{V}$ rejects. It picks a random $r_i \in \mathbb{F}_p$ and sends it to $\mathcal{P}$.
4. $\mathcal{P}$ sends the value $f(r_1, r_2, \dots, r_{\mu-1}, r_\mu)$ for the random values $r_1, r_2, \dots, r_\mu$ chosen by $\mathcal{V}$.
5. $\mathcal{V}$ checks that this value is correct by querying $\llbracket f \rrbracket$ at $(r_1, r_2, \dots, r_{\mu-1}, r_\mu)$.

Theorem A.10. Let $d := \max_{i \in [\mu]} \deg_{x_i}(f)$ be the maximum individual degree of f . PIOR 12 has perfect completeness and $d/|\mathbb{F}|$ round-by-round soundness error with the following efficiency measures:

rounds	#queries	oracle size	prover time	verifier time
μ	$\mu + 1$	$o(d\mu)$	$O(Nd \log^2 d)$	$O(d\mu)$

Proof. See [CBBZ23]. We follow the proof strategy of Canetti et al. [Can+19] in their proof of round-by-round soundness for the GKR protocol [GKR08].

We define a partial transcript as doomed if the initial claim for the corresponding step in the recursion is false. More formally, the state function $\text{State}_{\text{PIOP}}^{\text{rbr}}$ is as follows:

$$\text{State}_{\text{PIOP}}^{\text{rbr}}(\mathbb{x} = (\llbracket p \rrbracket, v), \text{tr}) := \begin{cases} 0 & (\text{tr} = \emptyset \vee \text{tr} = p_1) \wedge \sum_{x_1, \dots, x_\mu \in \{0,1\}^\mu} p(x_1, \dots, x_\mu) \neq v \\ 1 & (\text{tr} = \emptyset \vee \text{tr} = p_1) \wedge \sum_{x_1, \dots, x_\mu \in \{0,1\}^\mu} p(x_1, \dots, x_\mu) = v \\ 0 & \text{tr} = (p_1, \rho_1, \dots, p_i, \rho_i) \wedge \sum_{\substack{x_{i+1}, \dots, x_\mu \\ \in \{0,1\}^{\mu-i}}} p(\rho_1, \dots, \rho_i, x_{i+1}, \dots, x_\mu) \neq p_i(\rho_i) \\ 1 & \text{tr} = (p_1, \rho_1, \dots, p_i, \rho_i) \wedge \sum_{\substack{x_{i+1}, \dots, x_\mu \\ \in \{0,1\}^{\mu-i}}} p(\rho_1, \dots, \rho_i, x_{i+1}, \dots, x_\mu) = p_i(\rho_i) \end{cases}.$$

It is straightforward to see that the state function $\text{State}_{\text{PIOP}}^{\text{rbr}}$ satisfies the first two conditions Definition 3.6 by construction. For the third condition, i.e. the full transcript, if the full transcript is doomed, we have that $p(\rho_1, \dots, \rho_\mu) \neq p_\mu(\rho_\mu)$, which, by definition, is the same as the condition for the sumcheck verifier to reject. Hence, $\text{State}_{\text{PIOP}}^{\text{rbr}}$ is a valid round by round state function. We are left to compute the round-by-round soundness error. We do this for round $i = 1$ and $i > 1$ separately:

Round 1. Let $\tilde{p}_i(X) := \sum_{x_{i+1}, \dots, x_\mu \in \{0,1\}^{\mu-i}} p(\rho_1, \dots, \rho_{i-1}, X, x_{i+1}, \dots, x_\mu)$. Using this notation, and upon inlining the first two cases of $\text{State}_{\text{PIOP}}^{\text{rbr}}$ definition into Eq. (4), allows us to rewrite Eq. (4) as follows:

$$\begin{aligned} \epsilon_{\text{PIOP},1}^{\text{rbr}}(\mathbb{x}) &= \Pr \left[\begin{array}{c} \text{State}_{\text{PIOP}}^{\text{rbr}}(\llbracket p \rrbracket, \emptyset) = 0 \\ \text{State}_{\text{PIOP}}^{\text{rbr}}(\llbracket p \rrbracket, (p_1, \rho_1)) = 1 \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \mathbb{F} \end{array} \right] \\ &= \Pr \left[\begin{array}{c} \sum_{x_1, \dots, x_\mu \in \{0,1\}^\mu} p(x_1, \dots, x_\mu) \neq v \\ \sum_{x_2, \dots, x_\mu \in \{0,1\}^{\mu-1}} p(\rho_1, x_2, \dots, x_\mu) = p_1(\rho_1) \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \mathbb{F} \end{array} \right] \\ &= \Pr \left[\begin{array}{c} \tilde{p}_1(0) + \tilde{p}_1(1) \neq v \\ \tilde{p}_1(\rho_1) = p_1(\rho_1) \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \{0,1\}^{\mathbb{F}^*} \end{array} \right] \end{aligned}$$

Now since the prover was able to convince the verifier in the first round, this implies that $p_1(0) + p_1(1) = v$, and so we have:

$$\epsilon_{\text{PIOP},1}^{\text{rbr}}(\mathbb{x}) = \Pr \left[\begin{array}{c} \tilde{p}_1(0) + \tilde{p}_1(1) \neq p_1(0) + p_1(1) \\ \tilde{p}_1(\rho_1) = p_1(\rho_1) \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \{0,1\}^{\mathbb{F}^*} \end{array} \right]$$

By conditioning on event $p_1 \neq \tilde{p}_1$ we have:

$$\begin{aligned} \epsilon_{\text{PIOP},1}^{\text{rbr}}(\mathbb{x}) &= \Pr \left[\begin{array}{c} \tilde{p}_1(0) + \tilde{p}_1(1) \neq p_1(0) + p_1(1) \\ \tilde{p}_1(\rho_1) = p_1(\rho_1) \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \{0,1\}^{\mathbb{F}^*} \end{array} \right] \Pr[p \neq \tilde{p}_1] \\ &\quad + \Pr \left[\begin{array}{c} \tilde{p}_1(0) + \tilde{p}_1(1) \neq p_1(0) + p_1(1) \\ \tilde{p}_1(\rho_1) = p_1(\rho_1) \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \{0,1\}^{\mathbb{F}^*} \end{array} \right] \Pr[p_1 \neq \tilde{p}_1] \\ &= \Pr \left[\begin{array}{c} \tilde{p}_1(0) + \tilde{p}_1(1) \neq p_1(0) + p_1(1) \\ \tilde{p}_1(\rho_1) = p_1(\rho_1) \end{array} \middle| \begin{array}{c} p_1 \leftarrow \tilde{\mathbf{P}}^{\text{rbr}} \\ \rho_1 \leftarrow \{0,1\}^{\mathbb{F}^*} \end{array} \right] \Pr[p_1 \neq \tilde{p}_1] \end{aligned}$$

Finally, applying the Lemma 3.1 gives that $\epsilon_{\text{PIOP},1}^{\text{rbr}}(\mathbb{x}) = \deg(p_1)/|\mathbb{F}|$.

Round $i > 1$. Repeating the steps for the case $i = 1$ and inlining the last two cases of $\text{State}_{\text{PIOP}}^{\text{rbr}}$ definition into Eq. (4), we can rewrite Eq. (4) as:

$$\begin{aligned}
\epsilon_{\text{PIOP},i}^{\text{rbr}}(\mathbb{X}) &= \Pr \left[\begin{array}{l} \text{State}_{\text{PIOP}}^{\text{rbr}}(\llbracket p \rrbracket, (p_1, \rho_1, \dots, p_i)) = 0 \\ \text{State}_{\text{PIOP}}^{\text{rbr}}(\llbracket p \rrbracket, (p_1, \rho_1, \dots, p_i, \rho_i)) = 1 \end{array} \middle| \begin{array}{l} ([p_j]_{j=1}^i, [\rho_j]_{j=1}^{i-1}) \\ \leftarrow \mathbf{P}^{\text{rbr}} \\ \rho_i \leftarrow \mathbb{F} \end{array} \right] \\
&= \Pr \left[\begin{array}{l} \sum_{\substack{x_i, \dots, x_\mu \\ \in \{0,1\}^\mu}} p(\rho_1, \dots, \rho_{i-1}, x_i, \dots, x_\mu) \neq p_{i-1}(\rho_{i-1}) \\ \sum_{\substack{x_{i+1}, \dots, x_\mu \\ \in \{0,1\}^\mu}} p(\rho_1, \dots, \rho_i, x_{i+1}, \dots, x_\mu) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} ([p_j]_{j=1}^i, [\rho_j]_{j=1}^{i-1}) \\ \leftarrow \mathbf{P}^{\text{rbr}} \\ \rho_i \leftarrow \mathbb{F} \end{array} \right] \\
&= \Pr \left[\begin{array}{l} \tilde{p}_i(0) + \tilde{p}_i(1) \neq p_{i-1}(\rho_{i-1}) \\ \tilde{p}_i(\rho_i) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} ([p_j]_{j=1}^i, [\rho_j]_{j=1}^{i-1}) \\ \leftarrow \mathbf{P}^{\text{rbr}} \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right]
\end{aligned}$$

Now since the prover was able to convince the verifier in the $i - 1$ -th round, this implies that $p_{i-1}(\rho_{i-1}) = p_i(0) + p_i(1)$, and so we have:

$$\begin{aligned}
\epsilon_{\text{PIOP},i}^{\text{rbr}}(\mathbb{X}) &= \Pr \left[\begin{array}{l} \tilde{p}_i(0) + \tilde{p}_i(1) \neq p_{i-1}(\rho_{i-1}) \\ \tilde{p}_i(\rho_i) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} p_{i-1}(\rho_{i-1}) = p_i(0) + p_i(1) \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right] \\
&= \Pr \left[\begin{array}{l} \tilde{p}_i(0) + \tilde{p}_i(1) \neq p_i(0) + p_i(1) \\ \tilde{p}_i(\rho_i) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right]
\end{aligned}$$

By again conditioning on the event $p_i \neq \tilde{p}_i$, we have:

$$\begin{aligned}
\epsilon_{\text{PIOP},i}^{\text{rbr}}(\mathbb{X}) &= \Pr \left[\begin{array}{l} \tilde{p}_i(0) + \tilde{p}_i(1) \neq p_i(0) + p_i(1) \\ \tilde{p}_i(\rho_i) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} p_i = \tilde{p}_i \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right] \cdot \Pr[p_i = \tilde{p}_i] \\
&\quad + \Pr \left[\begin{array}{l} \tilde{p}_i(0) + \tilde{p}_i(1) \neq p_i(0) + p_i(1) \\ \tilde{p}_i(\rho_i) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} p_i \neq \tilde{p}_i \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right] \cdot \Pr[p_i \neq \tilde{p}_i]
\end{aligned}$$

The first term is zero, and for the second term we can use the identity $\Pr[A \cap B] \leq \Pr[A|B]$ to obtain:

$$\begin{aligned}
\epsilon_{\text{PIOP},i}^{\text{rbr}}(\mathbb{X}) &= \Pr \left[\begin{array}{l} \tilde{p}_i(0) + \tilde{p}_i(1) \neq p_i(0) + p_i(1) \\ \tilde{p}_i(\rho_i) = p_i(\rho_i) \end{array} \middle| \begin{array}{l} p_i \neq \tilde{p}_i \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right] \cdot \Pr[p_i \neq \tilde{p}_i] \\
&\leq \Pr \left[\begin{array}{l} \tilde{p}_i(\rho_i) = p_i(\rho_i) \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \middle| \begin{array}{l} p_i \neq \tilde{p}_i \\ \rho_i \leftarrow \{0, 1\}^{\mathbb{F}^*} \end{array} \right] \cdot \Pr[p_i \neq \tilde{p}_i]
\end{aligned}$$

According to the Schwartz-Zippel lemma [Zip79; Sch80], we have $\epsilon_{\text{PIOP},i}^{\text{rbr}}(\mathbb{X}) = \deg(p_i)/|\mathbb{F}|$. Hence, we have $\epsilon_{\text{PIOP}}^{\text{rbr}} = \max_{i \in [\mu]} \deg_i(p)/|\mathbb{F}|$. $\square$

Remark A.11 (Batching SumCheck claims). To prove multiple SumCheck claims $\{((s_i, \llbracket p_i \rrbracket), p_i)\}_{i=1}^t$ where each p_i is a μ -variate polynomial with individual degree d , $\mathcal{V}$ can sample t random field elements $\{c_i\}_{i=1}^t$ and run a single SumCheck on the polynomial $p^* = \sum_{i=1}^t c_i p_i$ and the sum $s^* = \sum_{i=1}^t c_i s_i$.

B Deferred proof of Theorem 5.4

Proof. We prove that Construction 5 satisfies the properties of a relaxed mod-PC scheme (see Definition 3.27), namely: 1. (Setup, Commit, Open) is a binding commitment scheme for the message space $\mathbb{Z}_B[X_1, \dots, X_\mu]$. 2. ROK_{Eval} is a succinct interactive public-coin argument system for the relation $\mathfrak{R}_{\text{Eval}}$.

Binding. If an adversary outputs $(\text{cm}, p, p', \text{opn}, \text{opn}')$ such that

$$\text{Open}(\text{pp}, \text{cm}, p, \text{opn}) = \text{Open}(\text{pp}, \text{cm}, p', \text{opn}') = 1$$

then by definition of the Open algorithm, we have

$$\begin{aligned} \text{CS.Open}(\text{pp}, \text{cm}, \text{IntegerEncode}(p), \text{opn}) &= \\ \text{CS.Open}(\text{pp}, \text{cm}, \text{IntegerEncode}(p'), \text{opn}') &= 1 \end{aligned}$$

By the binding property of the commitment scheme CS , we have that with overwhelming probability, $\text{IntegerEncode}(p) = \text{IntegerEncode}(p')$. And by the injectivity of the integer encoding function IntegerEncode , we have that $p = p'$. Thus, the commitment scheme is binding.

Completeness. This follows from inspection; notice that the Masdak check $l + m\alpha^N + r\alpha^{N+1} = e_{p,\alpha}^* \cdot e_y$ will succeed by construction, and that when all coefficients of the initial polynomial are bounded by B , all the terms in the product are also bounded by the respective bounds above and by choice of α , there is no overflow of any of the coefficients of the product polynomial. The rest of the completeness proof follows from completeness of Masdak and Range.

Extractability. We show that for an expected poly-time adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{A}_4)$ if the verifier accepts in ROK_{Eval} , then there exists an extractor algorithm Extract that outputs a witness for $\mathfrak{R}_{\text{Eval}}$, i.e. $w^* = (p^*, \text{opn}^*)$ such that

$$z = p^*(y) \text{ over } \mathbb{Q} \quad \text{and} \quad \text{PCS.Open}(\text{ck}, \text{cm}, p^*, \text{opn}^*) = 1$$

Note that we run the $\text{ROK}_{\text{Core-Eval}}$ protocol twice in ROK_{Eval} . At a high level, the first part of the proof follows the proof of Theorem 5.5 in [BMSS25], and we extract valid openings for both commitments (under α and β) that satisfy the equations

$$\begin{aligned} p_\alpha^*(y) &= z \bmod \alpha \\ p_\beta^*(y) &= z \bmod \beta \end{aligned}$$

However, due to the choice of $\beta = \alpha \cdot q$, we have that

$$p_\beta^*(y) = p_\alpha^*(y) \bmod \alpha$$

As we are in the non-adaptive setting, the evaluation point y is randomly sampled after commitments to p_α^* and p_β^* are produced. Hence, applying the Schwartz-Zippel lemma and noticing that α has only large prime factors, we have that $p = p'$ as rational polynomials with overwhelming probability. (Although typically we only have guarantees $\bmod \alpha$, since all coefficients of p_α and p_β are guaranteed to be bounded rationals, we can extend the guarantees to the rationals.)

As both the polynomials are the same, we can use the second equation to obtain the necessary guarantee module q . Now, noticing that this is also a random modulus that we check our equation on, applying a standard fingerprinting argument shows that the evaluation z is correct over the integers with overwhelming probability.

Formally, consider the first call to the $\text{ROK}_{\text{Core-Eval}}$ protocol. Since Construction 4 is a reduction of knowledge from $\mathfrak{R}_{\text{CoreEval}}$ to $\mathfrak{R}_{\text{Masdak}} \times \mathfrak{R}_{\text{Range}}$, we may assume that we have accepting witnesses for both the Masdak and Range relations.

These witnesses give us openings $e_{p,\alpha}, l, m, r$ to the intermediate commitments $\text{cm}_{\text{CS}}, \text{cm}_l, \text{cm}_m, \text{cm}_r$ such that

$$\begin{aligned} l + m\alpha^N + r\alpha^{N+1} &= e_{p,\alpha}^* \cdot e_y \\ 0 < l &< 2 \cdot N 2^{B(\mu+1)} \alpha^{N-2}, \\ 0 < m &< 2 \cdot N 2^{B(\mu+1)}, \\ 0 < r &< 2 \cdot 2^{B(\mu+1)} \alpha^{N-2} \end{aligned}$$

These factors of 2 show up due to the range proof only admitting bounds that are powers of 2.

Algorithm 4: Extract

Extract

1. Obtain $e_{p,\alpha}^*$ from the accepting witness for the Masdak relation.
 2. Compute $e_{p,\alpha}^*$ base α ; i.e. compute $p^* := \text{IntegerDecode}(e_{p,\alpha}^*, \alpha)$.
 3. To obtain the rational representation from this integer polynomial, do the following for each coefficient $i \in \{0, \dots, N-1\}$:
 - (a) Invoke the rational approximation algorithm to obtain $m_i/k_i \in \mathbb{Q}$ which is the closest fraction to p_i^*/α . If $k_i > N^{8\mu} \cdot (2\mu)^\lambda$, abort.
 - (b) Compute $n_i = p_i^* k_i - m_i \alpha$.
 - (c) Set $\tilde{p}_i = \frac{n_i}{k_i} + \frac{m_{i-1}}{k_{i-1}}$ where $m_{-1} = 0$ and $k_{-1} = 1$.
 4. Compute $p = \tilde{p} \bmod q$ and output $(p, \tilde{p})$.
-

We require the following results from [BF23], adapted to our notation.

Lemma B.1 (Corollary 1 from [BF23]). For all n such that $\log n \geq 8\mu^2 + \log_2(2\mu) \cdot \lambda$, and any μ -linear polynomial f coprime with n ,

$$\Pr_{x \leftarrow [0, m]^\mu} [f(x) \equiv 0 \bmod n] \leq 2^{-\lambda} + \frac{\mu}{m}.$$

Corollary B.2 (Local zero to global zero). Under the conditions of Lemma B.1, if $f(y) \equiv 0 \bmod n$ for a random $y \leftarrow [0, m]^\mu$, then f is the zero polynomial mod n with probability $\geq 1 - 2^{-\lambda} - \mu/m$.

Corollary B.3 (Modulus upper bound). If f is a μ -linear polynomial coprime with n , not identically zero, and $f(y) \equiv 0 \bmod n$ for random $y \leftarrow [0, m]^\mu$, then $n < N^{8\mu} \cdot (2\mu)^\lambda$.

Lemma B.4 (Lemma 1 from [BF23]: Evaluation bound). For any μ -linear polynomial f and $m \geq 2$:

$$\Pr_{x \leftarrow [0, m]^\mu} \left[|f(x)| \leq \frac{1}{m^\mu} \cdot \|f\|_\infty \right] \leq \frac{3\mu}{m}.$$

Correctness of extracted opening. We show that the extracted coefficients p_i^* admit a bounded rational representation and that the evaluation identity holds. From the check in Masdak, we have that

$$e_{p,\alpha}^* \cdot e_y = l_0^* + l_1^* \alpha^{N-2} + m^* \alpha^{N-1} + r^* \alpha^N$$

where $l_1^* < 2N 2^{B(\mu+1)}$, $m^* < 2N 2^{B(\mu+1)}$, and $r^* < 22^{B(\mu+1)} \alpha^{N-2}$.

We claim each p_i^* admits a representation

$$p_i^* = \frac{m_i \alpha + n_i}{k_i}, \quad m_i \geq 0, |n_i| < \alpha, k_i > 0, \gcd(k_i, \alpha) = 1. \quad (5)$$

(Such a representation exists trivially with $m_i = 0, n_i = p_i^*, k_i = 1$; we derive bounds on k_i, n_i .)

Step 1: Coefficient of α^{N-2} . Write $l^* = l_0^* + l_1^* \alpha^{N-2}$ where $l_1^* < 2N2^{B(\mu+1)}$. On the LHS, the coefficient of α^{N-2} is $\left\lfloor \frac{e_{\mathbf{p}, \alpha}^* \cdot e_{\mathbf{y}}}{\alpha^{N-2}} \right\rfloor \bmod \alpha$. Expanding:

$$\begin{aligned} \left\lfloor \frac{e_{\mathbf{p}, \alpha}^* \cdot e_{\mathbf{y}}}{\alpha^{N-2}} \right\rfloor \bmod \alpha &= \left\lfloor \frac{\sum_{i=0}^{N-1} \sum_{j=0}^{N-1} p_i^* \cdot y_{N-j-1} \cdot \alpha^{i+j}}{\alpha^{N-2}} \right\rfloor \bmod \alpha \\ &= \underbrace{\left\lfloor \frac{\sum_{0 \leq i+j \leq N-3} p_i^* \cdot y_{N-j-1} \cdot \alpha^{i+j}}{\alpha^{N-2}} \right\rfloor}_{:= [e]} + \sum_{i=0}^{N-2} p_i^* \cdot y_{i+1} \bmod \alpha \end{aligned} \quad (6)$$

where terms with $i + j > N - 2$ are absorbed into higher powers and vanish modulo α . The carry term e is bounded as

$$0 \leq [e] \leq \left\lfloor \frac{N^2 \cdot B^\mu \cdot \alpha^{N-2}}{\alpha^{N-2}} \right\rfloor = N^2 B^\mu.$$

Equating the coefficient of α^{N-2} on both sides gives $\sum_{i=0}^{N-2} p_i^* y_{i+1} = l_1^* - [e] \bmod \alpha$, so there exists $m \in \mathbb{Z}$ such that

$$\sum_{i=0}^{N-2} \frac{m_i \alpha + n_i}{k_i} y_{i+1} = (l_1^* - [e]) + m \alpha \quad (7)$$

for some $m \in \mathbb{Z}$. Setting $L := \text{lcm}(k_0, \dots, k_{N-2})$ and rearranging:

$$p_1(\mathbf{y}) := (l_1^* - [e])L - \sum_{i=0}^{N-2} \frac{L n_i}{k_i} y_{i+1} \equiv 0 \bmod \alpha. \quad (8)$$

Step 2: Bounding k_i . Since $\gcd(k_i, \alpha) = 1$ for all i , p_1 is coprime with α . Verifying $\alpha = B^{7\mu} > N^{8\mu} \cdot (2\mu)^\lambda$, we apply Corollary B.2: $p_1 \equiv 0 \bmod \alpha$ as a polynomial with overwhelming probability. Since all coefficients of p_1 are bounded by α , in fact $p_1 \equiv 0$ over $\mathbb{Z}$.

This implies $p_2(\mathbf{y}) := \sum_{i=0}^{N-2} \frac{L n_i}{k_i} y_{i+1} \equiv 0 \bmod L$. By Corollary B.3, $L < N^{8\mu} \cdot (2\mu)^\lambda$, and hence each $k_i < N^{8\mu} \cdot (2\mu)^\lambda$.

Step 3: Bounding n_i . By Lemma B.4 applied to $p_2(\mathbf{y}) = (l_1^* - [e])L$, we get $|n_i| \leq B^{2\mu+1} \cdot N \cdot k_i$ for $i \leq N - 2$.

For $i = N - 1$, we analyze the coefficient of α^{2N-2} (the highest non-trivial power). Let u' denote the coefficient of α^{2N-2} on the RHS; by the bound on r^* , we have $u' < B^{\mu+1}$. On the LHS:

$$\left\lfloor \frac{e_{\mathbf{p}, \alpha}^* \cdot e_{\mathbf{y}}}{\alpha^{2N-2}} \right\rfloor \bmod \alpha = \underbrace{\left\lfloor \frac{\sum_{i+j < 2N-2} p_i^* \cdot y_{N-j-1} \cdot \alpha^{i+j}}{\alpha^{2N-2}} \right\rfloor}_{:= [e']} + p_{N-1}^* \cdot y_0 \bmod \alpha$$

where $\lfloor e' \rfloor \leq N^2 B^\mu$ (bounded analogously to $\lfloor e \rfloor$). Equating both sides and applying (5) with the constraint $m_{N-1} = 0$:

$$\frac{n_{N-1}}{k_{N-1}} \cdot y_0 = (u' - \lfloor e' \rfloor) \bmod \alpha$$

By Corollary B.2 and the fact that $u' - \lfloor e' \rfloor < \alpha$, the polynomial $n_{N-1}y_0 - (u' - \lfloor e' \rfloor)k_{N-1}$ is identically zero over $\mathbb{Z}$. By Corollary B.3, $k_{N-1} < N^{8\mu} \cdot (2\mu)^\lambda$. By Lemma B.4, $|n_{N-1}| \leq NB^{2\mu+1}$.

Step 4: Valid opening. Since $\tilde{p}_i = n_i/k_i + m_{i-1}/k_{i-1}$ with $m_{-1} = 0, k_{-1} = 1$, the bounds imply $|\tilde{p}_i| \leq NB^{2\mu+1} + 1$ with denominator $\leq L < N^{8\mu} \cdot (2\mu)^\lambda$. By telescoping (splitting $p_i^* = n_i/k_i + m_i\alpha/k_i$, noting $m_{-1} = m_{N-1} = 0$), $\sum_i \tilde{p}_i \alpha^i = \sum_i p_i^* \alpha^i$, so the commitment is correctly opened.

Step 5: Coefficient of α^{N-1} (correct evaluation). We now show the extracted polynomial evaluates correctly. On the LHS, the coefficient of α^{N-1} is:

$$\begin{aligned} & \left\lfloor \frac{e_{\mathbf{p}, \alpha}^* \cdot e_{\mathbf{y}}}{\alpha^{N-1}} \right\rfloor \bmod \alpha \\ &= \left\lfloor \frac{\sum_{i+j < N-1} p_i^* \cdot y_{N-j-1} \cdot \alpha^{i+j}}{\alpha^{N-1}} + \sum_{i+j=N-1} p_i^* \cdot y_{N-j-1} \right\rfloor \bmod \alpha \\ &= \left\lfloor \frac{\sum_{i+j < N-2} p_i^* \cdot y_{N-j-1} \cdot \alpha^{i+j}}{\alpha^{N-1}} + \frac{\sum_{i=0}^{N-2} p_i^* \cdot y_{i+1}}{\alpha} \right\rfloor + \sum_{i=0}^{N-1} p_i^* \cdot y_i \bmod \alpha \\ &= \left\lfloor \frac{e + l_1^* - \lfloor e \rfloor}{\alpha} \right\rfloor + m + \sum_{i=0}^{N-1} p_i^* \cdot y_i \bmod \alpha \end{aligned}$$

where the second-to-last equality separates the sum at $i + j = N - 2$ (using (6)), and the last uses (7). Since $0 \leq e + l_1^* - \lfloor e \rfloor < \alpha$ (by the bound on l_1^*), the floor term vanishes. We then expand using the rational representation:

$$\begin{aligned} \text{LHS coeff of } \alpha^{N-1} &= \sum_{i=0}^{N-1} p_i^* \cdot y_i + m \bmod \alpha \\ &= \sum_{i=0}^{N-1} \left(\frac{m_i \alpha + n_i}{k_i} \right) y_i + \sum_{i=0}^{N-2} \frac{m_i}{k_i} y_{i+1} \bmod \alpha \\ &= \sum_{i=0}^{N-1} \left(\frac{n_i}{k_i} + \frac{m_{i-1}}{k_{i-1}} \right) y_i \bmod \alpha \\ &= \sum_{i=0}^{N-1} \tilde{p}_i \cdot y_i \bmod \alpha \end{aligned}$$

where we define $m_{-1} = 0, k_{-1} = 1$, and the penultimate equality follows from (7) (giving $m = \sum_{i=0}^{N-2} \frac{m_i}{k_i} y_{i+1} \bmod \alpha$). On the RHS, this coefficient equals m , so $\sum_{i=0}^{N-1} \tilde{p}_i \cdot y_i = m \bmod \alpha$.

By an identical argument, we also get the same extracted polynomial with β , implying that $p^*(\mathbf{y}) \equiv z \pmod{\beta}$. Since $q|\beta$, we have $p^*(\mathbf{y}) \equiv z \pmod{q}$. However, q was chosen after the polynomial was committed to, implying that $p^*(\mathbf{y}) = z$ over the rationals.

Efficiency considerations. We justify the running times for the prover, verifier and proof size as given in the theorem.

The prover time is dominated by the running time of the sub-protocols Masdak and Range which are called twice each. The other operations done by the prover include the three way decomposition as well as the integer encoding and multiplication of $e_{p,\alpha}^* \cdot e_y$ – as these integers are of size α^N , these operations can cost up to $O(\log \alpha^N)$ bit operations, leading to a cost of $O(N \log N)$ bit operations. Group operation costs include committing to various integers (in step 5), which varies depending on the commitment scheme used.

Although in Construction 4, for simplicity, we specify that the verifier explicitly computes e_y from the y , this would lead to a linear verifier. In the actual protocol, we only need to compute this value modulo a random prime used in Masdak – this can be done easily since the expansion has a specific form that allows the verifier to compute it in logarithmic time.

Specifically, we notice that $e_y \bmod \ell$ has a succinct representation

$$\prod_{i=1}^{\mu} (1 + y_{\mu-i} \cdot \alpha^{2^{i-1}}) \bmod \ell$$

allowing the verifier to compute this product of μ terms in $O(\mu)$ time, rather than $O(2^\mu)$. The rest of the verifier time is dominated by the Masdak and Range verifiers.

The proof size is a constant number of group elements followed by the sizes of the Masdak and Range proof.

□

C Reductions (and Arguments) of Knowledge from Gen-PIOR + Gen-PCS

In this section, we present a blueprint for constructing reductions of knowledge (RoK, Definition 3.16) for NP relations using a generalization of the popular PIOP + PC paradigm, which is Gen-PIOR (see Definition 3.4) + Gen-PCS (see Definition 3.22). Since arguments of knowledge (ARK, Definition 3.21) are a special case of RoKs, this compiler can also be used to build ARKs

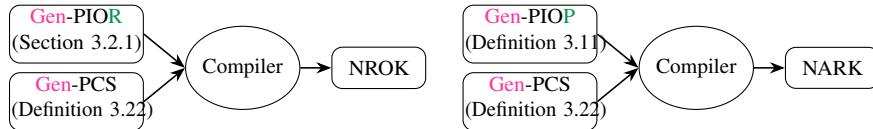


Figure 3: Description of the reduction of knowledge from Gen-PIOR + Gen-PCS.

The high-level idea is that the RoK prover and verifier internally simulate the Gen-PIOR prover and verifier. They use a Gen-PCS to materialize the prover’s oracle messages and then rely on the Gen-PCS evaluation argument to answer the verifier’s queries. The key difference from the standard PIOP + PC paradigm is that, while the prover’s messages are committed in the ambient ring $\mathbb{R}$, all subsequent messages and checks are then interpreted over the evaluation ring $\mathbb{S}$ via a randomly sampled evaluation map.

Construction 7: Reduction of Knowledge from Gen-PIOR + Gen-PCS

$pp \leftarrow \mathcal{S}(1^\lambda, N)$

1. Parse the security parameter λ and the size bound N .

2. Use the size bound N to derive the size bound N' for the polynomial commitment scheme.
3. Invoke $(ck_{PCS}, vk_{PCS}) \leftarrow \text{Gen-PCS.Setup}(1^\lambda, N)$ to obtain the commitment and verification key for the Gen-PCS.
4. Output $pp := (ck_{PCS}, vk_{PCS})$.

$(pk, vk, i') \leftarrow \mathcal{I}(pp, i)$

1. Invoke the Gen-PIOR indexer $\mathcal{I}$ on i to obtain (ι, l, i') where i' is the derived index for the target relation, and ι and l are the short and implicit indices for the Gen-PIOR, respectively. More Specifically, l includes the round-0 oracles: $\Pi_0 = \{p_{0,j}\}_{j=1}^{n_\Pi(0)} \subseteq \mathbb{R}_{v(0,j)}^{\text{ind} \leq 1}$
2. Next, commit to the index polynomials using the Gen-PCS:

$$cm_0 \leftarrow \text{Gen-PCS.Commit}(ck_{PCS}, \Pi_0).$$

3. Set $pk := (ck_{PCS}, i)$ and $vk := (vk_{PCS}, \iota, cm_0)$, and output the derived index i' for the target relation.

$\langle \mathcal{P}(pk, i, x, w), \mathcal{V}(vk, x) \rangle$

1. **Commitment phase.** The RoK prover and verifier internally invoke the Gen-PIOR prover $\mathcal{P}$ on instance-witness pair (x, w) and Gen-PIOR verifier $\mathcal{V}$ on instance x . In the i -th round of the Gen-PIOR: The RoK verifier simulates $\mathcal{V}$ to obtain public random challenges ϱ_i , then forwards them to the RoK prover. The RoK prover forwards ϱ_i to the simulated $\mathcal{P}$ and obtains the polynomials $\Pi_i = \{p_{i,j}\}_{j \in [n_\Pi(i)]} \subseteq \mathbb{R}_\mu^{\text{ind} \leq 1}$. The RoK prover commits to all polynomials using the Gen-PCS:

$$cm_i \leftarrow \text{Gen-PCS.Commit}(ck_{PCS}, \Pi_i),$$

and sends this commitment to the verifier.

2. **Evaluation-map sampling.** After the interaction The RoK verifier samples a random evaluation map $\varphi : \mathbb{R} \rightarrow \mathbb{S}$ from the prescribed family Φ and sends its description to the prover. From this point onward, all PIOR messages and checks are interpreted over $\mathbb{S}$ via φ .
3. **Query phase.**
 - (a) The RoK verifier samples and sends query randomness ϱ_{k+1} .
 - (b) The RoK prover computes the query set Q using the Gen-PIOR verifier's query algorithm on $(x; \varrho_1, \dots, \varrho_{k+1})$.
 - (c) The RoK prover answers the queries by evaluating the committed polynomials and applying the evaluation map, obtaining $\nu := \varphi(\Pi(Q))$, and produces the corresponding Gen-PCS opening proof.
4. **PIOR Verification.** The verifier checks the opening proof using the Gen-PCS verification algorithm. If it accepts, the RoK verifier feeds ν into the simulated $\mathcal{V}$ as oracle answers and accepts if and only if $\mathcal{V}$ accepts.

$$(x', \perp) := \mathcal{V}^{\Pi, l, \nu}(\iota, x, \varrho)$$

5. **PCS Verification.** The verifier checks the opening proof using the Gen-PCS verification algorithm. If it accepts, the RoK verifier feeds ν into the simulated $\mathcal{V}$ as oracle answers and accepts if and only if $\mathcal{V}$ accepts.
6. **Outputs.** The prover outputs a new witness w' and the verifier outputs a new instance x' for the target relation.

Next, we provide a formal statement of the security and the asymptotics of Construction 8.

Theorem C.1. Let $\mathfrak{R}$ and $\mathfrak{R}'$ be indexed relations. Consider the following components:

- A generalized polynomial interactive oracle reduction (see Definition 3.4) of the form $\text{Gen-PIOR} = (\text{Algs}, \text{Eval-Conf}, \text{Metrics})$ for $\mathfrak{R}$ with round-by-round soundness error ϵ^{RBR} , where
 - $\text{Algs} = (\mathcal{I}, \mathcal{P}, \mathcal{V})$ are the indexer, prover, and verifier algorithms.
 - $\text{Eval-Conf} = (\mathbb{R}, \mathbb{S}, \Phi)$ specifies the ambient ring, evaluation ring, and evaluation map domain.
 - $\text{Metrics} = (k, n_\Pi, n_\varrho, s)$ are the round and query complexity parameters where $n := \sum_{i=1}^k n_\Pi(i)$.

- A generalized non-adaptive polynomial commitment scheme (see Definition 3.22) $\text{Gen-PCS} = (\text{Eval-Conf}, \text{Setup}, \text{Commit}, \text{Open}, \text{ROK}_{\text{Eval}})$ with an evaluation argument $\text{ROK}_{\text{Eval}} = (\mathcal{P}_{\text{Eval}}, \mathcal{V}_{\text{Eval}})$. Then, Construction 8 is a reduction of knowledge (see Definition 3.16) ROK from $\mathfrak{R}$ to $\mathfrak{R}'$ with the following time complexities:

Setup	Indexer	prover time	verifier time
$T(\text{Setup})$	$T(\mathcal{I}) + n_{\Pi}(0)T(\text{Commit})$	$T(\mathcal{P}) + nT(\text{Commit}) + T(\mathcal{P}_{\text{Eval}})$	$T(\mathcal{V}) + T(\mathcal{V}_{\text{Eval}})$

and the following size complexities:

Public parameter	Proving Key	Verifying Key	proof size
$ \text{pp} $	$ \text{ck}_{\text{PCS}} + n_{\Pi}(0)d(0)$	$ \text{vk}_{\text{PCS}} + n_{\Pi}(0) \text{cm} $	$n \text{cm} + s \mathbb{S} + \pi_{\text{Eval}} $